

TRAILHEAD
TECHNOLOGY PARTNERS

Pragmatic AI in .NET

10 AI Features Users Want in Your .NET Apps



Jonathan "J." Tower

Saturday. 5:58 a.m.



Trailhead Guides

Search trails, parks, or ask a question

Explore Trips Ask Saved

Explore › Glacier National Park › Avalanche Lake Trail



Avalanche Lake Trail

Glacier National Park · 4.6 mi · Moderate · waterfall, lake, gorge, family-friendly

Conditions this week

AI SUMMARY

From 40 recent trip reports

- **Trail to Avalanche Lake is closed: the footbridge over the creek at the gorge washed out.**
- No other closures or hazards reported, including the Trail of the Cedars boardwalk.
- Moderate crowding at the trailhead and along Going-to-the-Sun Road.

Elevation gain	Trip reports this season	Latest report
730 ft	40	2 days ago

You might also like

Trails that feel the same, from what hikers wrote

Gunsight Lake Approach Hard
Glacier National Park · 12.4 mi

Black Lake via Glacier Gorge Hard
Rocky Mountain NP · 10.4 mi

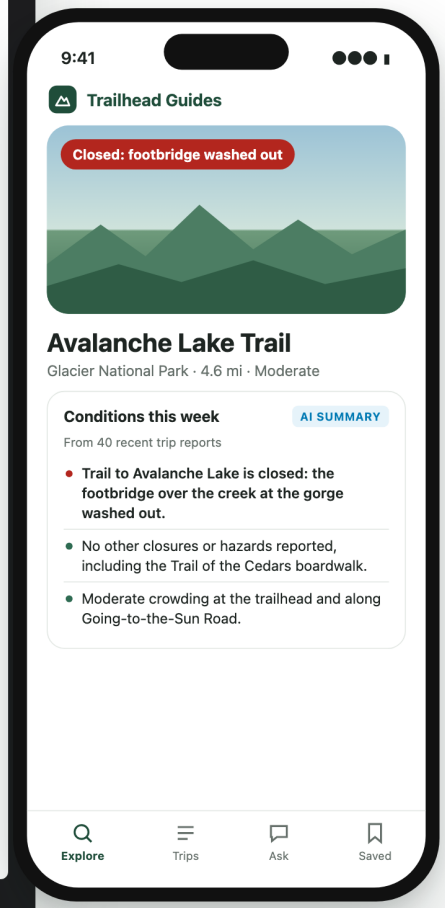
Piegan Pass Trail Hard
Glacier National Park · 9.0 mi

Plan a trip

AI AGENT

3 days in Glacier, mid-September

- Day 1: Highline Trail, before the rain
- Day 2: Trail of the Cedars, short and sheltered
- **Avalanche Lake skipped: bridge out**





Avalanche Lake Trail.

40 trip reports.





She only read 3.

"The footbridge at the creek is gone. We had to turn around."

Report #14 of 40





Mile 2 is where she found out.

The answer was there, but it was buried in a mountain of text.



Ten **shippable** AI features that solve problems your users **already have**, and the judgment to know **when to build each**.

TRAILHEAD
TECHNOLOGY PARTNERS

What We'll Talk About



F01 Summarize



F02 Extract



F03 Sentiment



F04 Search



F05 RAG



F06 Recommend



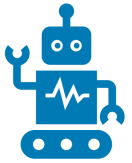
F07 Route



F08 Detect



F09 Approve

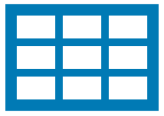


F10 Agent

TRAILHEAD
TECHNOLOGY PARTNERS

What We'll Talk About

F01 Summarize



F02 Extract

F03 Sentiment

F04 Search



F05 RAG

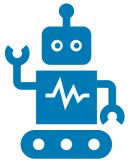
F06 Recommend



F07 Route

F08 Detect

F09 Approve



F10 Agent

TRAILHEAD
TECHNOLOGY PARTNERS

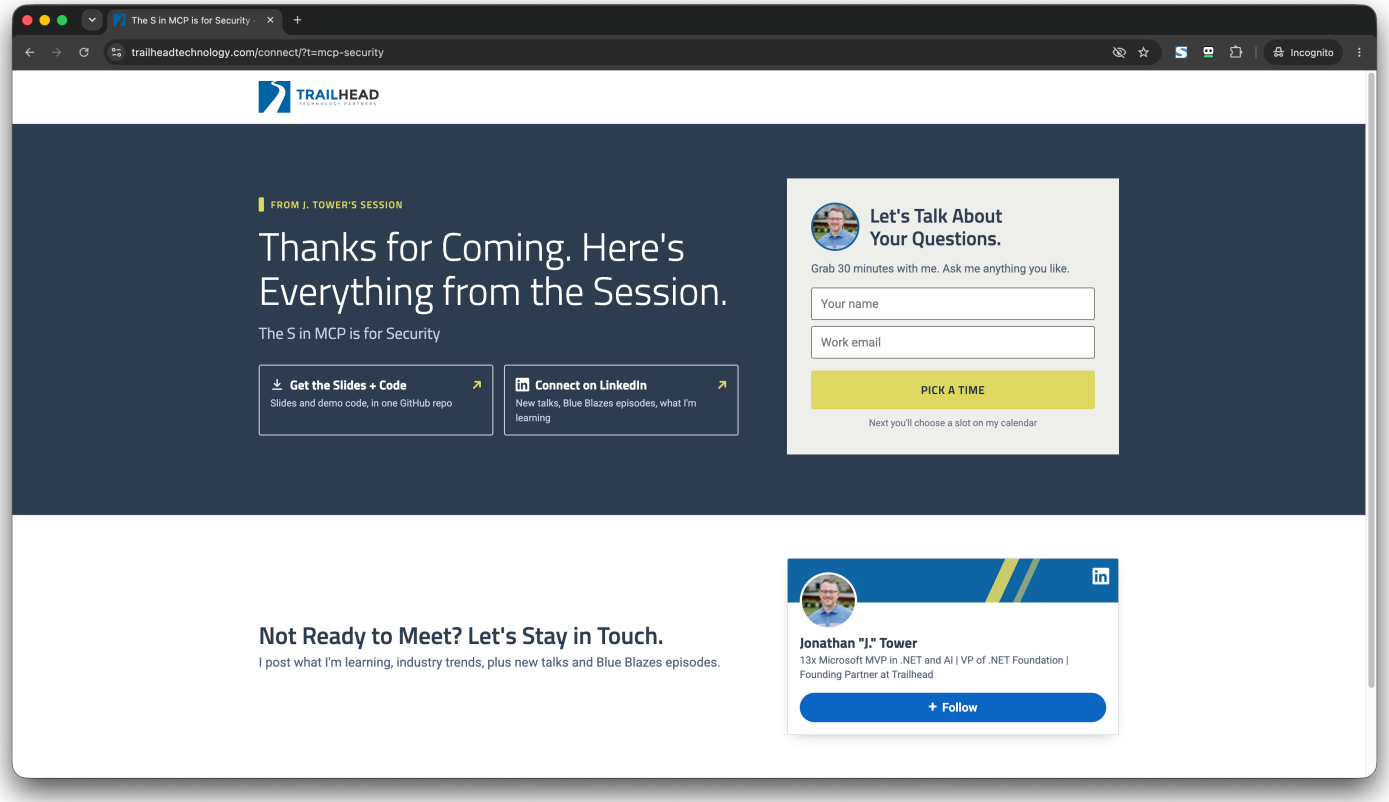
Jonathan "J." Tower



TRAILHEAD
TECHNOLOGY PARTNERS

Founding Partner

- 13x Microsoft MVP in .NET & AI
- VP of .NET Foundation
- Blue Blazes podcast host


The S in MCP is for Security
trailheadtechnology.com/connect/?t=mcp-security
Incognito
TRAILHEAD
FROM J. TOWER'S SESSION
Thanks for Coming. Here's Everything from the Session.
The S in MCP is for Security
Get the Slides + Code
Slides and demo code, in one GitHub repo
Connect on LinkedIn
New talks, Blue Blazes episodes, what I'm learning
Let's Talk About Your Questions.
Grab 30 minutes with me. Ask me anything you like.
Your name
Work email
PICK A TIME
Next you'll choose a slot on my calendar
Not Ready to Meet? Let's Stay in Touch.
I post what I'm learning, industry trends, plus new talks and Blue Blazes episodes.
Jonathan "J." Tower
13x Microsoft MVP in .NET and AI | VP of .NET Foundation | Founding Partner at Trailhead
+ Follow

The Wrong Question and the Better One

❌ "Where can we use AI?"

The Wrong Question and the Better One

❌ "Where can we use AI?"

✅ "What problems can AI *best* solve for our users?"

Local Models, Cloud When Needed

Local Models (~5 GB, no GPU):

- 💻 `llama3.2`
- 💻 `phi3`
- 💻 `nomic-embed-text`

Cloud Model (my subscription):

- ☁️ Cloud models where it helps: F03, F05, F10

Every Model Is an HTTP Call

`localhost:11434` → `/api/chat` · `/api/embed`

Every Model Is an HTTP Call

`localhost:11434` → `/api/chat` · `/api/embed`

`…openai.azure.com/…/chat/completions` + `api-key`

Every Model Is an HTTP Call

`localhost:11434` → `/api/chat` · `/api/embed`

`…openai.azure.com/…/chat/completions` + `api-key`

Same request shape · a URL and a key apart

One Interface, Local or Cloud

```
// Ollama on this laptop: free, private, small.
public static IChatClient Local(string model = "llama3.2") =>
    new OllamaApiClient(new Uri("http://localhost:11434"), model);

// Azure OpenAI on Foundry: same interface, different constructor.
public static IChatClient Cloud(string deployment) =>
    new AzureOpenAIClient(new Uri(Endpoint), new ApiKeyCredential(Key))
        .GetChatClient(deployment)
        .AsIChatClient();

// Embeddings stay local in every demo.
public static IEmbeddingGenerator<string, Embedding<float>> Embedder() =>
    new OllamaApiClient(new Uri("http://localhost:11434"), "nomic-embed-text");
```

One Interface, Local or Cloud

```
// Ollama on this laptop: free, private, small.
public static IChatClient Local(string model = "llama3.2") =>
    new OllamaApiClient(new Uri("http://localhost:11434"), model);

// Azure OpenAI on Foundry: same interface, different constructor.
public static IChatClient Cloud(string deployment) =>
    new AzureOpenAIClient(new Uri(Endpoint), new ApiKeyCredential(Key))
        .GetChatClient(deployment)
        .AsIChatClient();

// Embeddings stay local in every demo.
public static IEmbeddingGenerator<string, Embedding<float>> Embedder() =>
    new OllamaApiClient(new Uri("http://localhost:11434"), "nomic-embed-text");
```

One Interface, Local or Cloud

```
// Ollama on this laptop: free, private, small.
public static IChatClient Local(string model = "llama3.2") =>
    new OllamaApiClient(new Uri("http://localhost:11434"), model);

// Azure OpenAI on Foundry: same interface, different constructor.
public static IChatClient Cloud(string deployment) =>
    new AzureOpenAIClient(new Uri(Endpoint), new ApiKeyCredential(Key))
        .GetChatClient(deployment)
        .AsIChatClient();

// Embeddings stay local in every demo.
public static IEmbeddingGenerator<string, Embedding<float>> Embedder() =>
    new OllamaApiClient(new Uri("http://localhost:11434"), "nomic-embed-text");
```

F01 · Summarization

TRAILHEAD
TECHNOLOGY PARTNERS



Turn a long document into the short version its reader needs.

TRAILHEAD
TECHNOLOGY PARTNERS

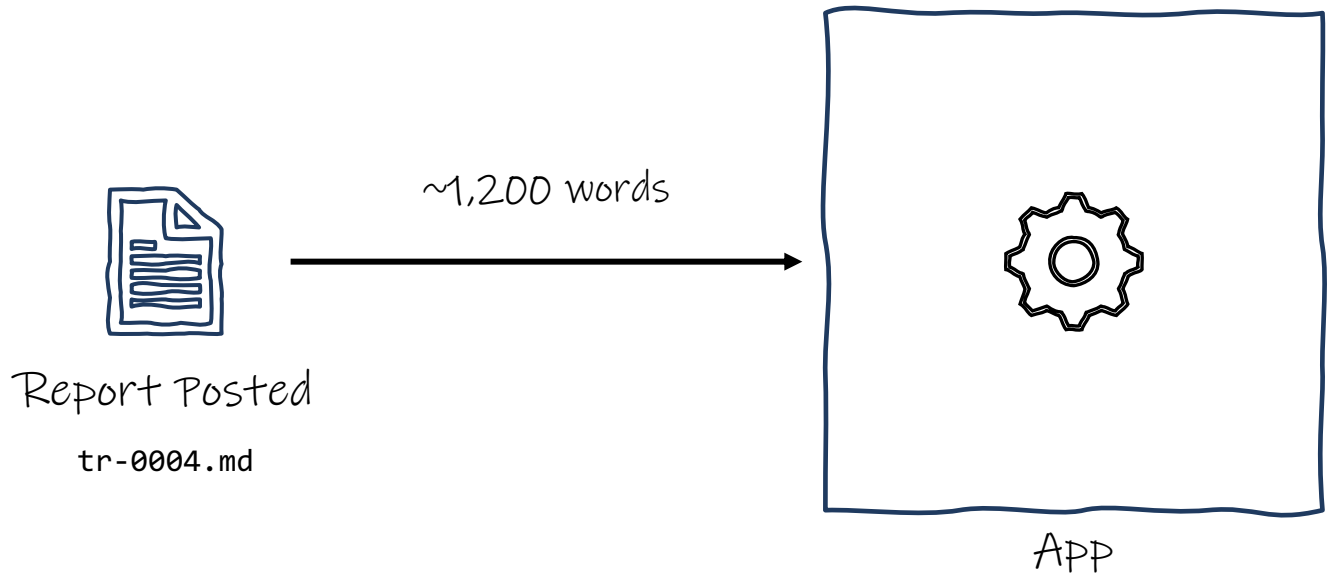
F01 · How It Works



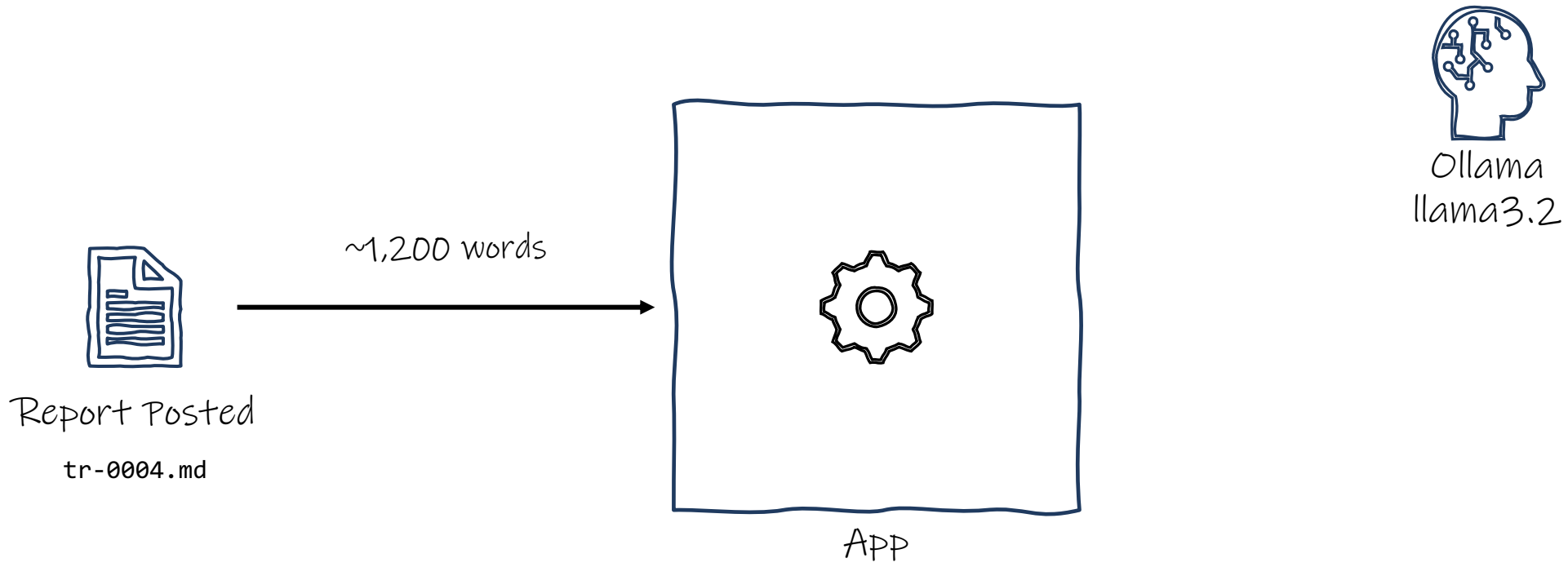
Report Posted

`tr-0004.md`

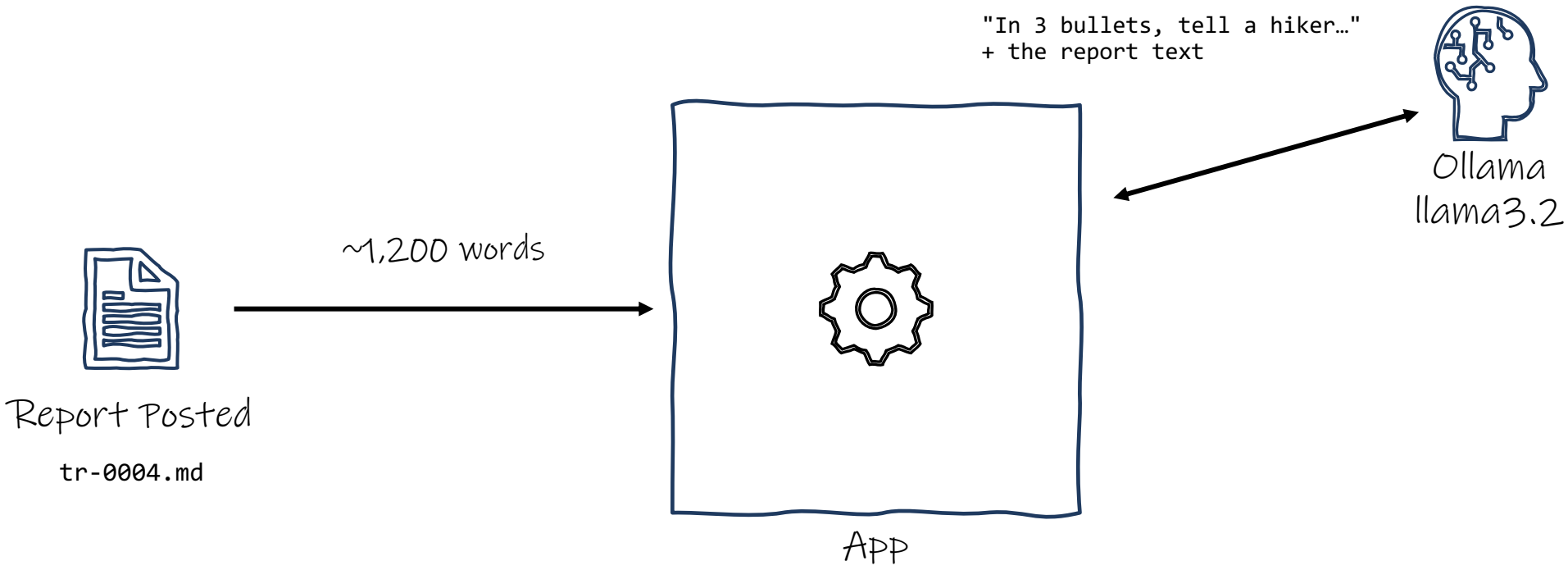
F01 · How It Works



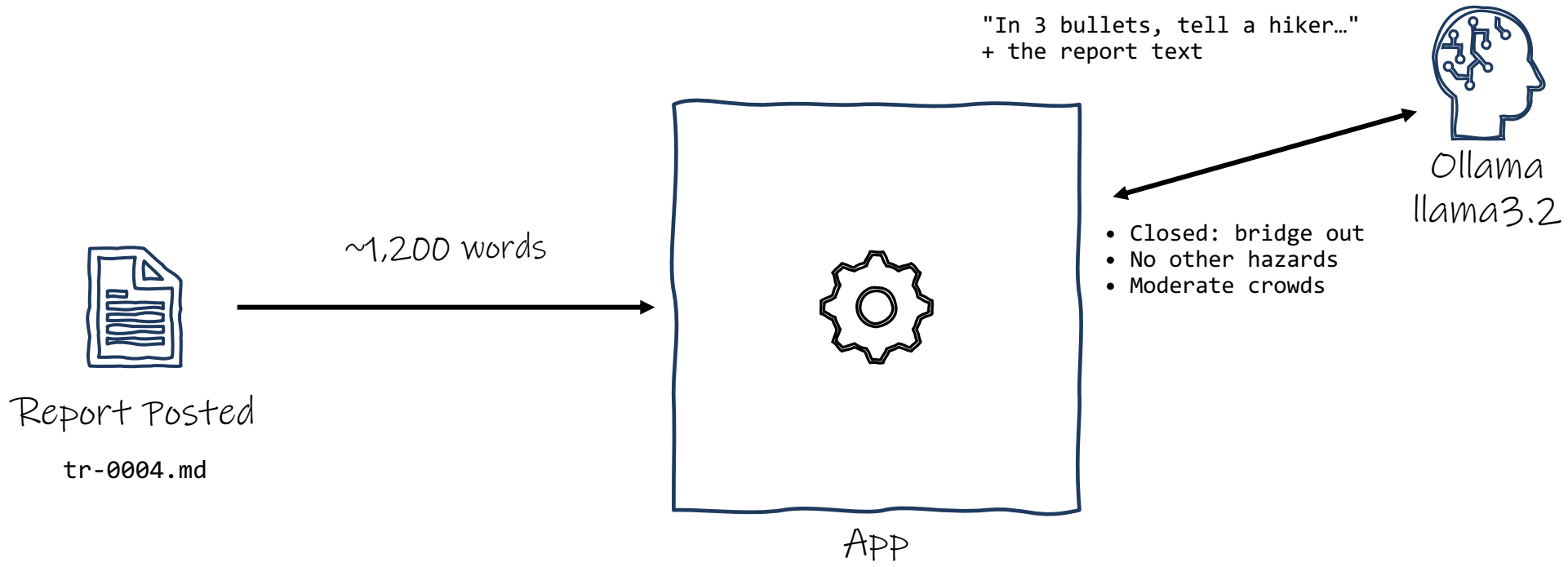
F01 · How It Works



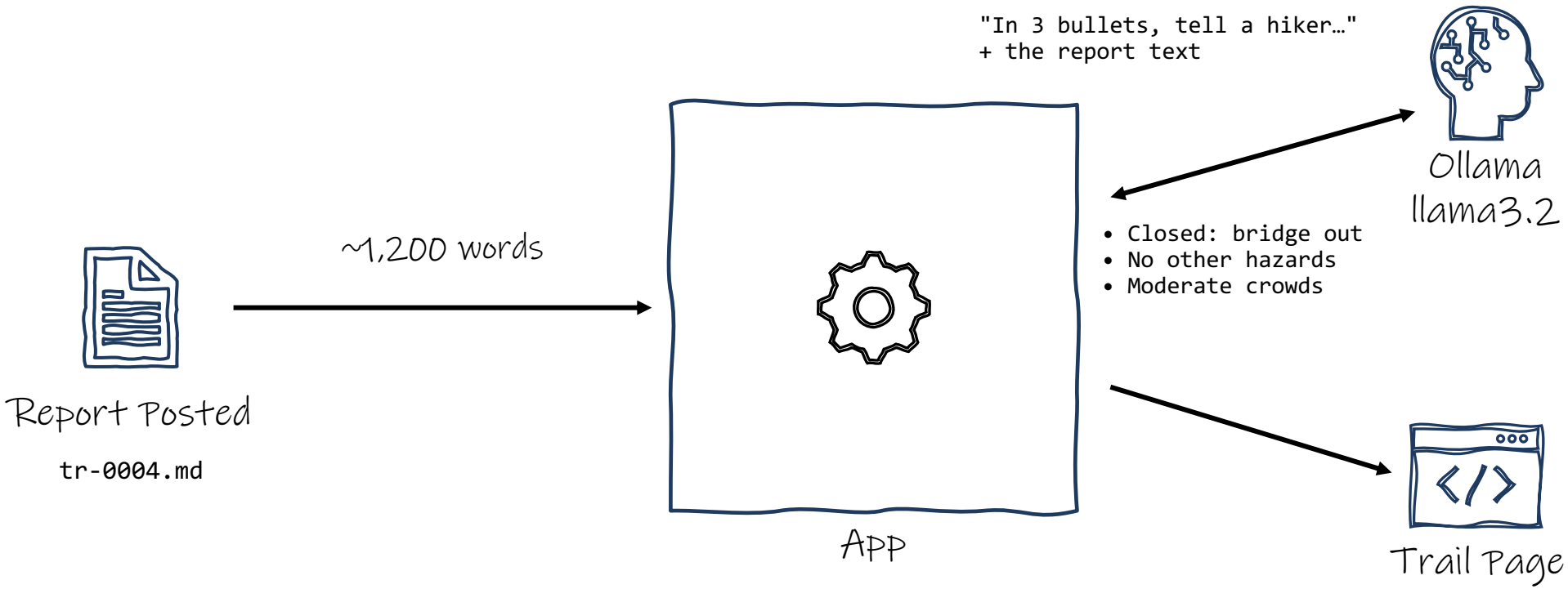
F01 · How It Works



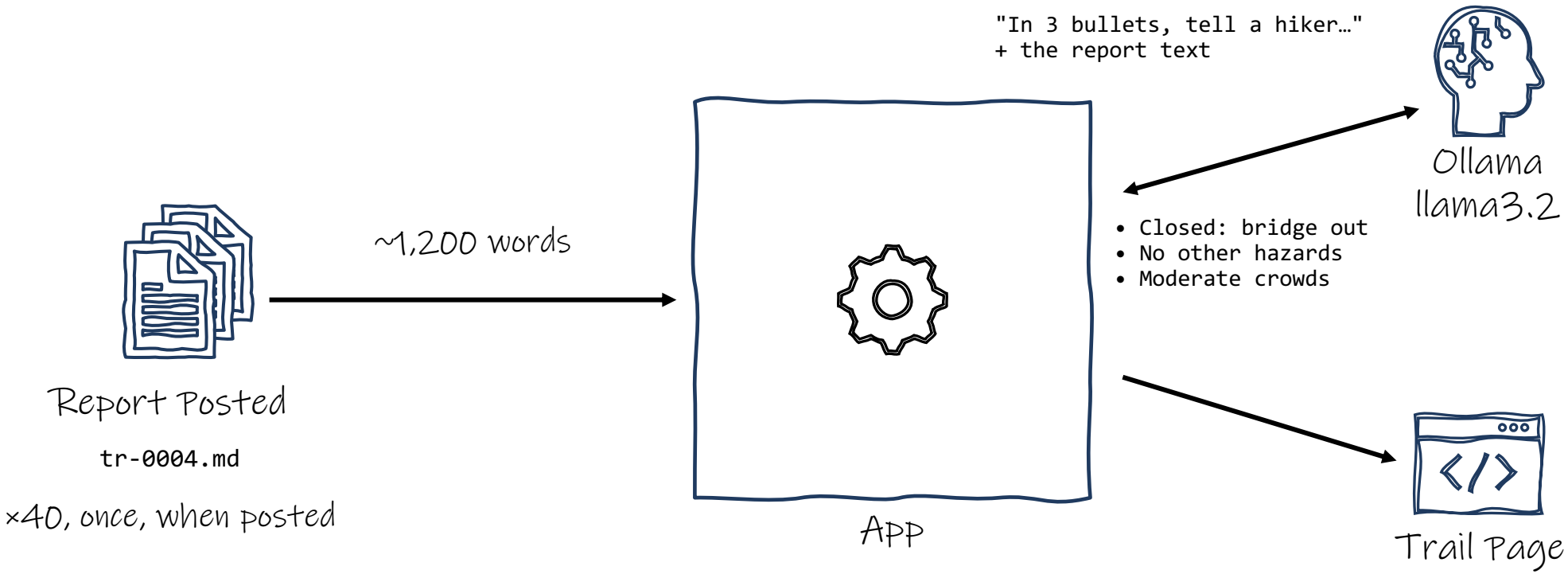
F01 · How It Works



F01 · How It Works



F01 · How It Works



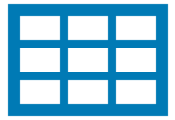


Use it when there's
too much text for anyone to read it all.

TRAILHEAD
TECHNOLOGY PARTNERS

F02 · Extraction

TRAILHEAD
TECHNOLOGY PARTNERS



Pull facts out of free-form text

Into a format your code can use.

TRAILHEAD
TECHNOLOGY PARTNERS

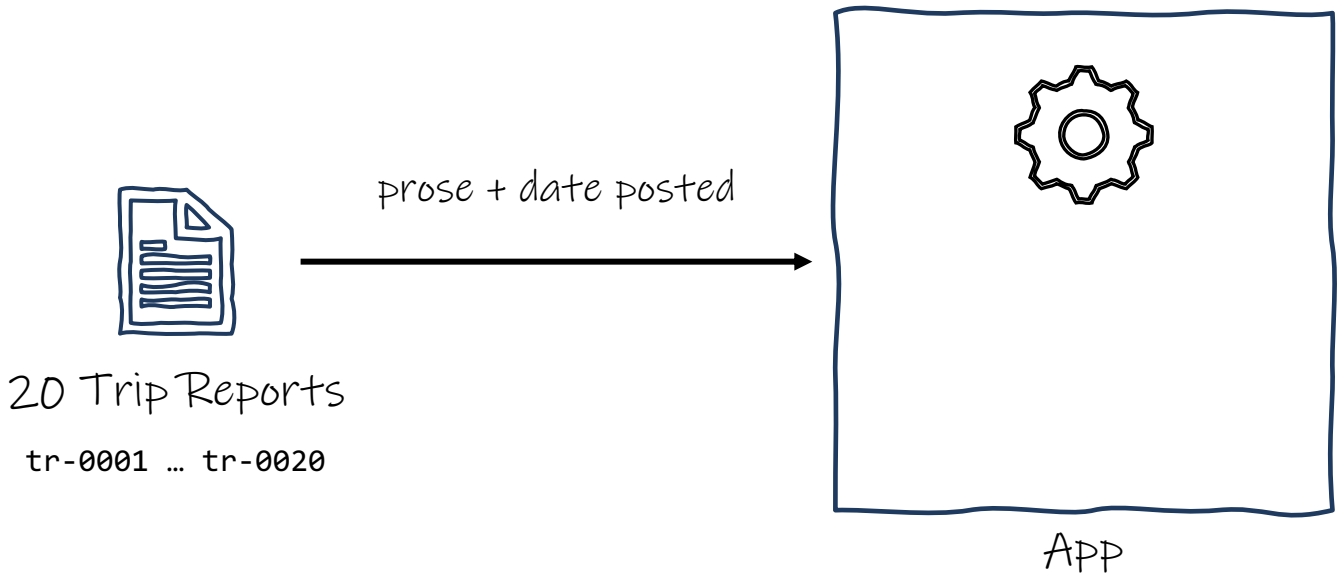
F02 · How It Works



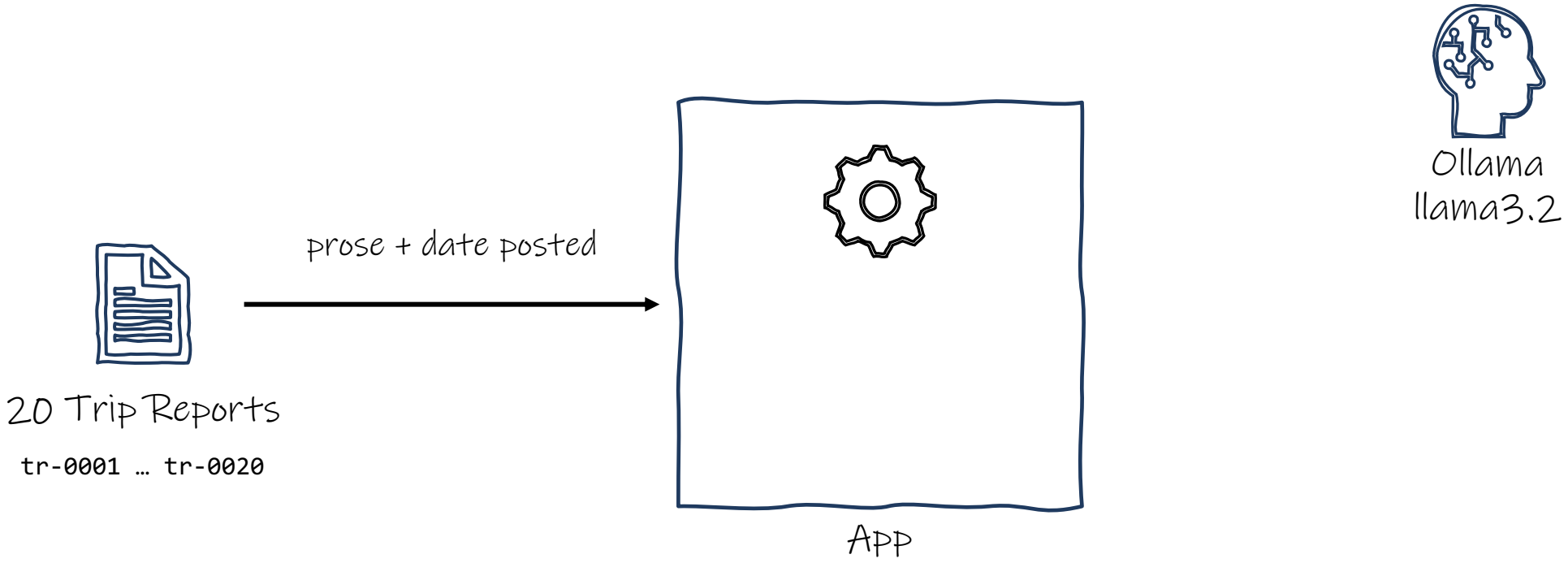
20 Trip Reports

`tr-0001 … tr-0020`

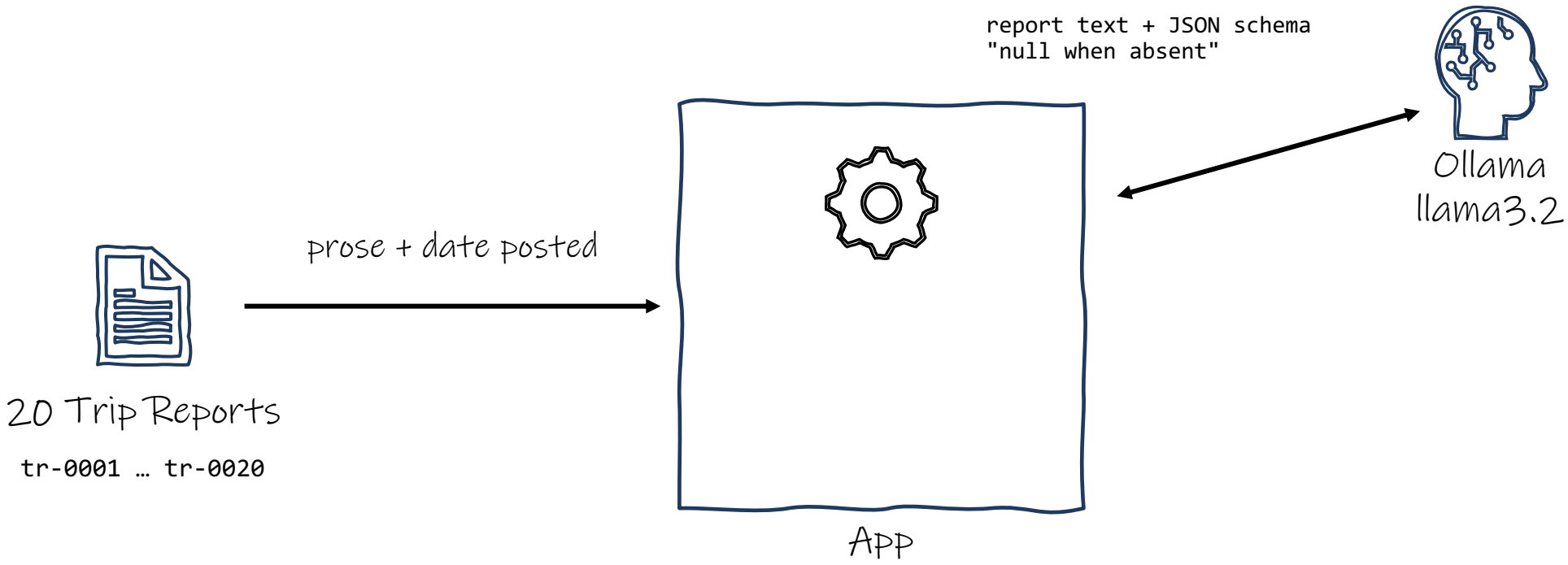
F02 · How It Works



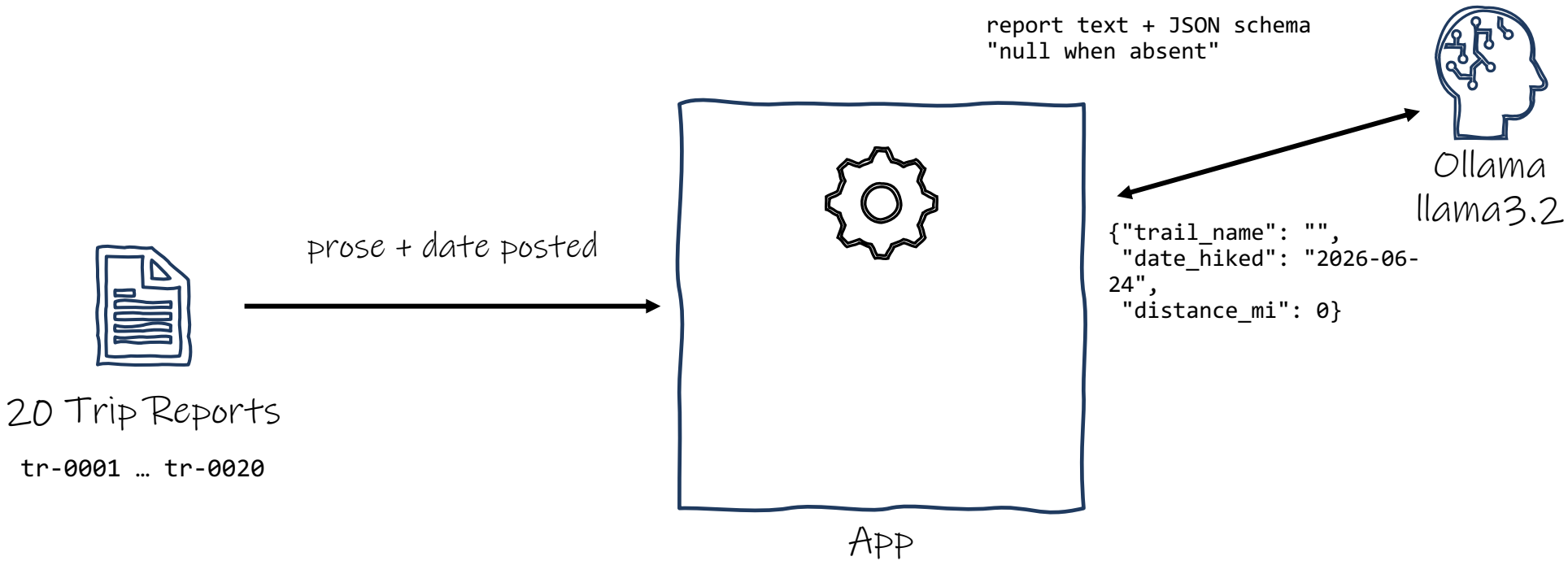
F02 · How It Works



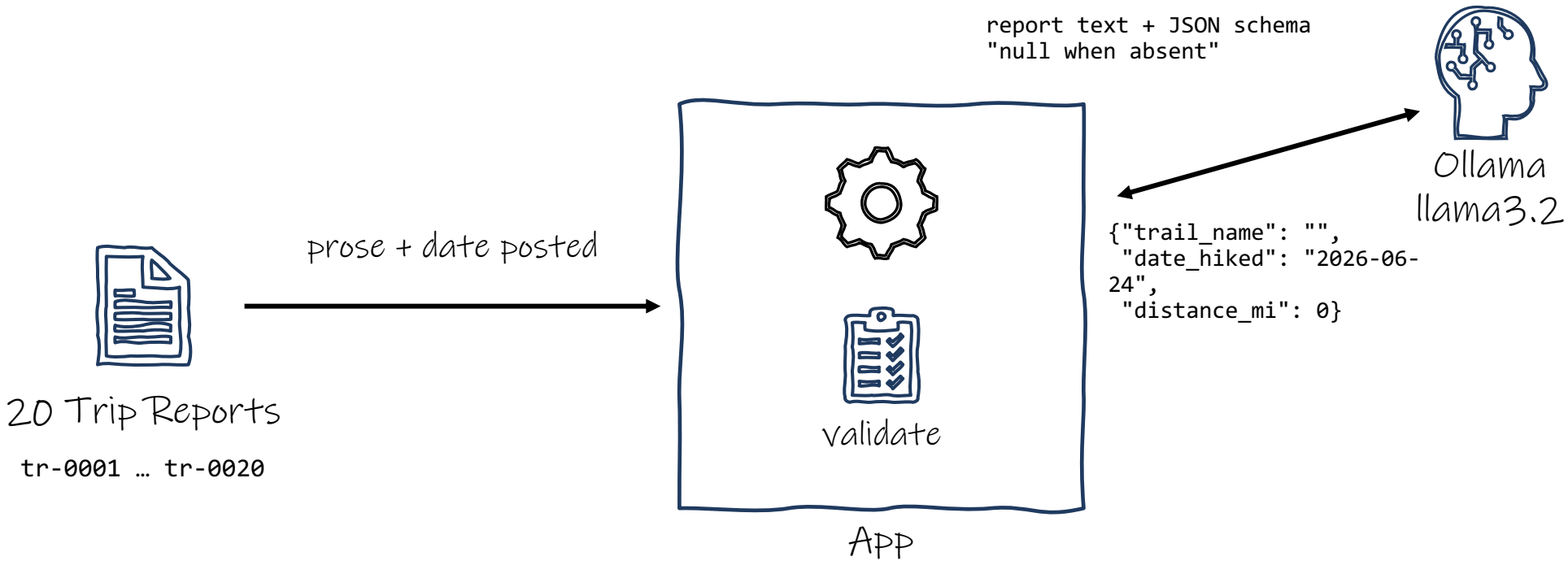
F02 · How It Works



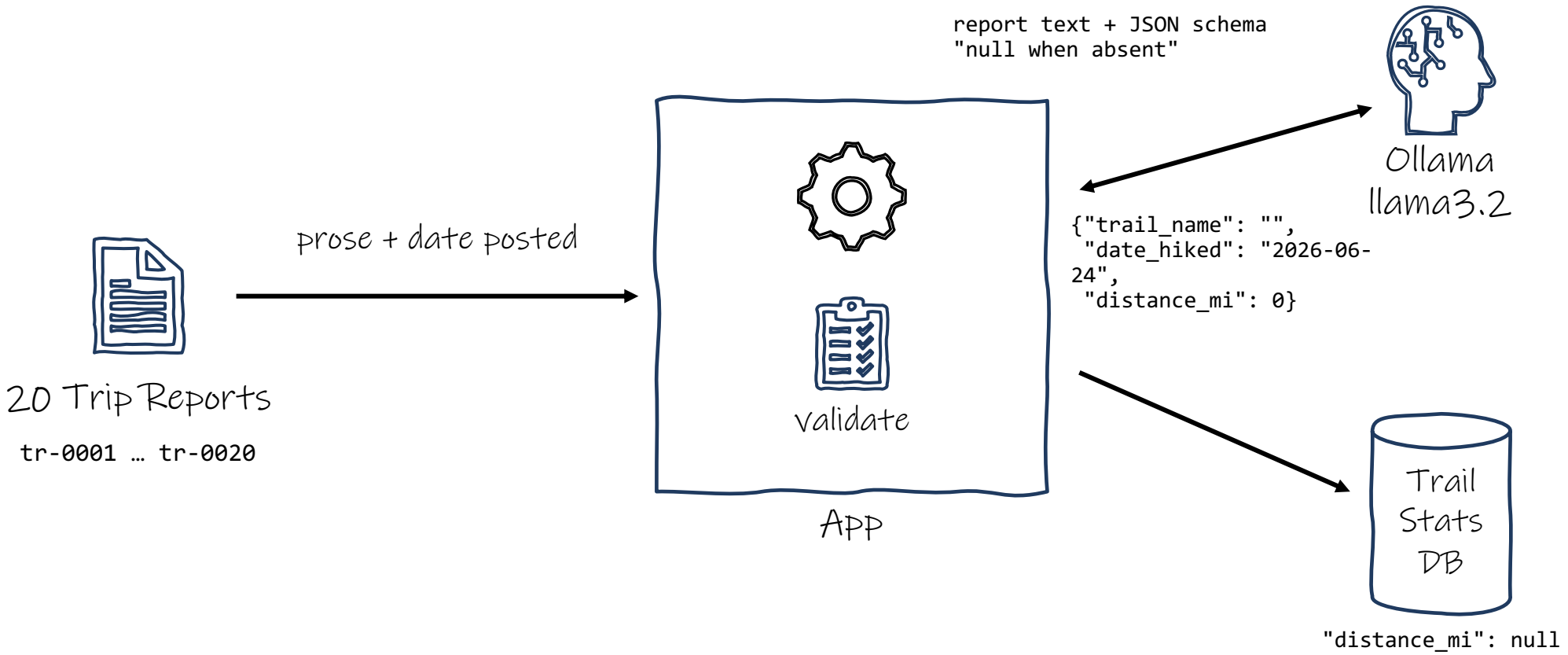
F02 · How It Works



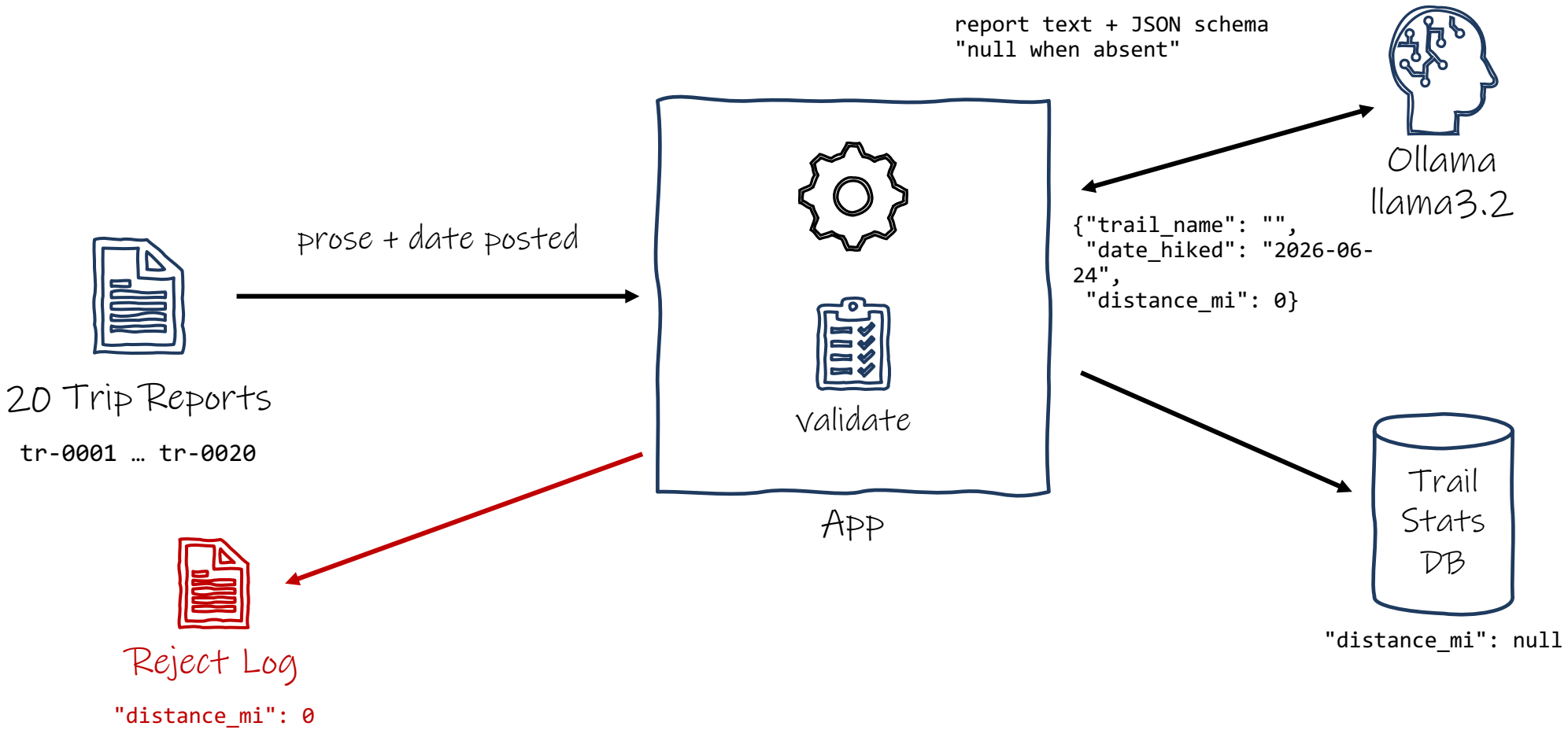
F02 · How It Works



F02 · How It Works



F02 · How It Works



F02 · The Call

```
var (report, posted, park) = await Load(file);

// Prose in, typed object out. The record is the schema.
var response = await client.GetResponseAsync<TripFacts>($"""
    Extract the trail facts from this trip report, posted {posted}.
    Use null for any field the report does not state,
    and empty arrays when nothing applies. Do not guess.

    {report}
    """, options);
TripFacts raw = response.Result;

// The schema guarantees it parses, not that it is true.
var verdicts = Validate(raw, report);
ShowRow(id, raw, verdicts);
```

F02 · The Call

```
var (report, posted, park) = await Load(file);

// Prose in, typed object out. The record is the schema.
var response = await client.GetResponseAsync<TripFacts>($"""
    Extract the trail facts from this trip report, posted {posted}.
    Use null for any field the report does not state,
    and empty arrays when nothing applies. Do not guess.

    {report}
    """, options);
TripFacts raw = response.Result;

// The schema guarantees it parses, not that it is true.
var verdicts = Validate(raw, report);
ShowRow(id, raw, verdicts);
```

F02 · The Schema

```
record TripFacts(
    string? TrailName,
    string? DateHiked,
    [property: Description("Round-trip distance in miles, as stated " +
        "in the report. null, never 0, if the report gives no " +
        "distance. Never estimate.")]
    double? DistanceMi,
    double? ElevationGainFt,
    string[] Wildlife,
    string[] Conditions,
    string[] Hazards);
```

F02 · The Schema

```
record TripFacts(
    string? TrailName,
    string? DateHiked,
    [property: Description("Round-trip distance in miles, as stated " +
        "in the report. null, never 0, if the report gives no " +
        "distance. Never estimate.")]
    double? DistanceMi,
    double? ElevationGainFt,
    string[] Wildlife,
    string[] Conditions,
    string[] Hazards);
```

F02 · The Validator

```
static List<Verdict> Validate(TripFacts f, string sourceText) =>
[
    // A name the source text never contains does not get to be a fact.
    Grounded("trail_name", f.TrailName, sourceText),
    ValidDate("date_hiked", f.DateHiked),

    // 0 is the dangerous near-miss: it is a value, so a pipeline stores it
    // and nothing downstream ever questions it. The honest answer is null.
    InRange("distance_mi", f.DistanceMi, max: 100, unit: "mi"),
    InRange("elevation_gain_ft", f.ElevationGainFt, max: 20000, unit: "ft"),
];
```

F02 · The Validator

```
static List<Verdict> Validate(TripFacts f, string sourceText) =>
[
    // A name the source text never contains does not get to be a fact.
    Grounded("trail_name", f.TrailName, sourceText),
    ValidDate("date_hiked", f.DateHiked),

    // 0 is the dangerous near-miss: it is a value, so a pipeline stores it
    // and nothing downstream ever questions it. The honest answer is null.
    InRange("distance_mi", f.DistanceMi, max: 100, unit: "mi"),
    InRange("elevation_gain_ft", f.ElevationGainFt, max: 20000, unit: "ft"),
];
```

F02 · The Validator

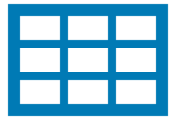
```
static List<Verdict> Validate(TripFacts f, string sourceText) =>
[
    // A name the source text never contains does not get to be a fact.
    Grounded("trail_name", f.TrailName, sourceText),
    ValidDate("date_hiked", f.DateHiked),

    // 0 is the dangerous near-miss: it is a value, so a pipeline stores it
    // and nothing downstream ever questions it. The honest answer is null.
    InRange("distance_mi", f.DistanceMi, max: 100, unit: "mi"),
    InRange("elevation_gain_ft", f.ElevationGainFt, max: 20000, unit: "ft"),
];
```

DEMO · F02 Extraction



TRAILHEAD
TECHNOLOGY PARTNERS



Use it when the data you need for an app is stuck in documents nobody wants to re-type.

TRAILHEAD
TECHNOLOGY PARTNERS

F03 · Sentiment

TRAILHEAD
TECHNOLOGY PARTNERS



Read text and judge the feeling behind it:
positive, negative, or mixed.

TRAILHEAD
TECHNOLOGY PARTNERS

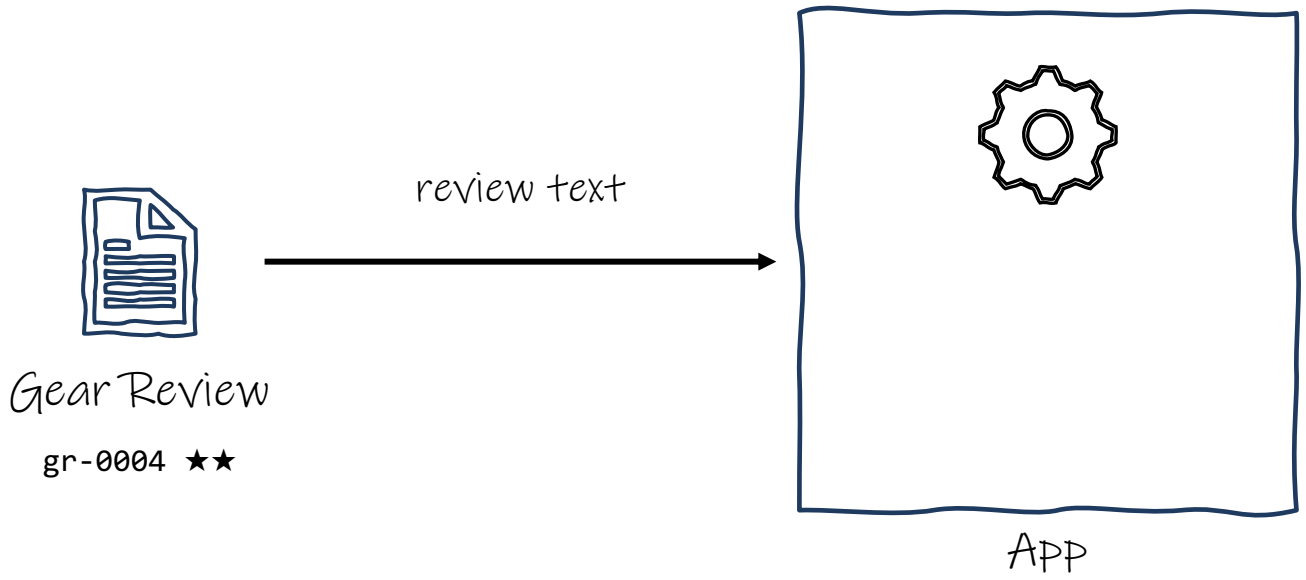
F03 · How It Works



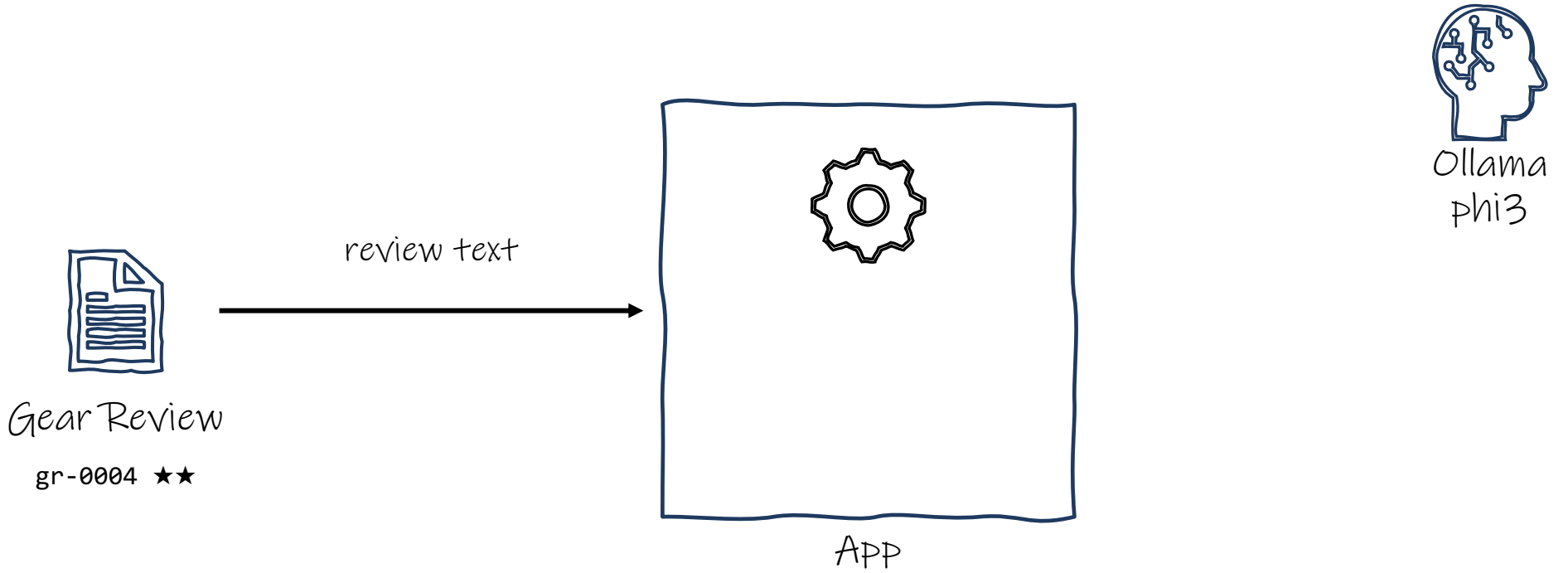
Gear Review

gr-0004 ★★

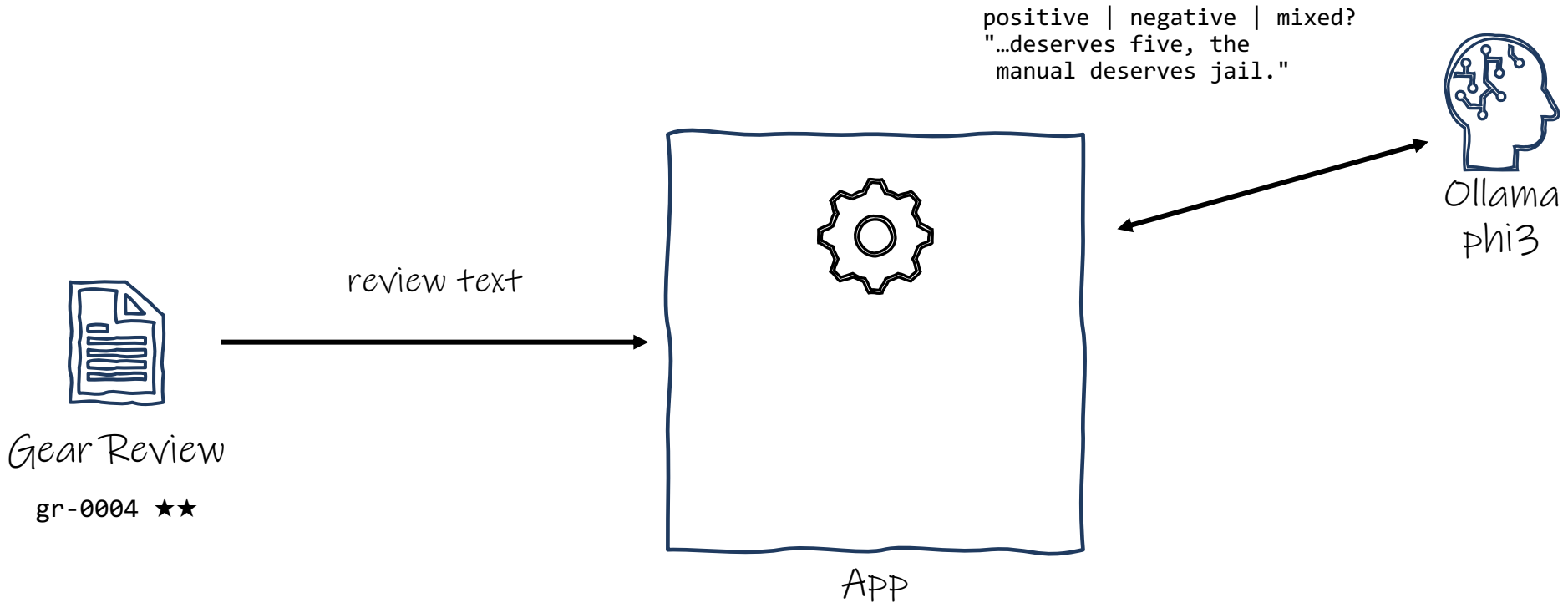
F03 · How It Works



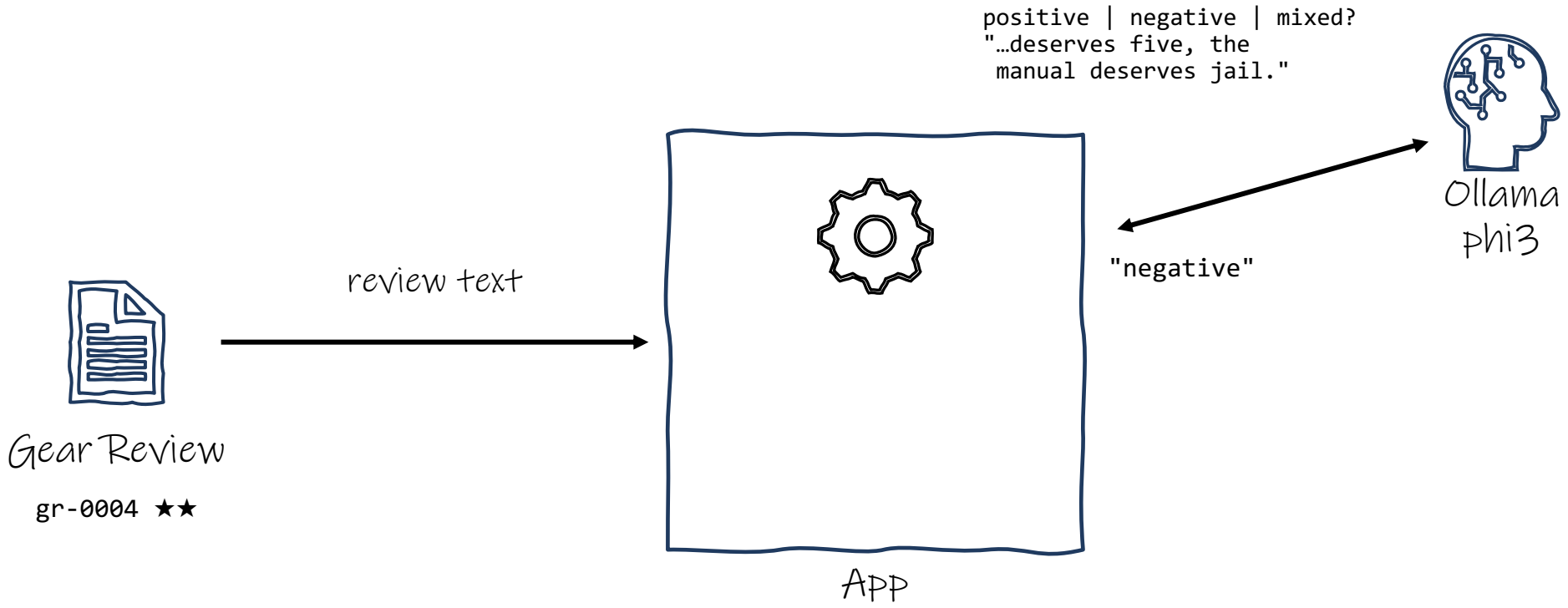
F03 · How It Works



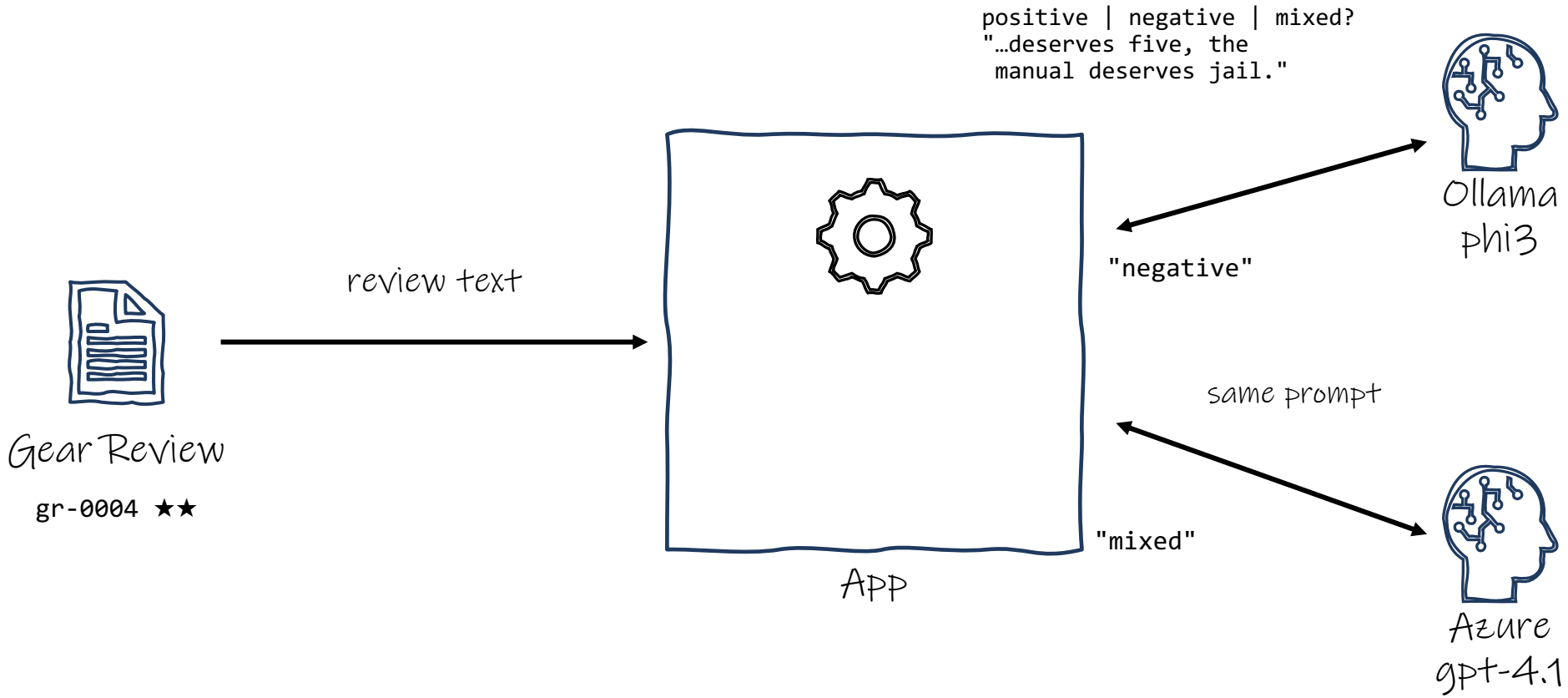
F03 · How It Works



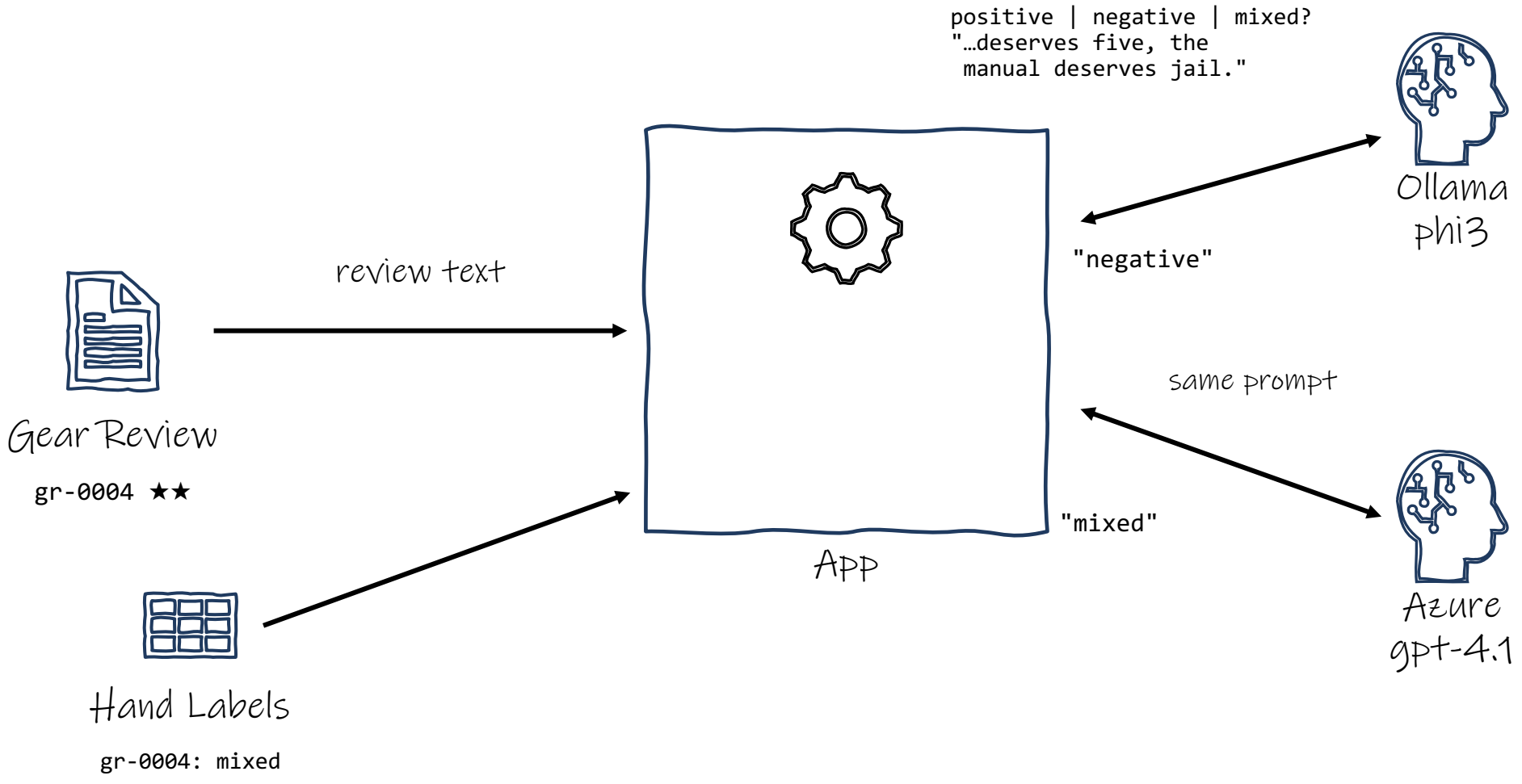
F03 · How It Works



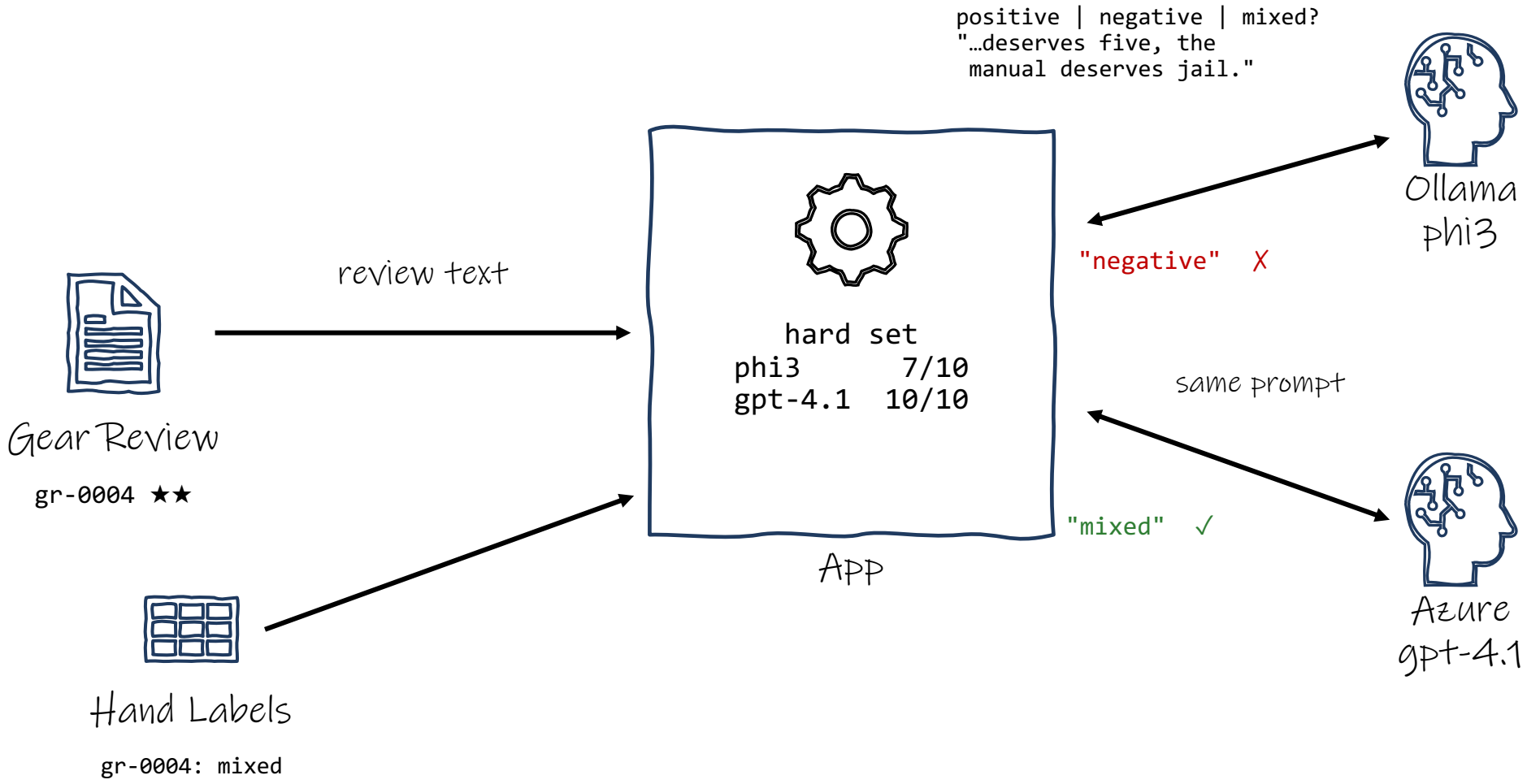
F03 · How It Works



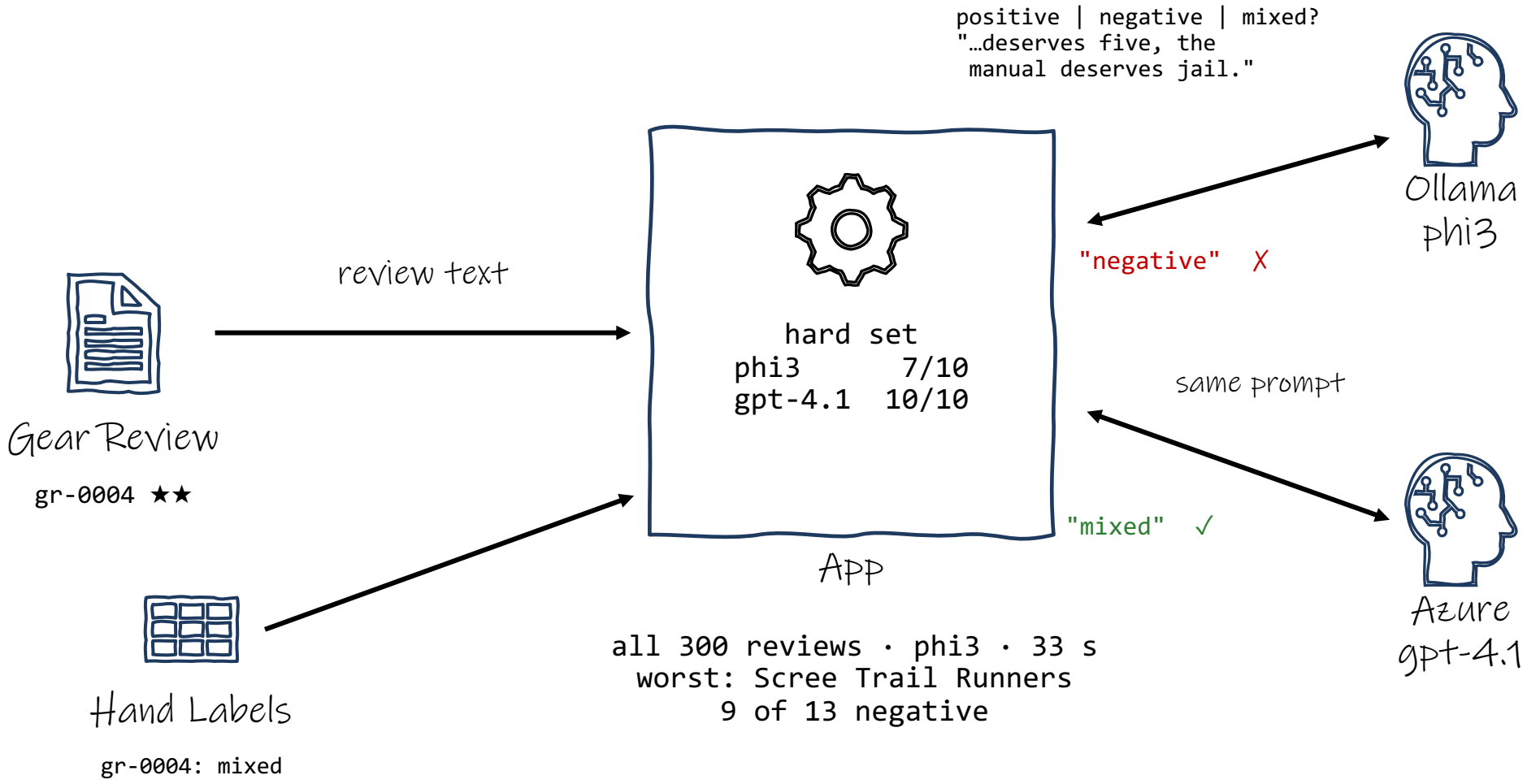
F03 · How It Works



F03 · How It Works



F03 · How It Works





Use it when a high-volume text stream needs a judgment call.

TRAILHEAD
TECHNOLOGY PARTNERS

F04 · Semantic Search

TRAILHEAD
TECHNOLOGY PARTNERS

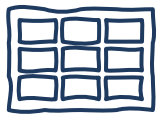


Search by what words mean,
not by which words appear.

TRAILHEAD
TECHNOLOGY PARTNERS

F04 · How It Works

1 · Index, once

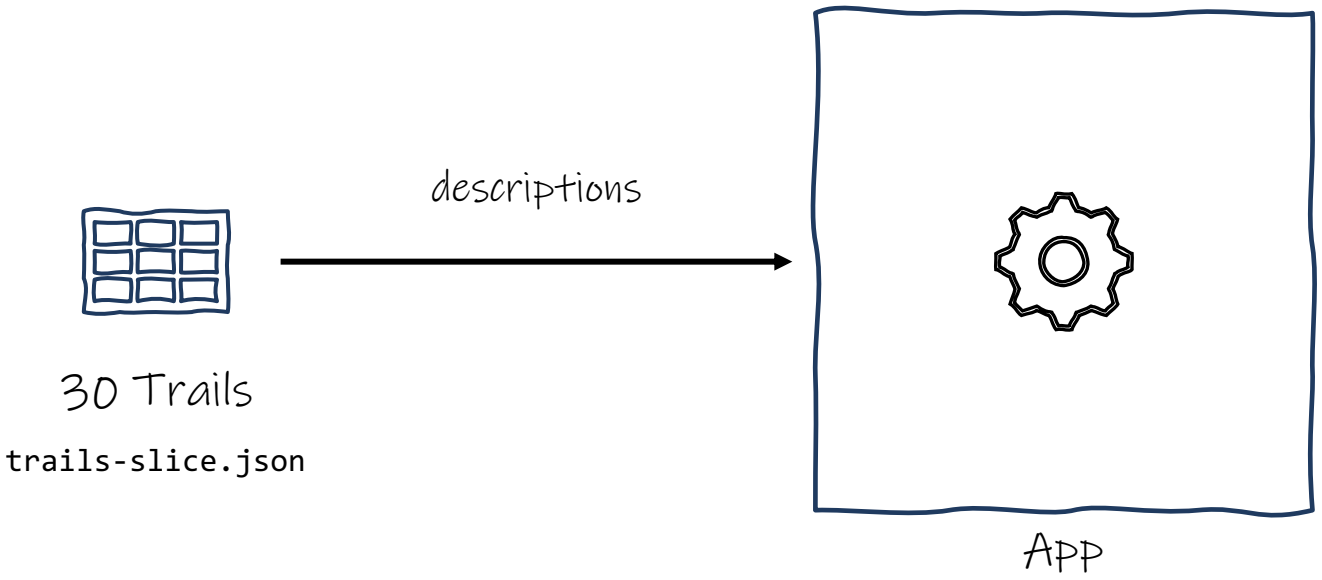


30 Trails

`trails-slice.json`

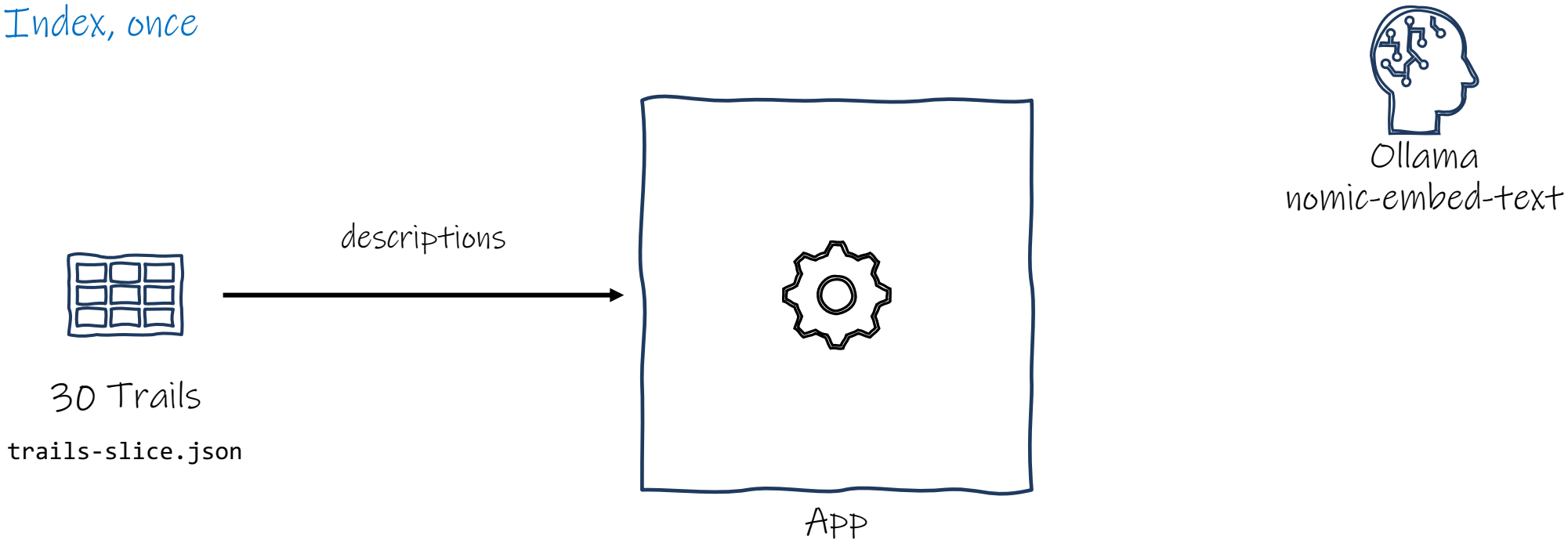
F04 · How It Works

1 · Index, once



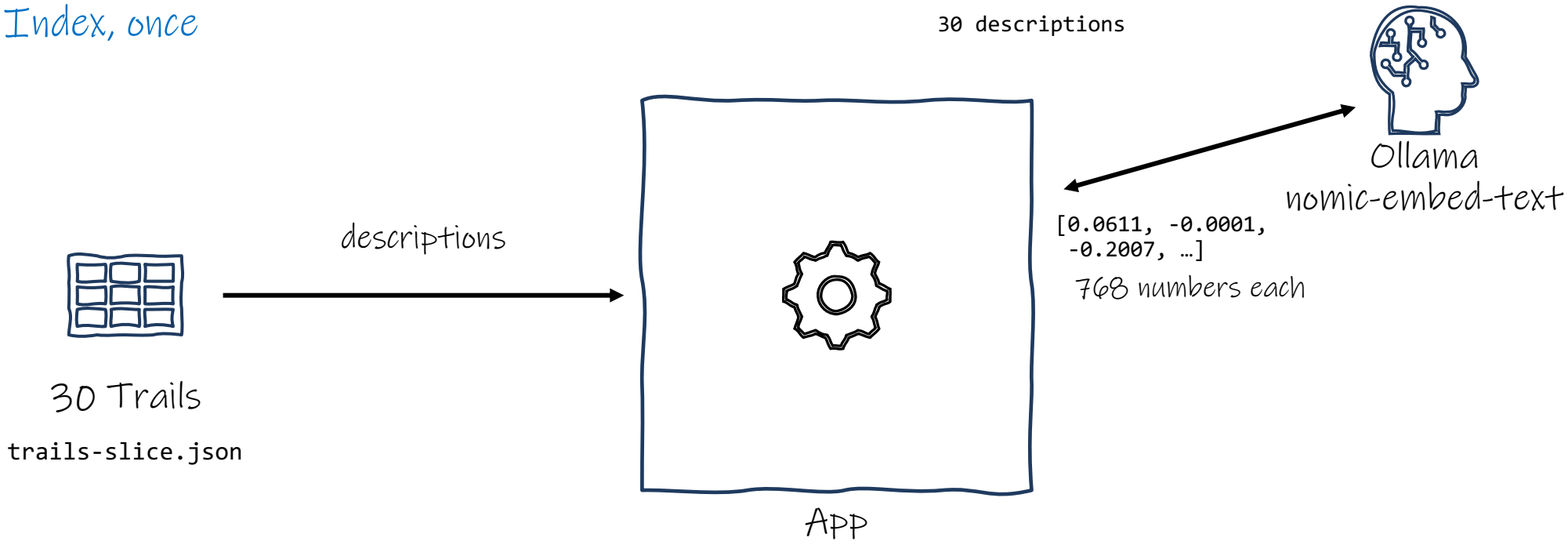
F04 · How It Works

1 · Index, once



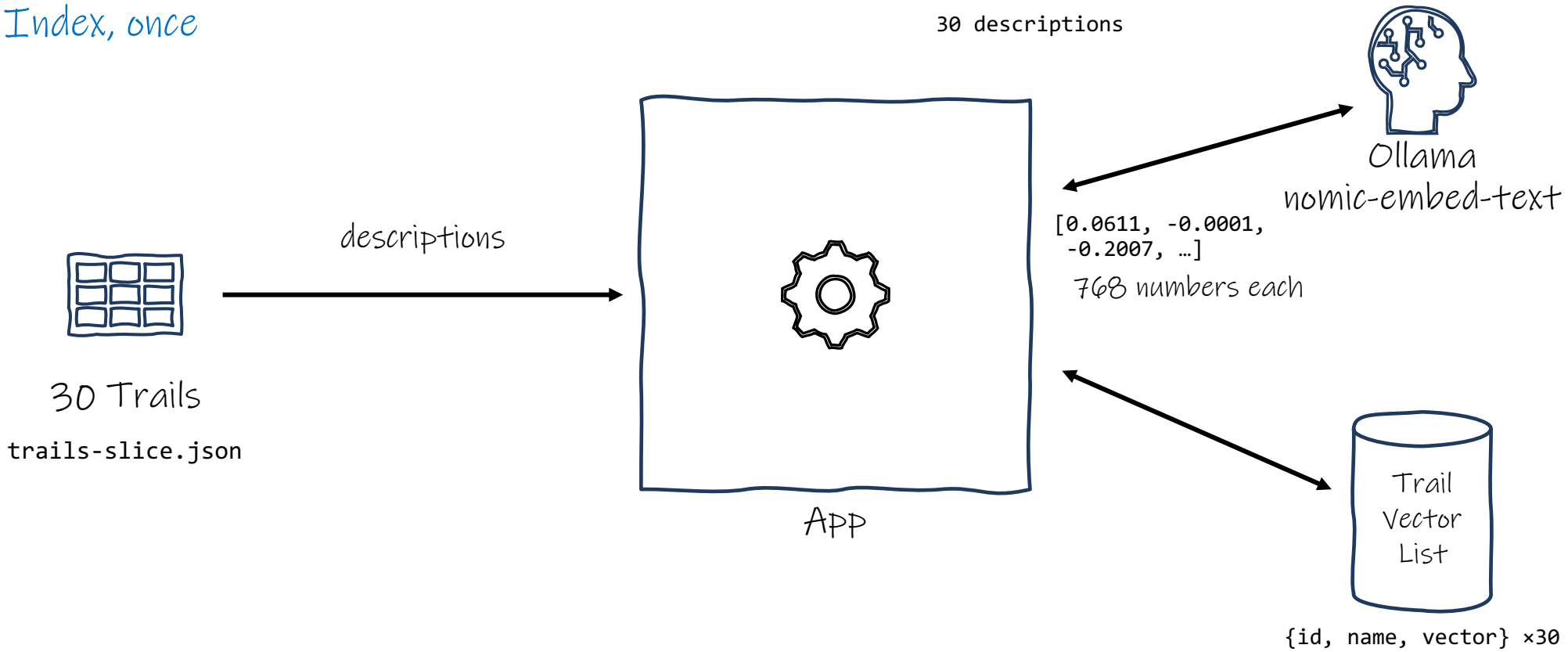
F04 · How It Works

1 · Index, once



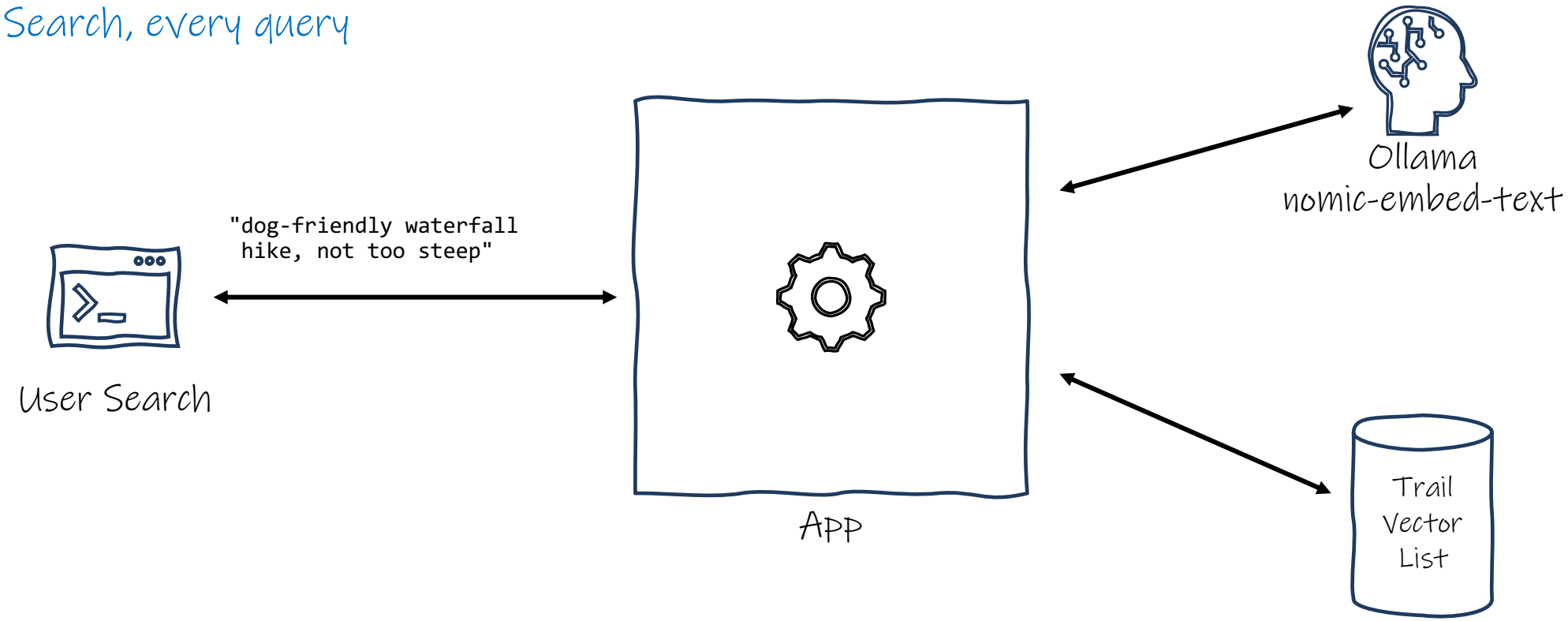
F04 · How It Works

1 · Index, once



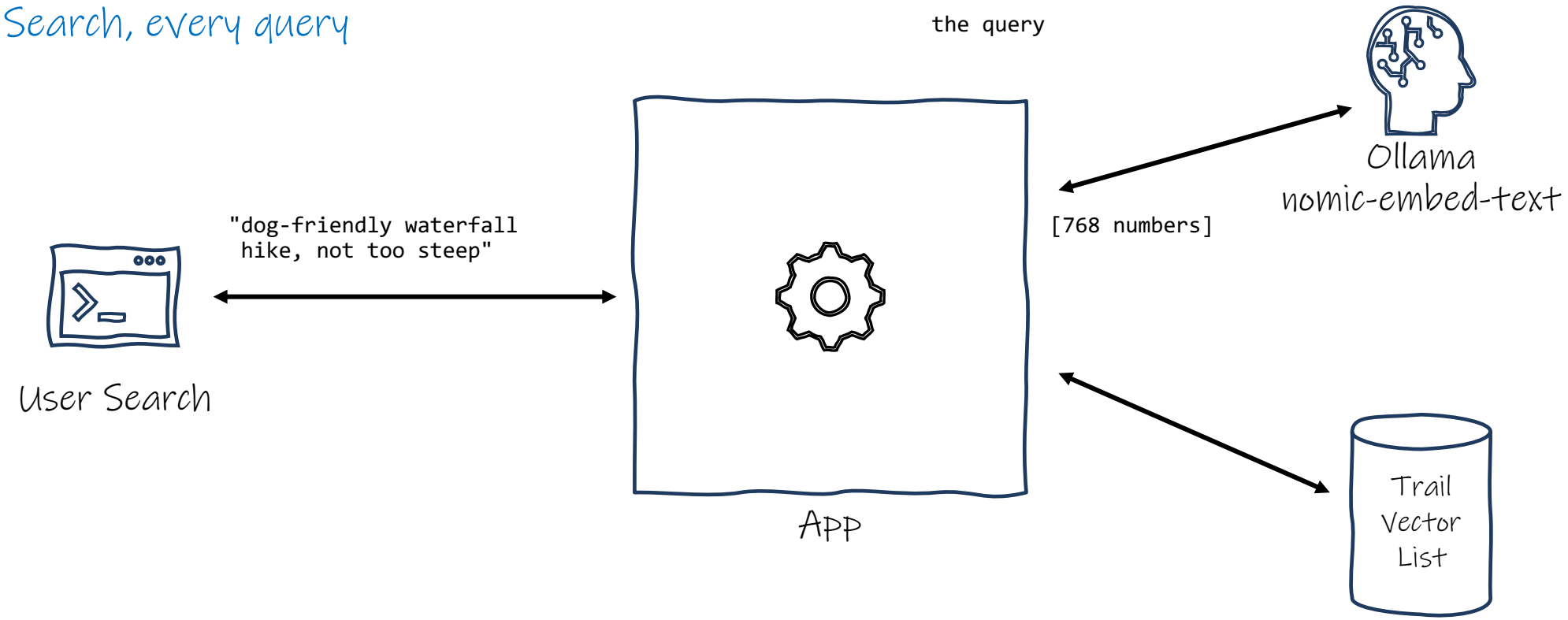
F04 · How It Works

2 · Search, every query



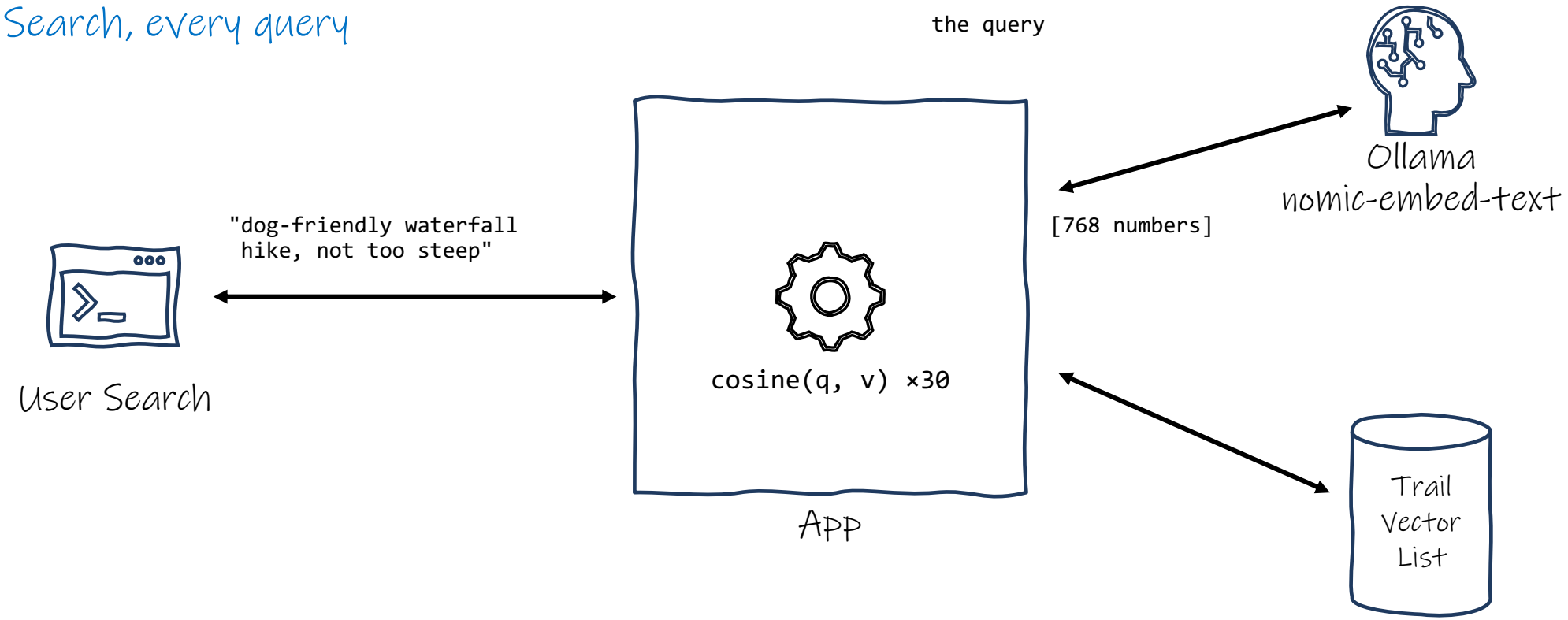
F04 · How It Works

2 · Search, every query



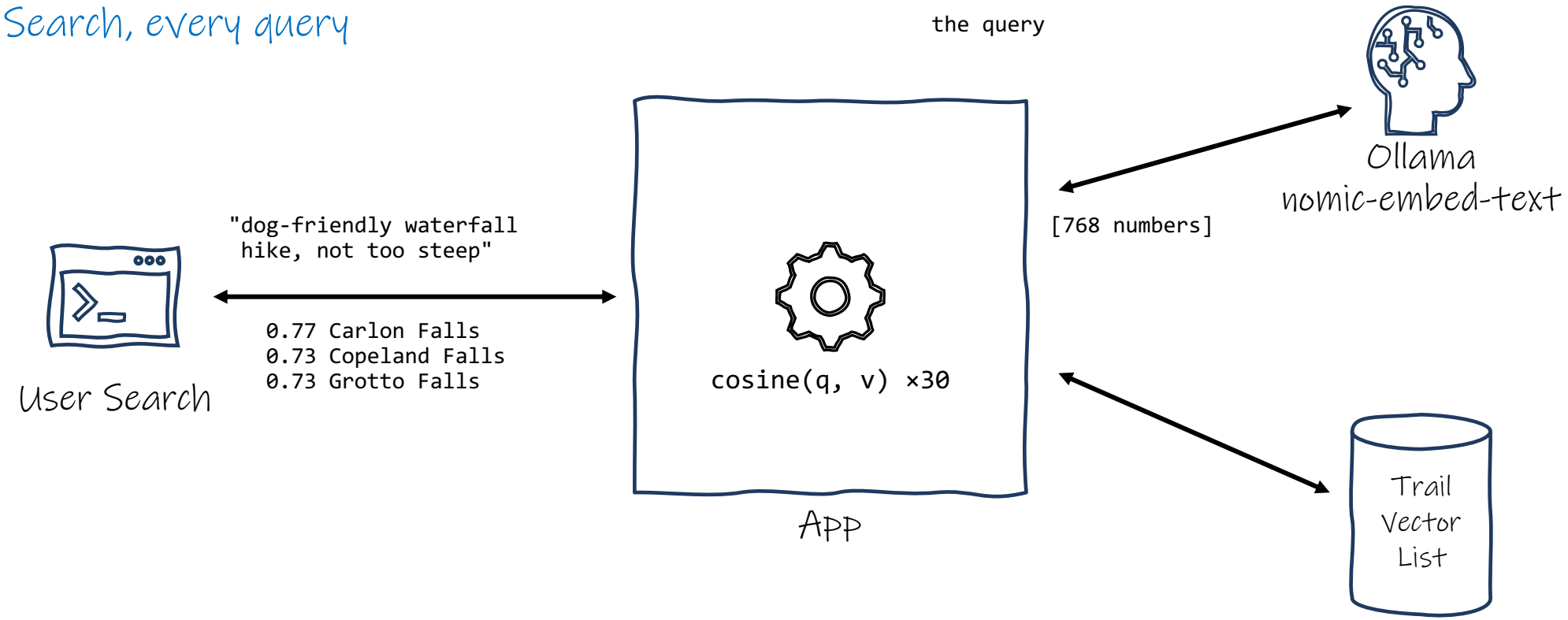
F04 · How It Works

2 · Search, every query



F04 · How It Works

2 · Search, every query





Use it when users describe what they want, and search only knows what things are called.

TRAILHEAD
TECHNOLOGY PARTNERS

F05 · RAG

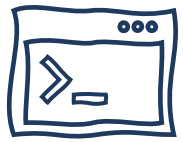
TRAILHEAD
TECHNOLOGY PARTNERS



Look in your documents as the source; the model composes an answer from what it finds.

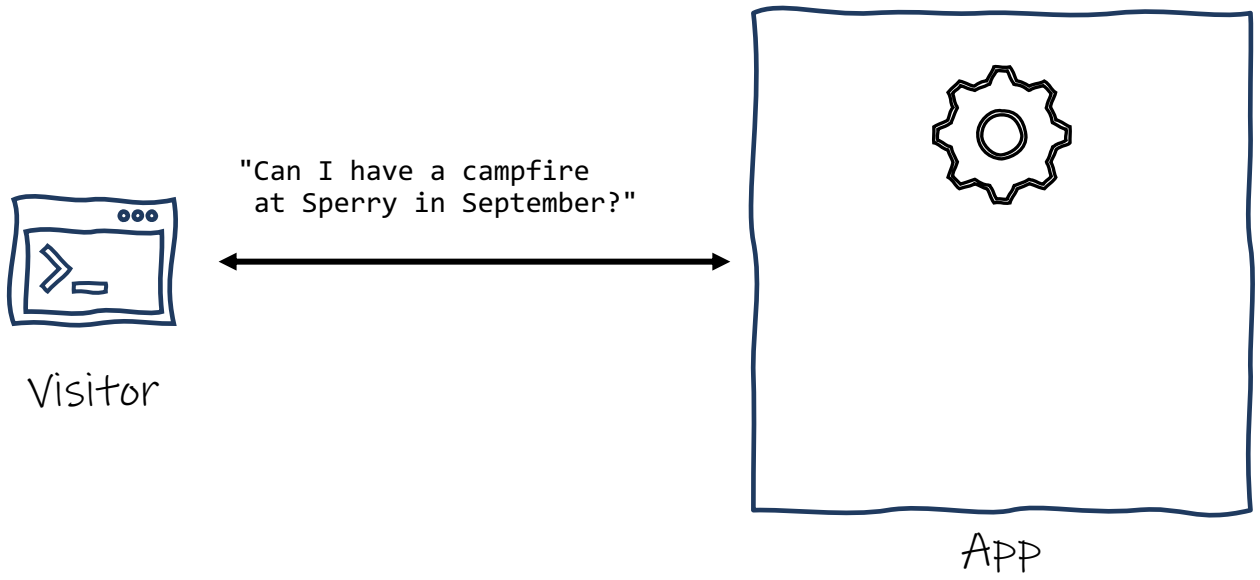
TRAILHEAD
TECHNOLOGY PARTNERS

F05 · How It Works

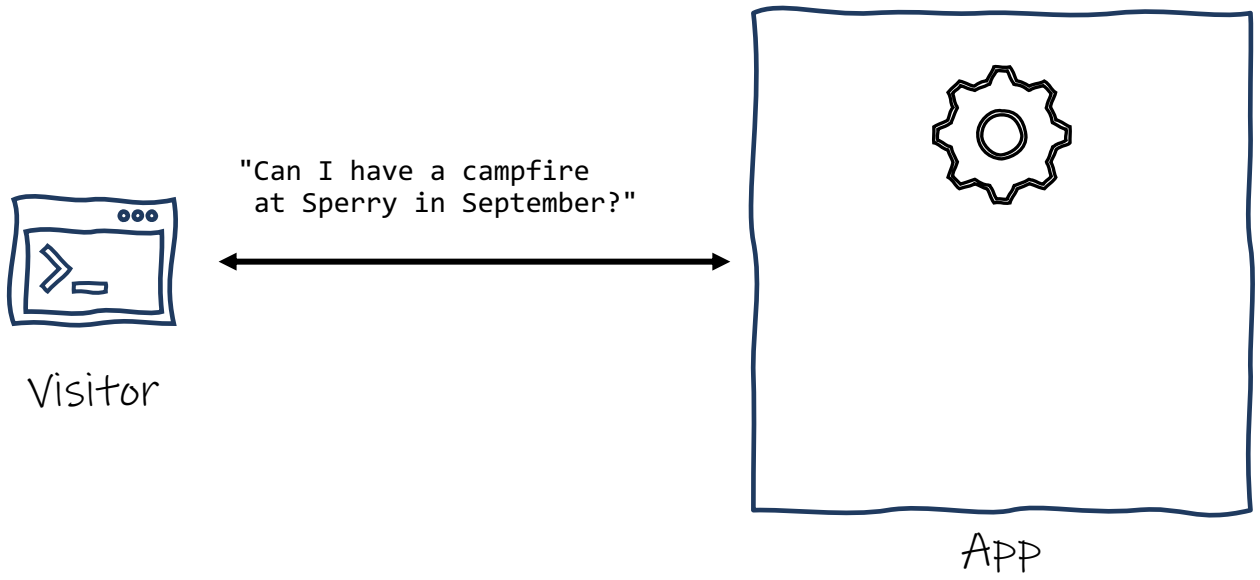


Visitor

F05 · How It Works



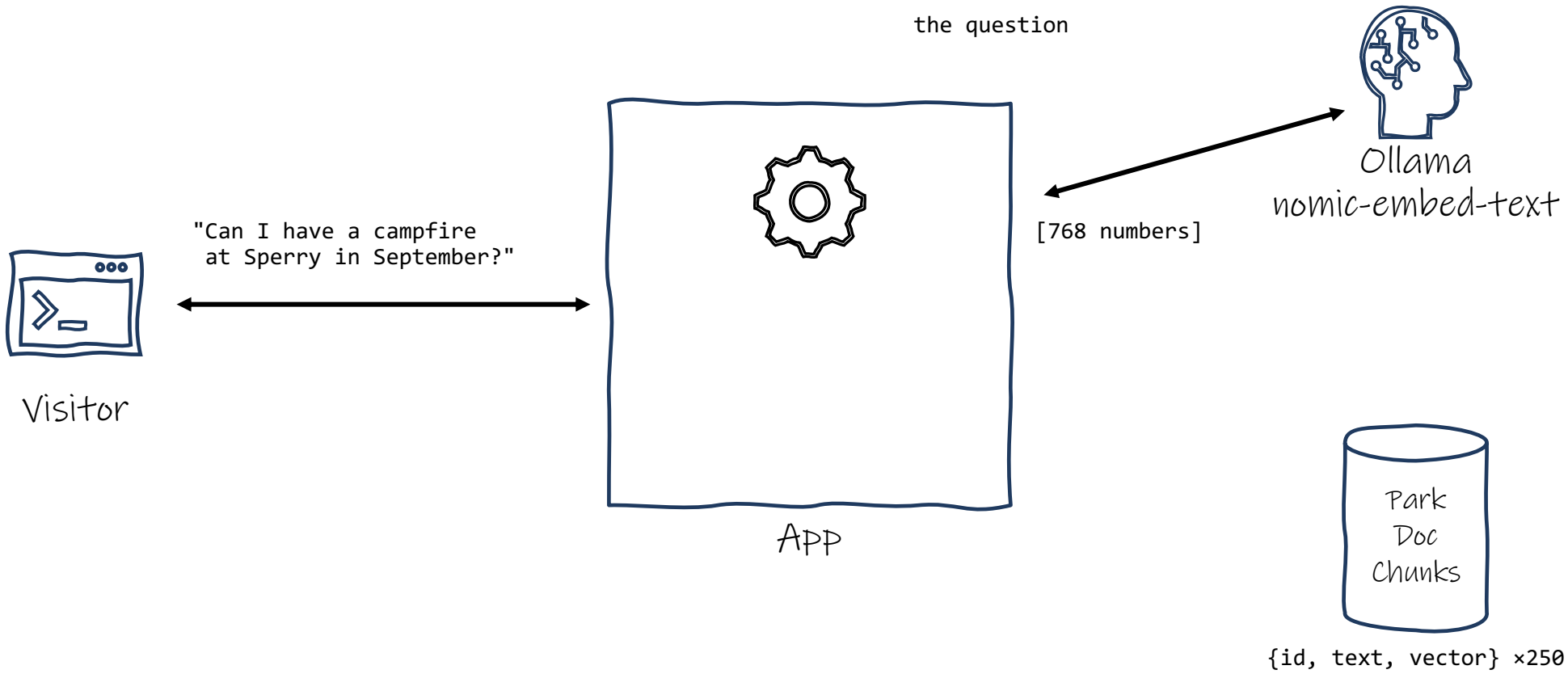
F05 · How It Works



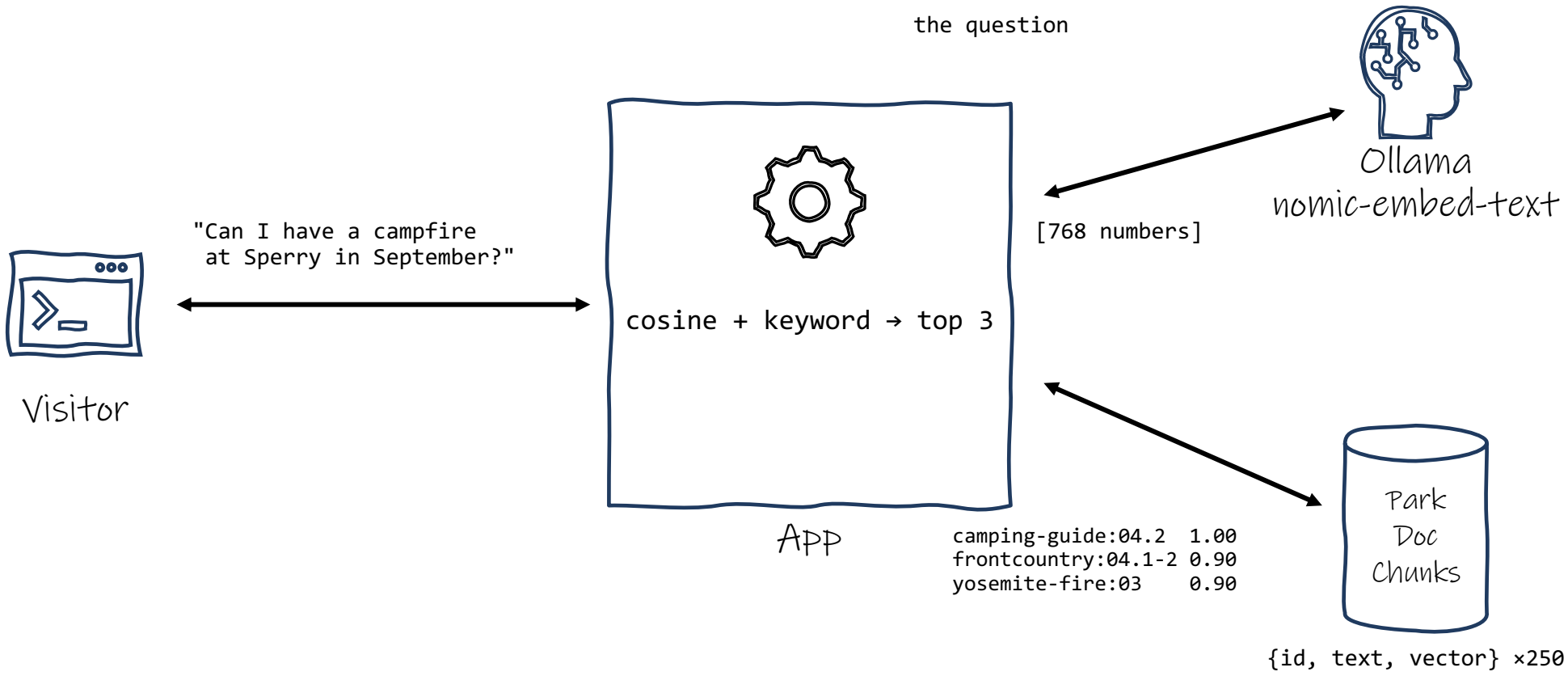


{id, text, vector} ×250

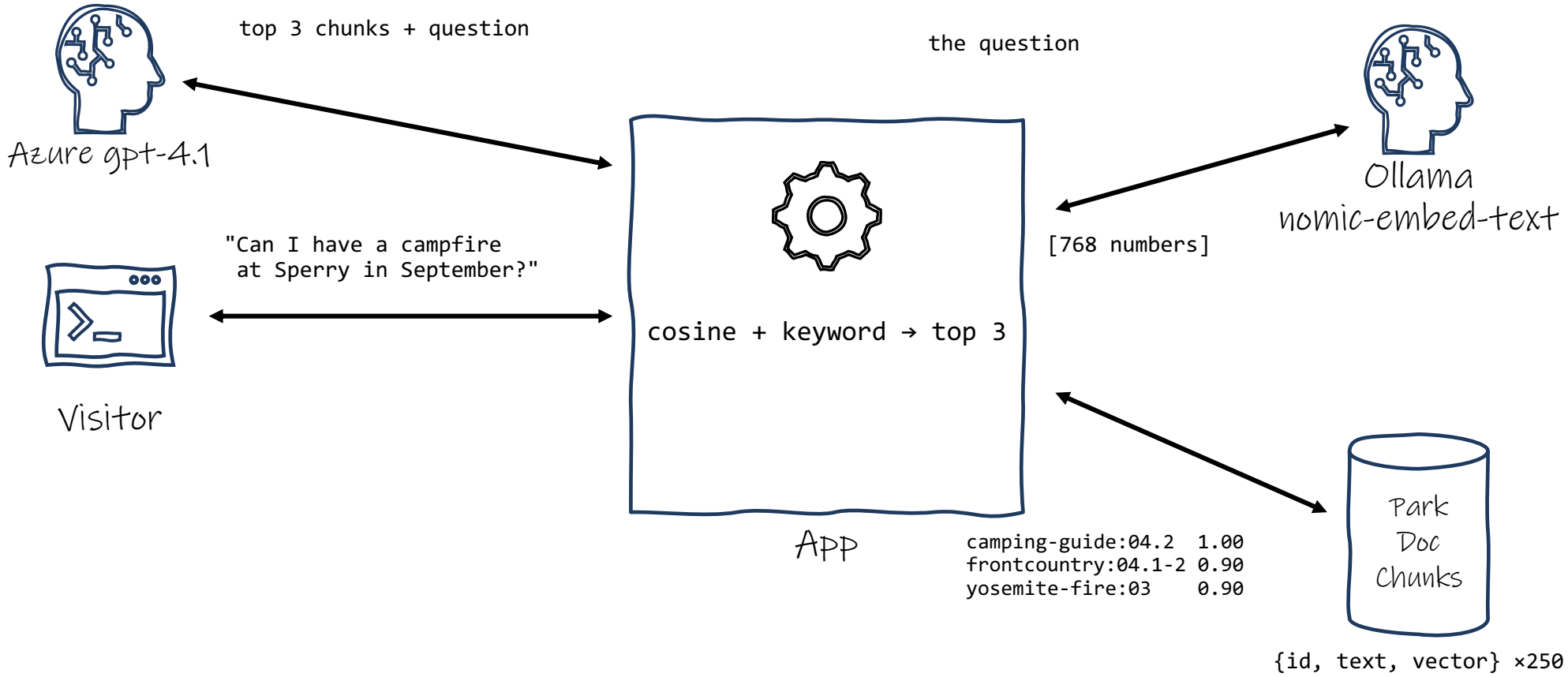
F05 · How It Works



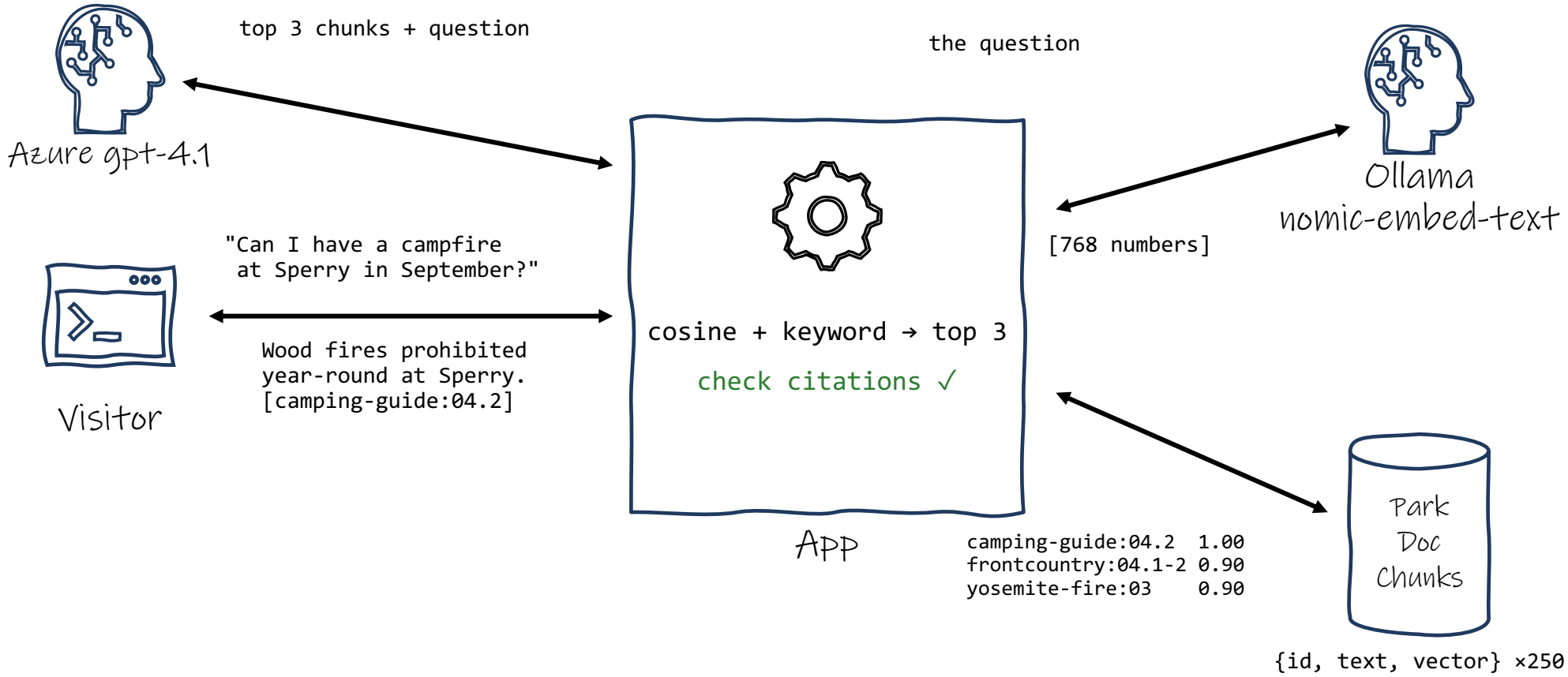
F05 · How It Works



F05 · How It Works



F05 · How It Works



F05 · Before, and Retrieve

```
if (question == "") question = "Is the Avalanche Lake Trail open right now?";
IChatClient llm = Clients.Cloud("gpt-4.1");

// Before: no documents. The model answers from memory.
if (!grounded) { await Stream(llm, question); return; }

// 1. Retrieve: the three chunks closest to the question.
var top = await Retrieve(question, topK: 3);
```

F05 · Before, and Retrieve

```
if (question == "") question = "Is the Avalanche Lake Trail open right now?";
IChatClient llm = Clients.Cloud("gpt-4.1");

// Before: no documents. The model answers from memory.
if (!grounded) { await Stream(llm, question); return; }

// 1. Retrieve: the three chunks closest to the question.
var top = await Retrieve(question, topK: 3);
```

F05 · Ground

```
var answer = await Stream(llm, $"""
    You are a park information assistant. Answer the visitor's
    question using ONLY the context below.
    - Do not use outside knowledge.
    - Cite the chunk_id of each chunk you relied on, in square
      brackets, exactly as it appears in the context.
    - If none of the context is relevant, reply exactly: "{Refusal}"
      A question about "right now" is answered from the context:
      today is September 23, 2026, and a notice in effect "until
      further notice" is still in effect.

    Context:
    {string.Join("\n\n", top.Select(c => $"chunk_id: {c.Id}\n{c.Text}"))}
    Question: {question}
    """);
```

F05 · Ground

```
var answer = await Stream(llm, $"""
    You are a park information assistant. Answer the visitor's
    question using ONLY the context below.
    - Do not use outside knowledge.
    - Cite the chunk_id of each chunk you relied on, in square
      brackets, exactly as it appears in the context.
    - If none of the context is relevant, reply exactly: "{Refusal}"
      A question about "right now" is answered from the context:
      today is September 23, 2026, and a notice in effect "until
      further notice" is still in effect.

    Context:
    {string.Join("\n\n", top.Select(c => $"chunk_id: {c.Id}\n{c.Text}"))}
    Question: {question}
    """);
```

F05 · Ground

```
var answer = await Stream(llm, $"""
    You are a park information assistant. Answer the visitor's
    question using ONLY the context below.
    - Do not use outside knowledge.
    - Cite the chunk_id of each chunk you relied on, in square
      brackets, exactly as it appears in the context.
    - If none of the context is relevant, reply exactly: "{Refusal}"
      A question about "right now" is answered from the context:
      today is September 23, 2026, and a notice in effect "until
      further notice" is still in effect.

    Context:
    {string.Join("\n\n", top.Select(c => $"chunk_id: {c.Id}\n{c.Text}"))}
    Question: {question}
    """);
```

F05 · Verify

```
// 3. Verify: every citation must point at a chunk we retrieved.
var bad = Citations(answer).Where(id => !top.Any(c => c.Id == id));
Console.WriteLine(bad.Any()
    ? $"\n!! INVALID CITATIONS: {string.Join(", ", bad)}"
    : $"\n[citations valid: {string.Join(", ", Citations(answer).Distinct())}]");
```

DEMO · F05 RAG



TRAILHEAD
TECHNOLOGY PARTNERS



Use it when the answer is written down,
but people still ask.

TRAILHEAD
TECHNOLOGY PARTNERS

F06 · Recommendations

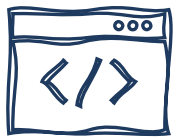
TRAILHEAD
TECHNOLOGY PARTNERS



Suggest the next thing a user will want,
from what they liked/did already.

TRAILHEAD
TECHNOLOGY PARTNERS

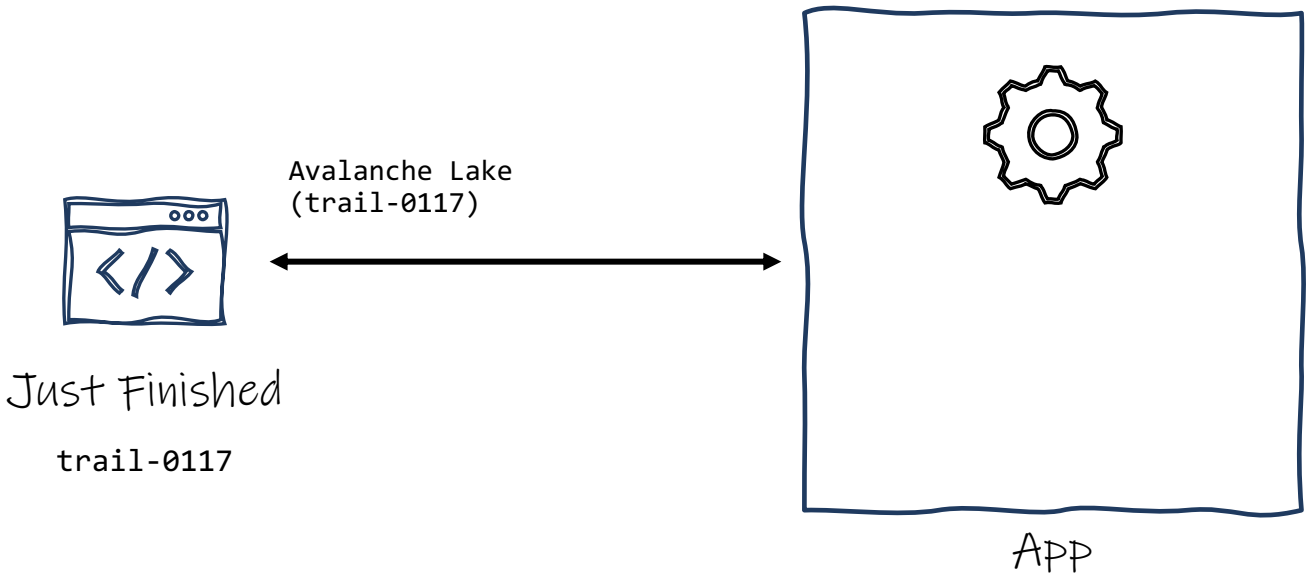
F06 · How It Works



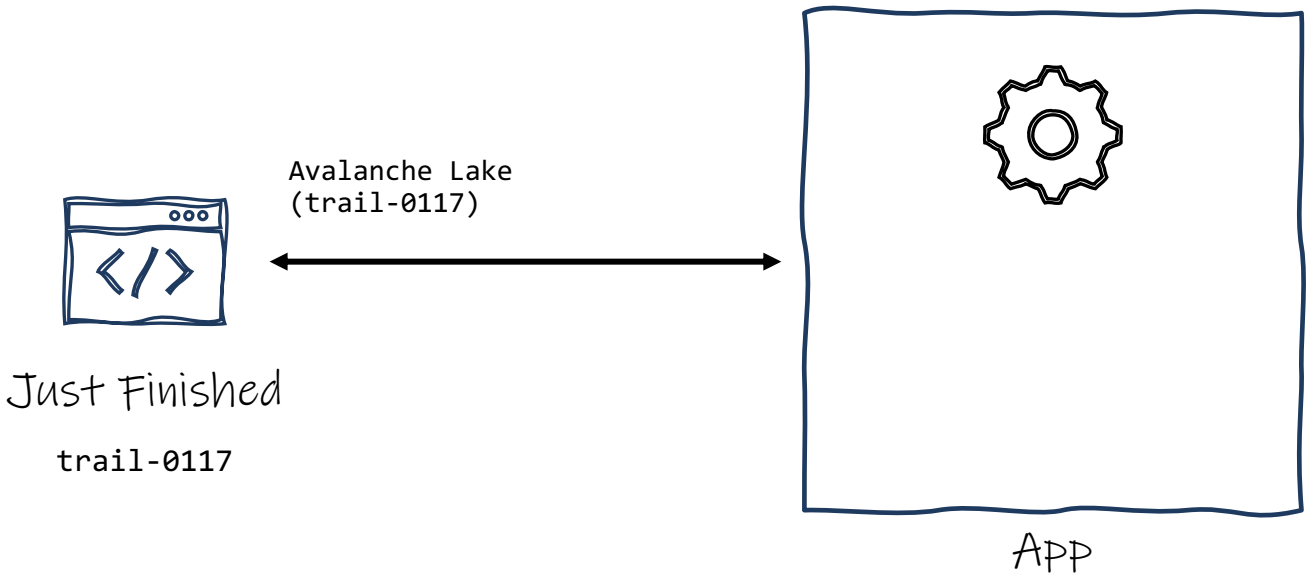
Just Finished

`trail-0117`

F06 · How It Works



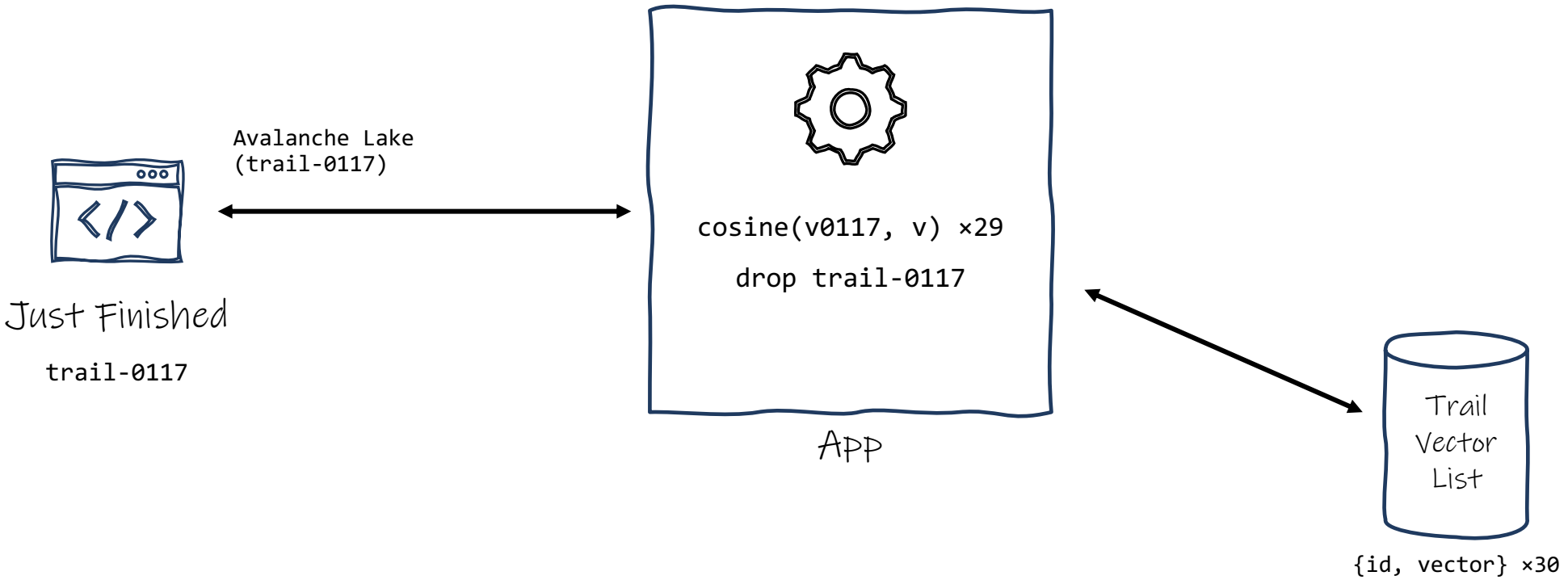
F06 · How It Works



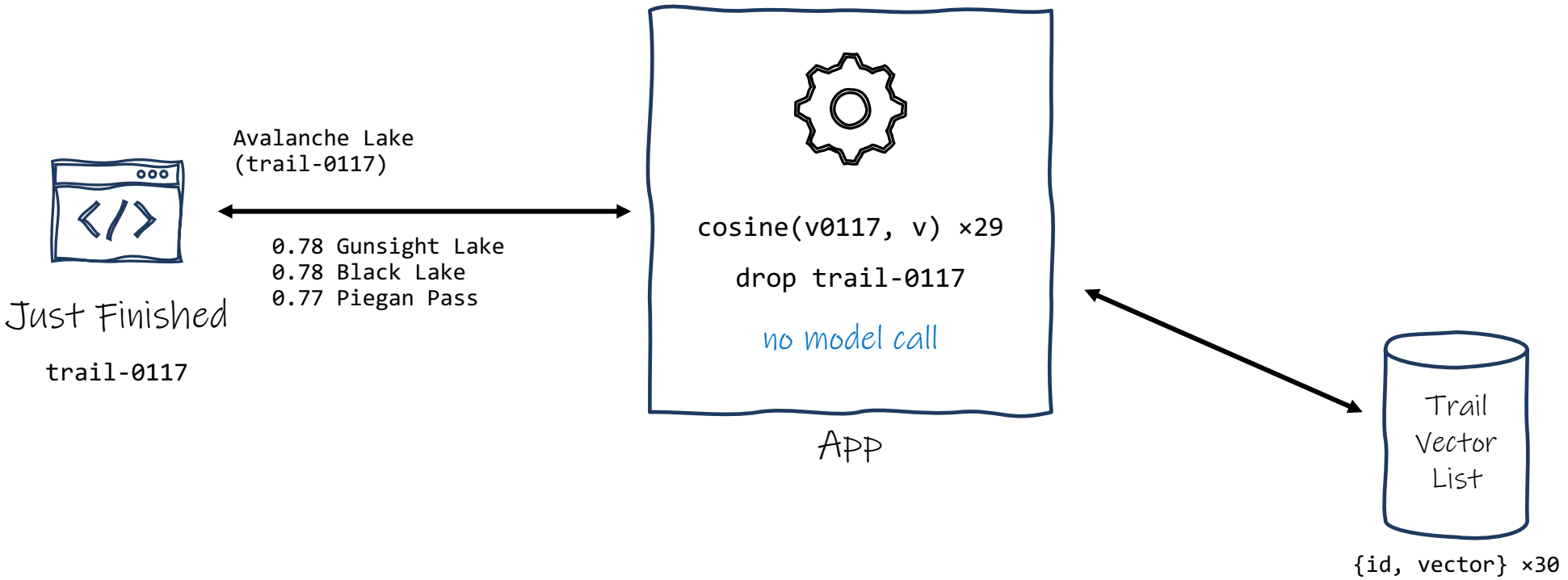


{id, vector} ×30

F06 · How It Works



F06 · How It Works



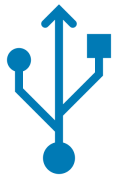


Use it when users finish one thing and you want to suggest the next.

TRAILHEAD
TECHNOLOGY PARTNERS

F07 · Classification & Routing

TRAILHEAD
TECHNOLOGY PARTNERS



Give each message a label, and
let the label pick the queue.

TRAILHEAD
TECHNOLOGY PARTNERS

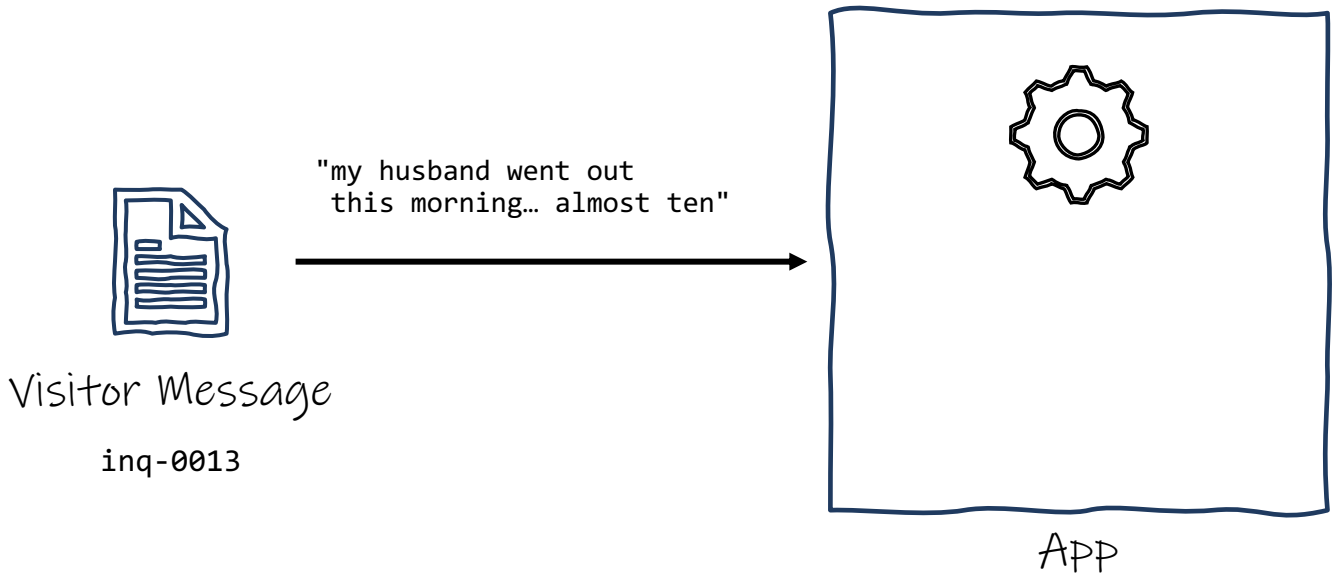
F07 · How It Works



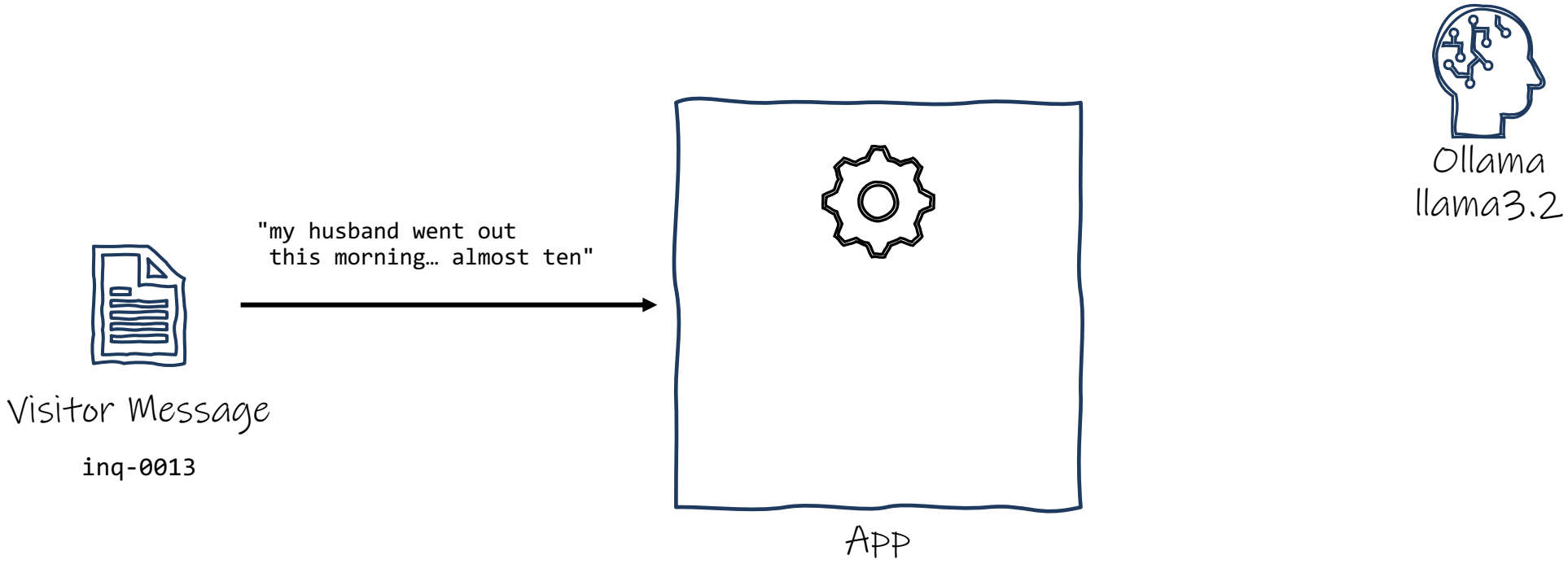
Visitor Message

inq-0013

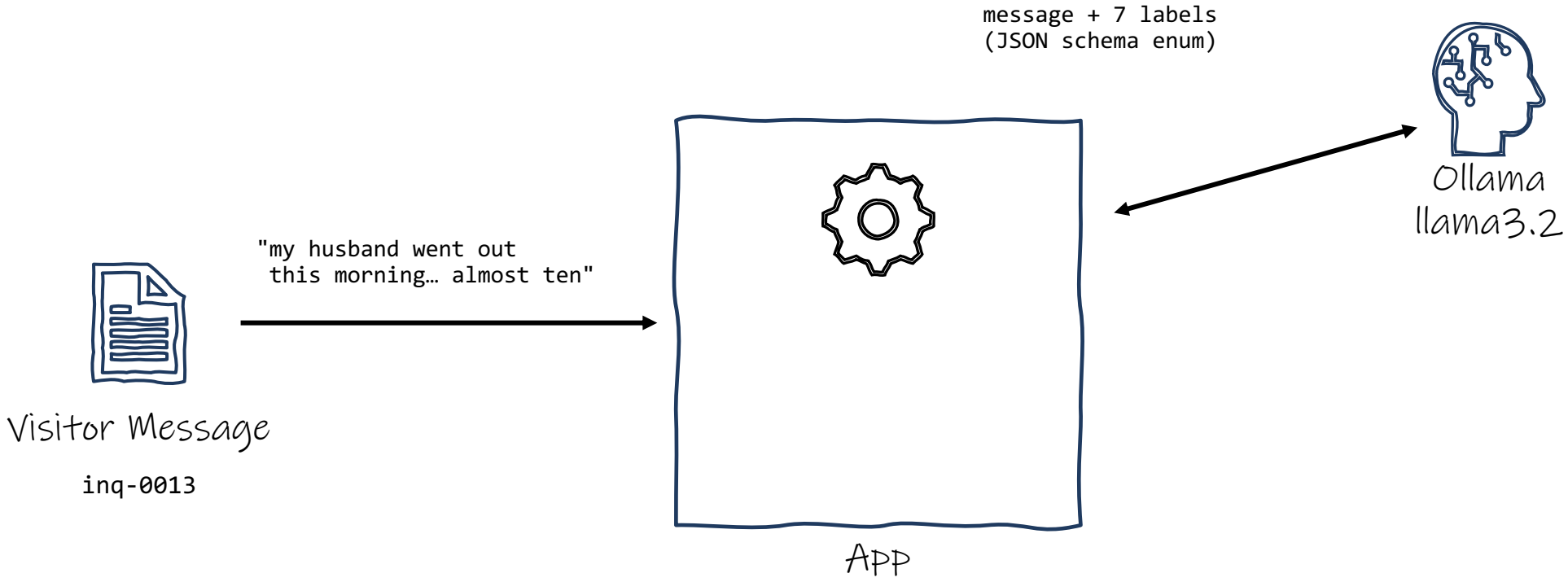
F07 · How It Works



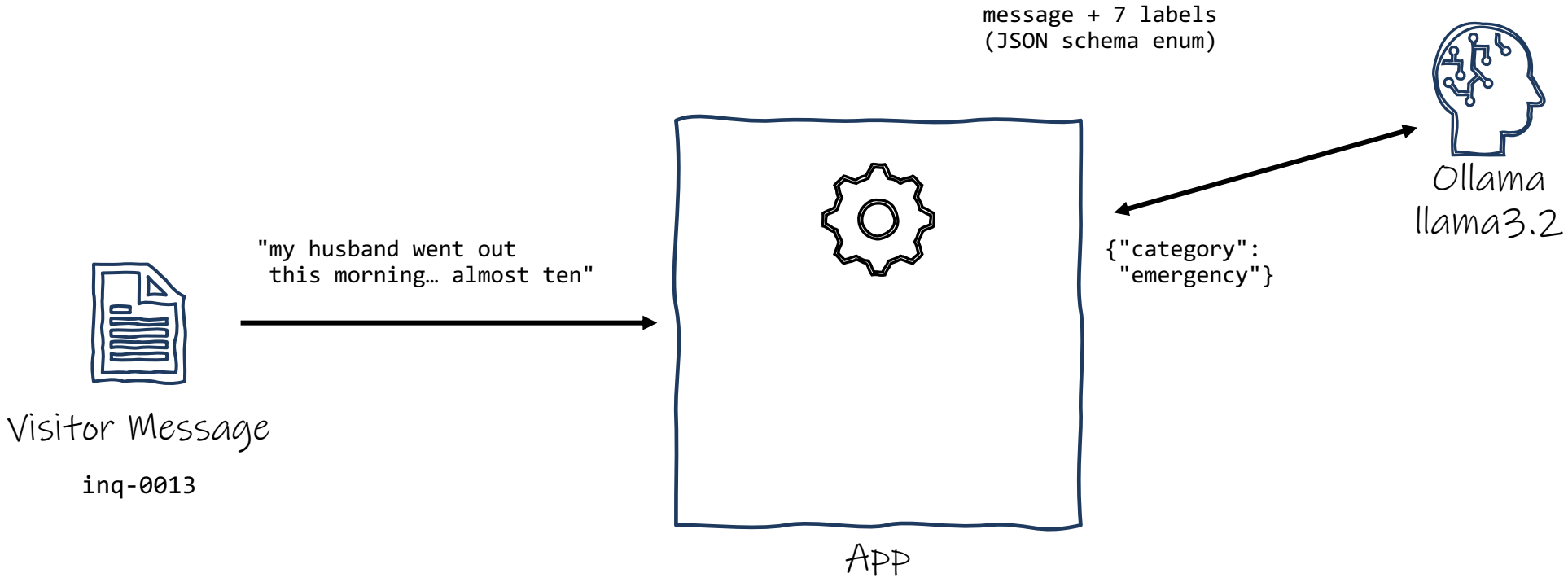
F07 · How It Works



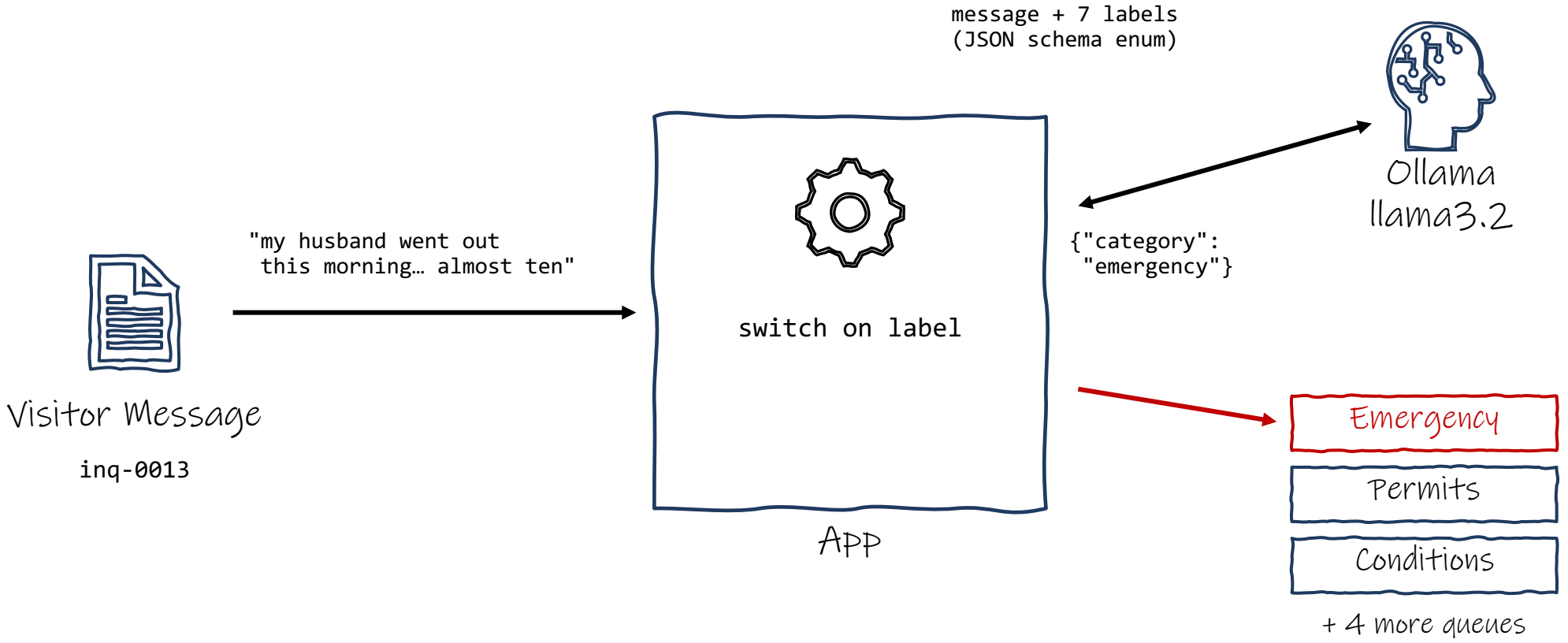
F07 · How It Works



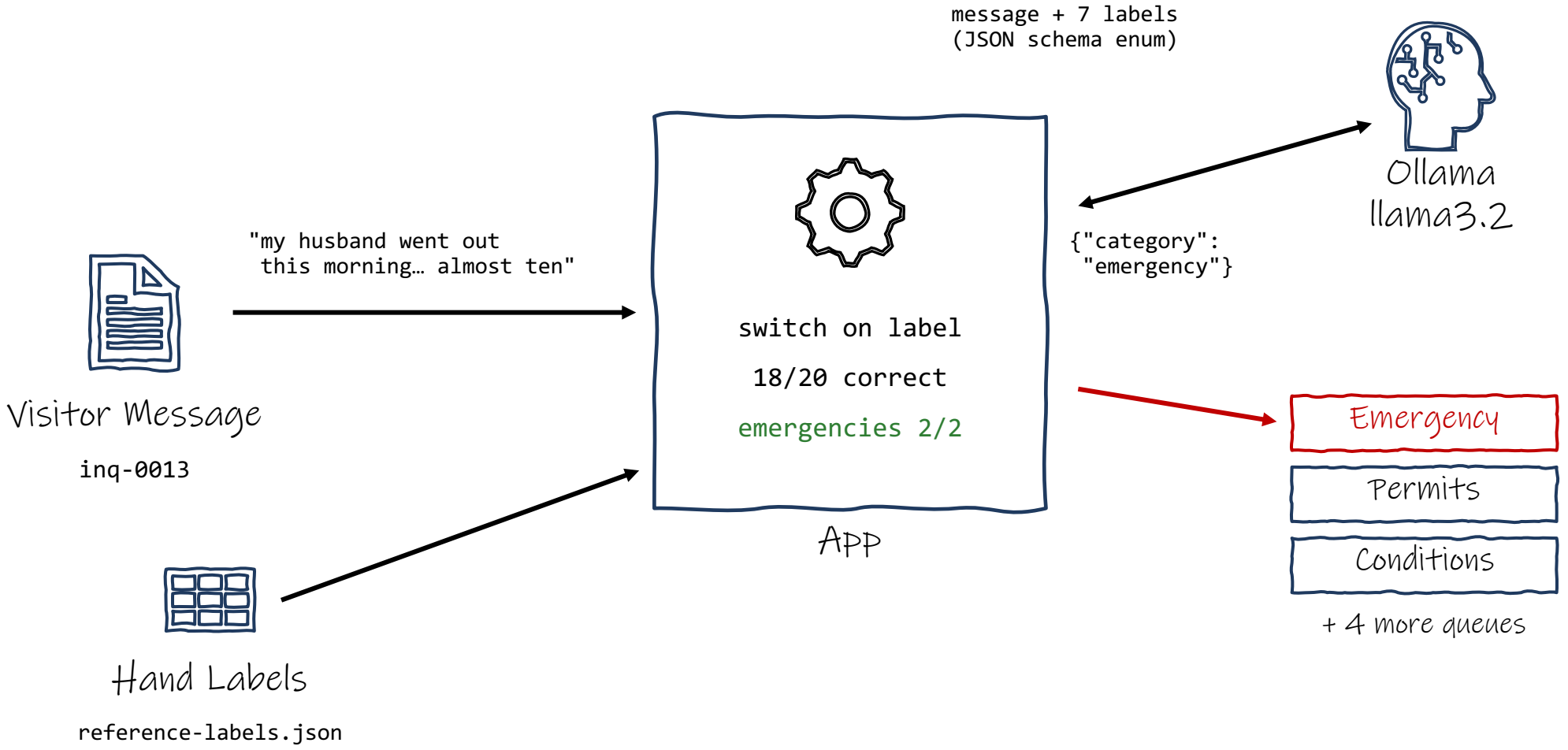
F07 · How It Works



F07 · How It Works



F07 · How It Works



F07 · The Loop

```
IChatClient client = Clients.Local("llama3.2");
// Same message, same queue, every run. Scoring needs that.
var options = new ChatOptions { Temperature = 0 };
foreach (var inquiry in LoadInquiries())
{
    // Only a Category can come back: every label has a queue.
    var response = await client.GetResponseAsync<TriageResult>(
        Prompt(inquiry.text), options);
    var queue = Route(response.Result.Category);
    queues[queue].Add(inquiry);   // the label picks the queue
    results.Add((inquiry, response.Result.Category));
}

// Accuracy is the headline. Emergency recall decides whether it ships:
// a missed emergency is a person waiting in a queue nobody watches.
Score(results);
```

F07 · The Loop

```
IChatClient client = Clients.Local("llama3.2");
// Same message, same queue, every run. Scoring needs that.
var options = new ChatOptions { Temperature = 0 };
foreach (var inquiry in LoadInquiries())
{
    // Only a Category can come back: every label has a queue.
    var response = await client.GetResponseAsync<TriageResult>(
        Prompt(inquiry.text), options);
    var queue = Route(response.Result.Category);
    queues[queue].Add(inquiry);   // the label picks the queue
    results.Add((inquiry, response.Result.Category));
}

// Accuracy is the headline. Emergency recall decides whether it ships:
// a missed emergency is a person waiting in a queue nobody watches.
Score(results);
```

F07 · The Loop

```
IChatClient client = Clients.Local("llama3.2");
// Same message, same queue, every run. Scoring needs that.
var options = new ChatOptions { Temperature = 0 };
foreach (var inquiry in LoadInquiries())
{
    // Only a Category can come back: every label has a queue.
    var response = await client.GetResponseAsync<TriageResult>(
        Prompt(inquiry.text), options);
    var queue = Route(response.Result.Category);
    queues[queue].Add(inquiry);   // the label picks the queue
    results.Add((inquiry, response.Result.Category));
}

// Accuracy is the headline. Emergency recall decides whether it ships:
// a missed emergency is a person waiting in a queue nobody watches.
Score(results);
```

F07 · The Taxonomy Is the Prompt

```
- permit: reserving, changing, canceling, or paying for a permit, pass,
  or reservation, including billing problems and missing confirmations
  for a permit application, and asking whether an activity such as
  commercial photography needs a special use permit.
- conditions: asking whether a trail, road, or area is open, safe, or
  passable right now: snow, water levels, washouts, wildlife activity, closures.
- complaint: unhappy about a park facility, service, or staff member
  and wants it acknowledged or fixed.
- lost-and-found: reporting a lost or found physical item.
- emergency: a person may be hurt, missing, or in danger right now and
  needs immediate human attention.
- general: anything else: park rules, fees, trip planning, questions that fit none of the ab
- unsure: two different queues both have to act before this message can
  be resolved, so no single queue owns it.
```

F07 · The Taxonomy Is the Prompt

```
- permit: reserving, changing, canceling, or paying for a permit, pass,
  or reservation, including billing problems and missing confirmations
  for a permit application, and asking whether an activity such as
  commercial photography needs a special use permit.
- conditions: asking whether a trail, road, or area is open, safe, or
  passable right now: snow, water levels, washouts, wildlife activity, closures.
- complaint: unhappy about a park facility, service, or staff member
  and wants it acknowledged or fixed.
- lost-and-found: reporting a lost or found physical item.
- emergency: a person may be hurt, missing, or in danger right now and
  needs immediate human attention.
- general: anything else: park rules, fees, trip planning, questions that fit none of the ab
- unsure: two different queues both have to act before this message can
  be resolved, so no single queue owns it.
```

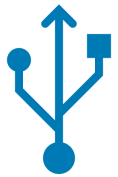
F07 · The Taxonomy Is the Prompt

```
- permit: reserving, changing, canceling, or paying for a permit, pass,
  or reservation, including billing problems and missing confirmations
  for a permit application, and asking whether an activity such as
  commercial photography needs a special use permit.
- conditions: asking whether a trail, road, or area is open, safe, or
  passable right now: snow, water levels, washouts, wildlife activity, closures.
- complaint: unhappy about a park facility, service, or staff member
  and wants it acknowledged or fixed.
- lost-and-found: reporting a lost or found physical item.
- emergency: a person may be hurt, missing, or in danger right now and
  needs immediate human attention.
- general: anything else: park rules, fees, trip planning, questions that fit none of the ab
- unsure: two different queues both have to act before this message can
  be resolved, so no single queue owns it.
```

DEMO · F07 Classification & Routing



TRAILHEAD
TECHNOLOGY PARTNERS



Use it when every message waits for someone to read it and pass it on.

TRAILHEAD
TECHNOLOGY PARTNERS

F08 · Anomaly Detection

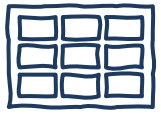
TRAILHEAD
TECHNOLOGY PARTNERS



Spot the item in a stream
that doesn't look like the rest.

TRAILHEAD
TECHNOLOGY PARTNERS

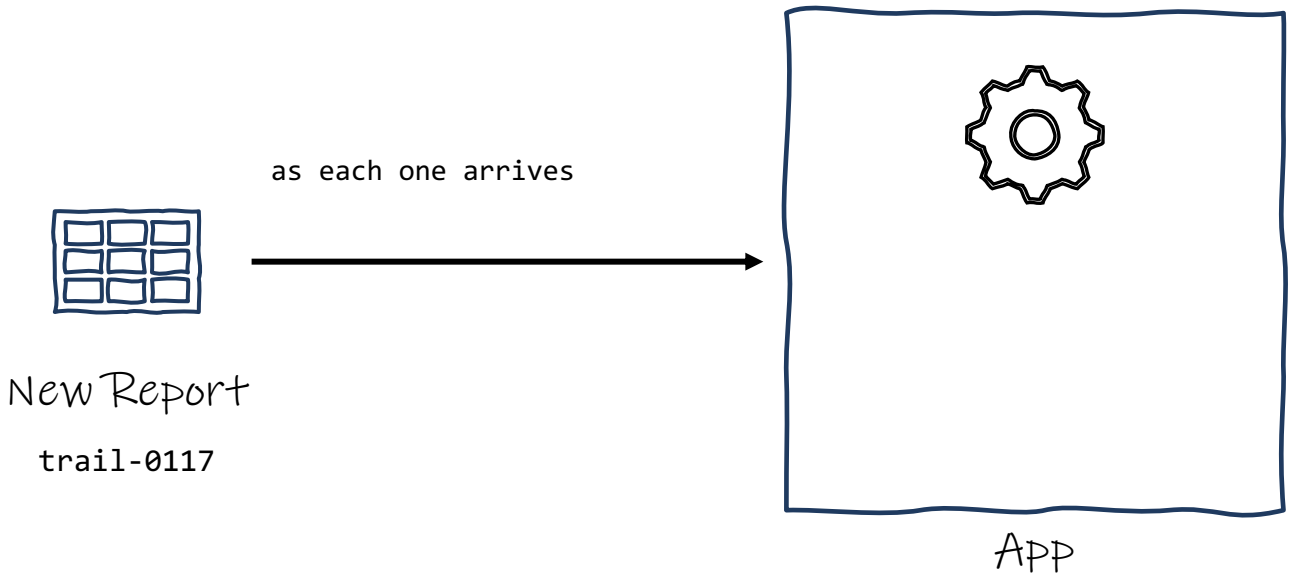
F08 · How It Works



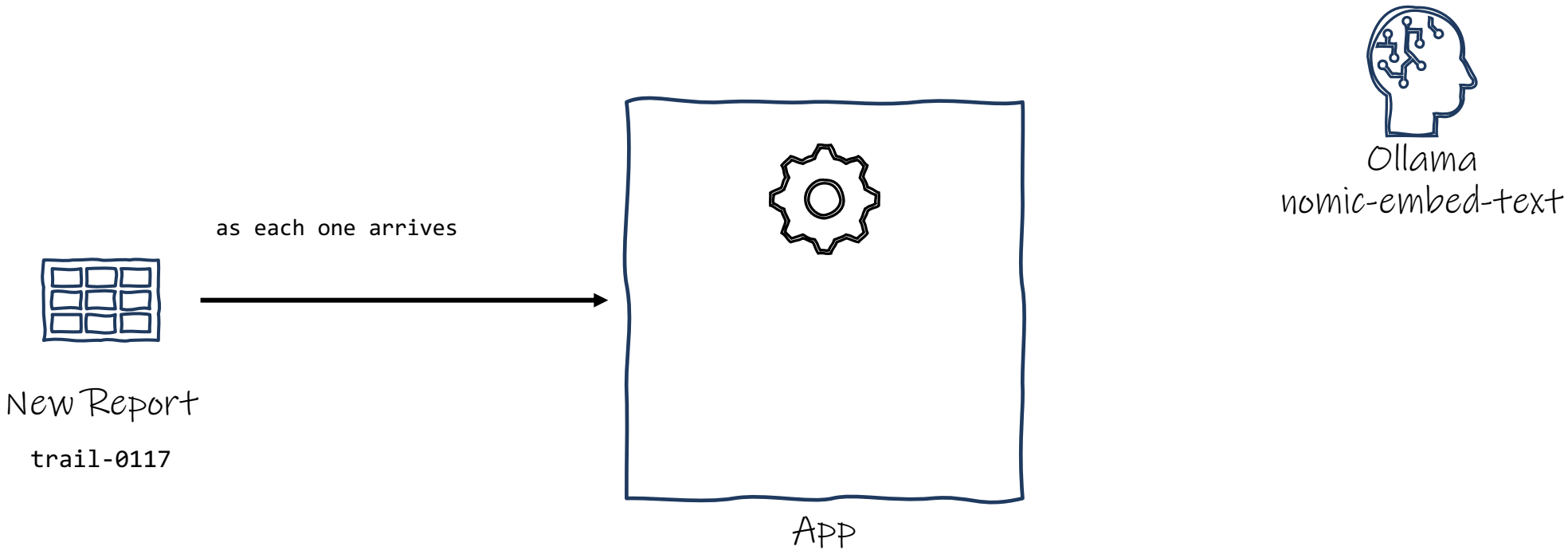
New Report

trail-0117

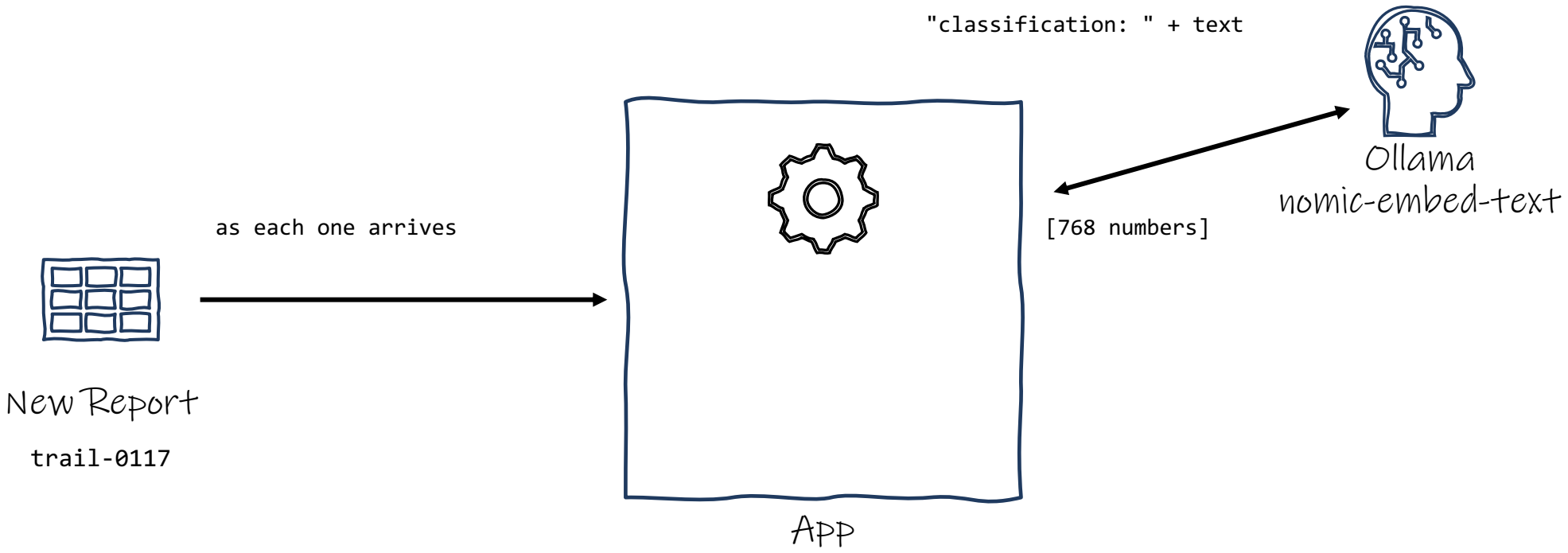
F08 · How It Works



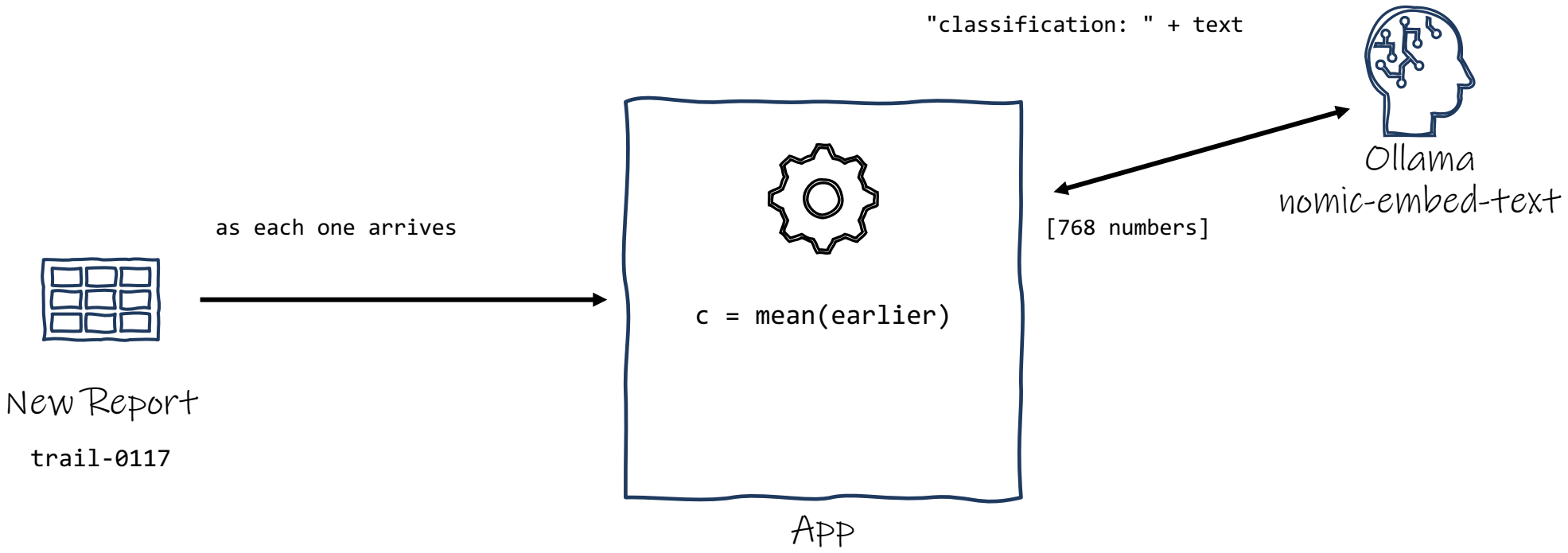
F08 · How It Works



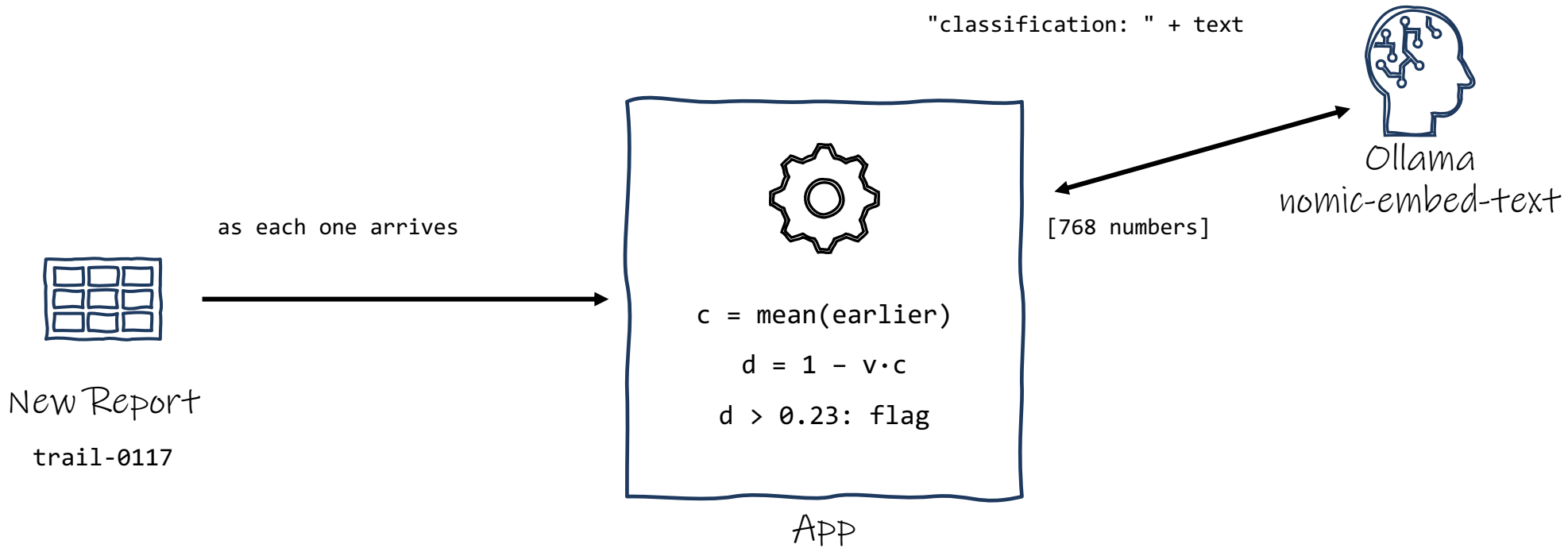
F08 · How It Works



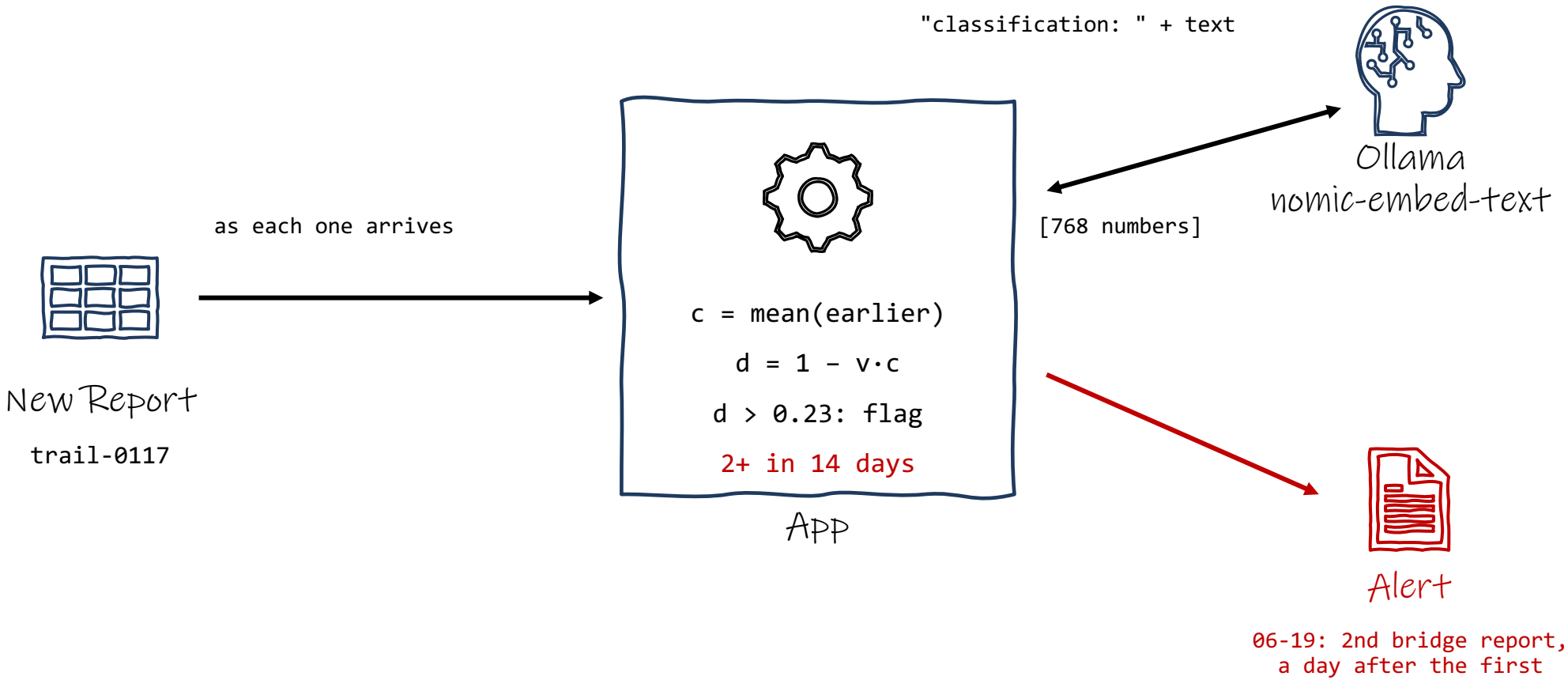
F08 · How It Works



F08 · How It Works



F08 · How It Works





Use it when an exception is expensive to miss.

TRAILHEAD
TECHNOLOGY PARTNERS

F09 · Human-in-the-Loop

TRAILHEAD
TECHNOLOGY PARTNERS



The AI drafts it, but a person approves before anything goes out.

TRAILHEAD
TECHNOLOGY PARTNERS

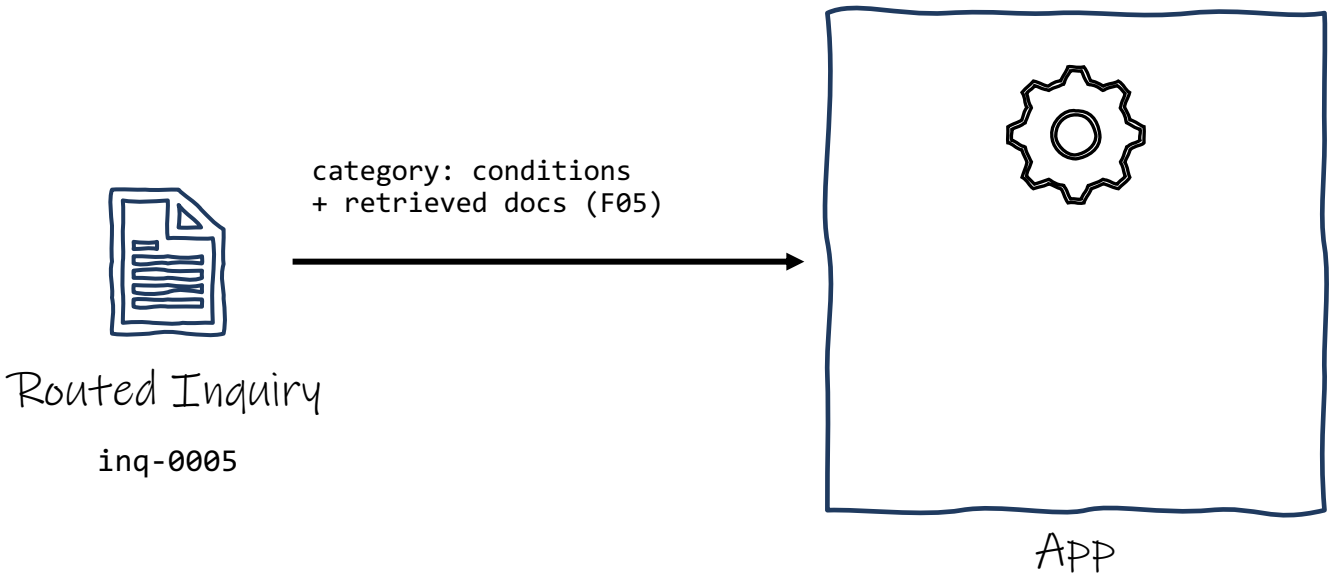
F09 · How It Works



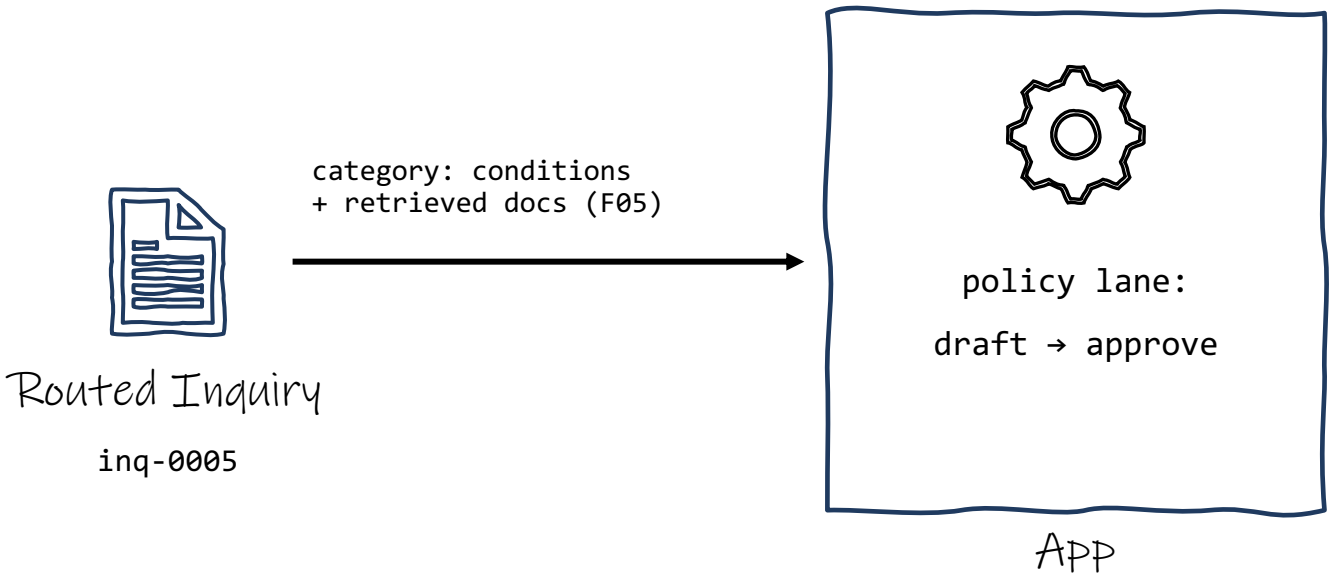
Routed Inquiry

inq-0005

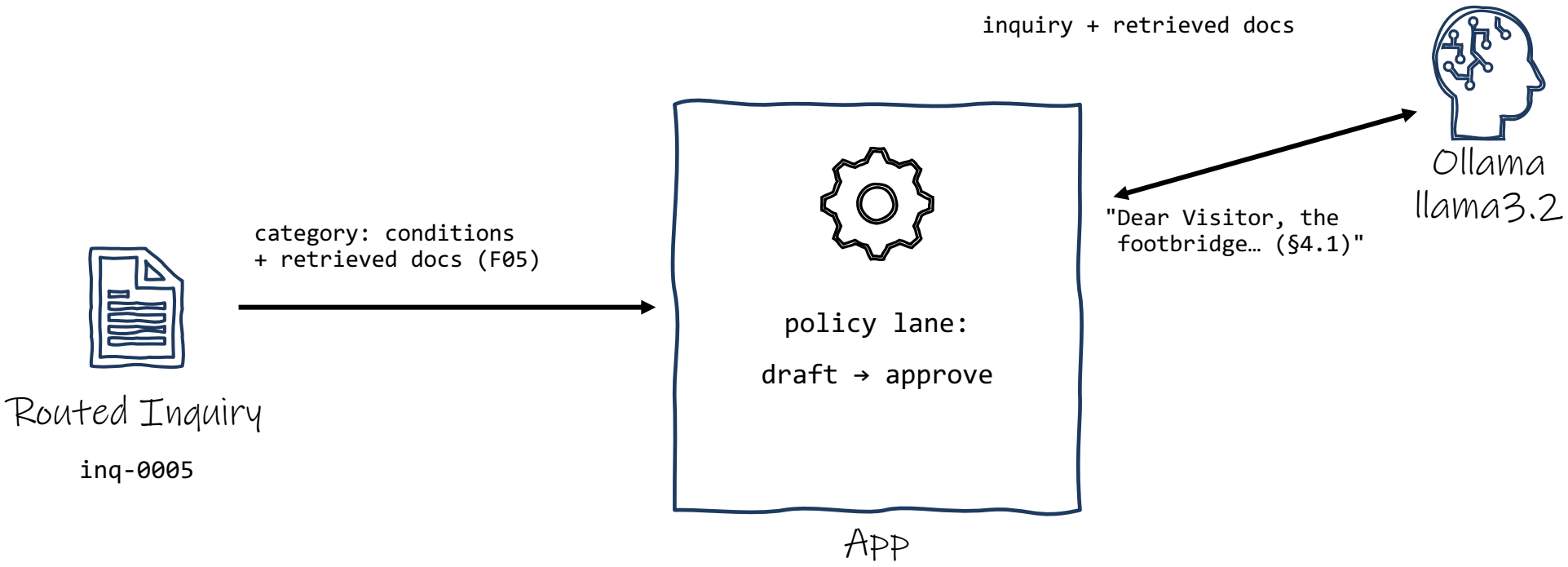
F09 · How It Works



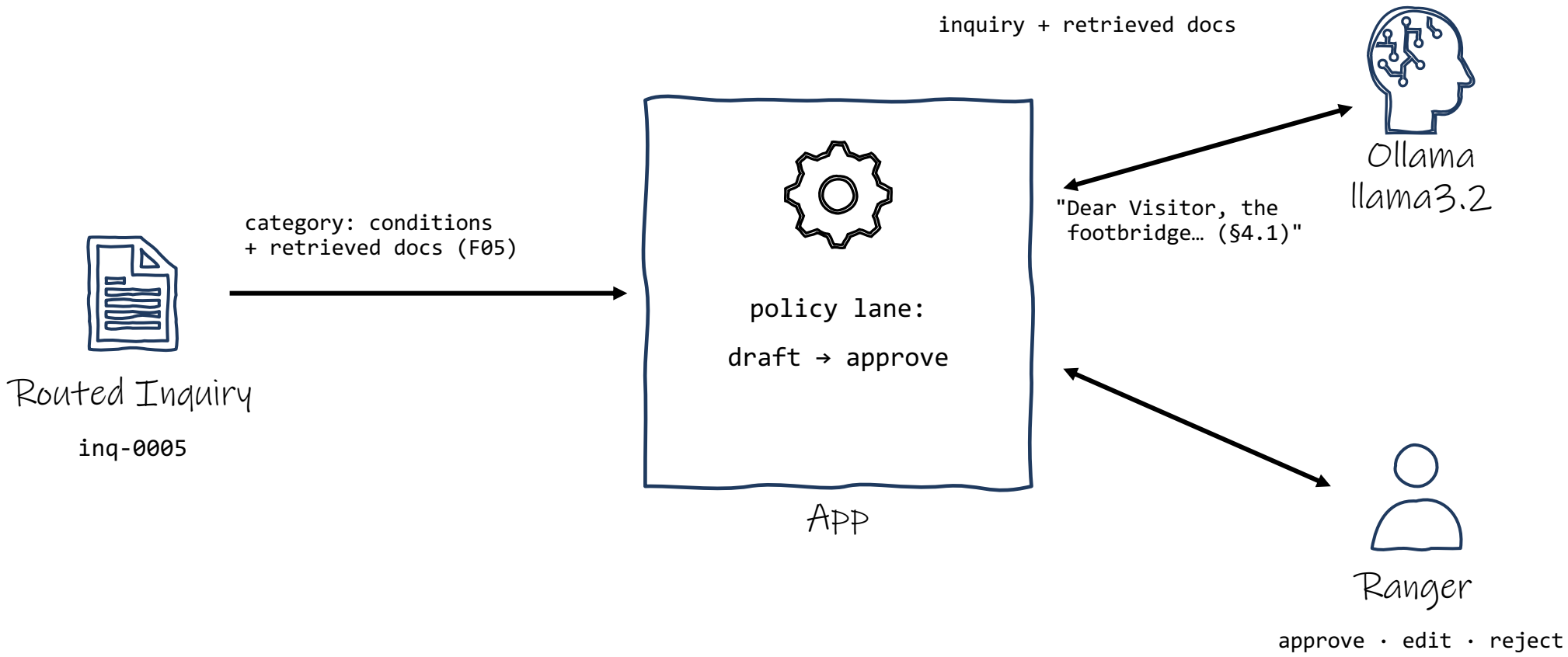
F09 · How It Works



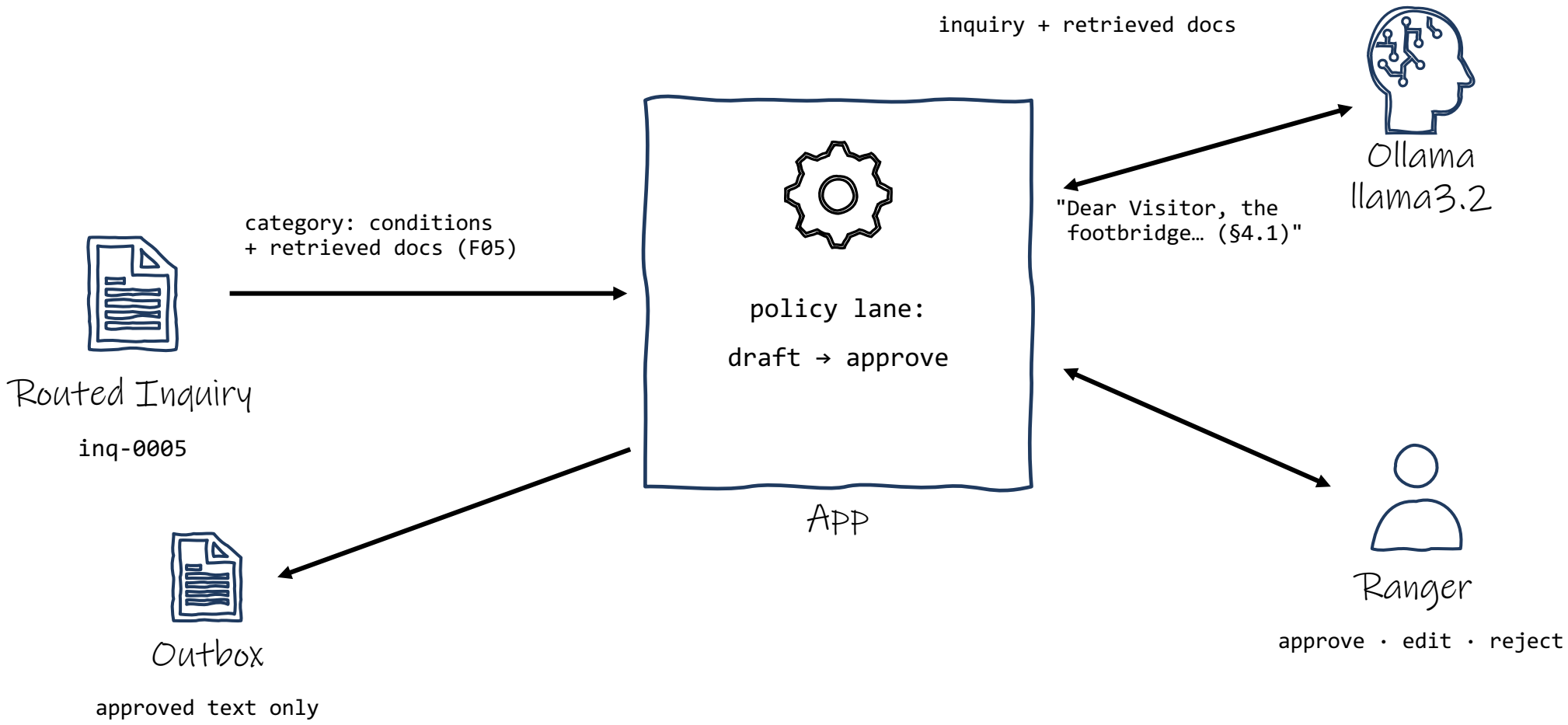
F09 · How It Works



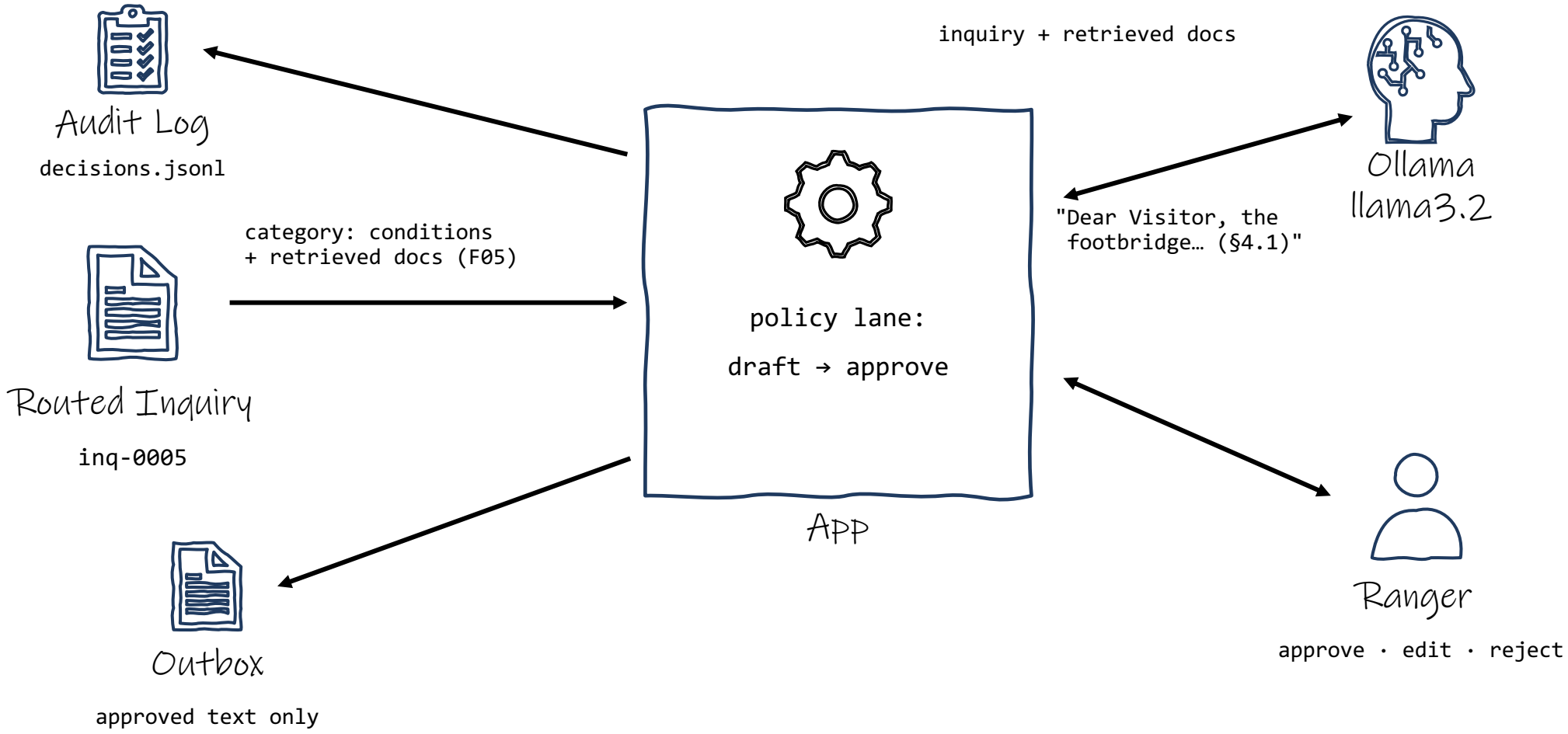
F09 · How It Works



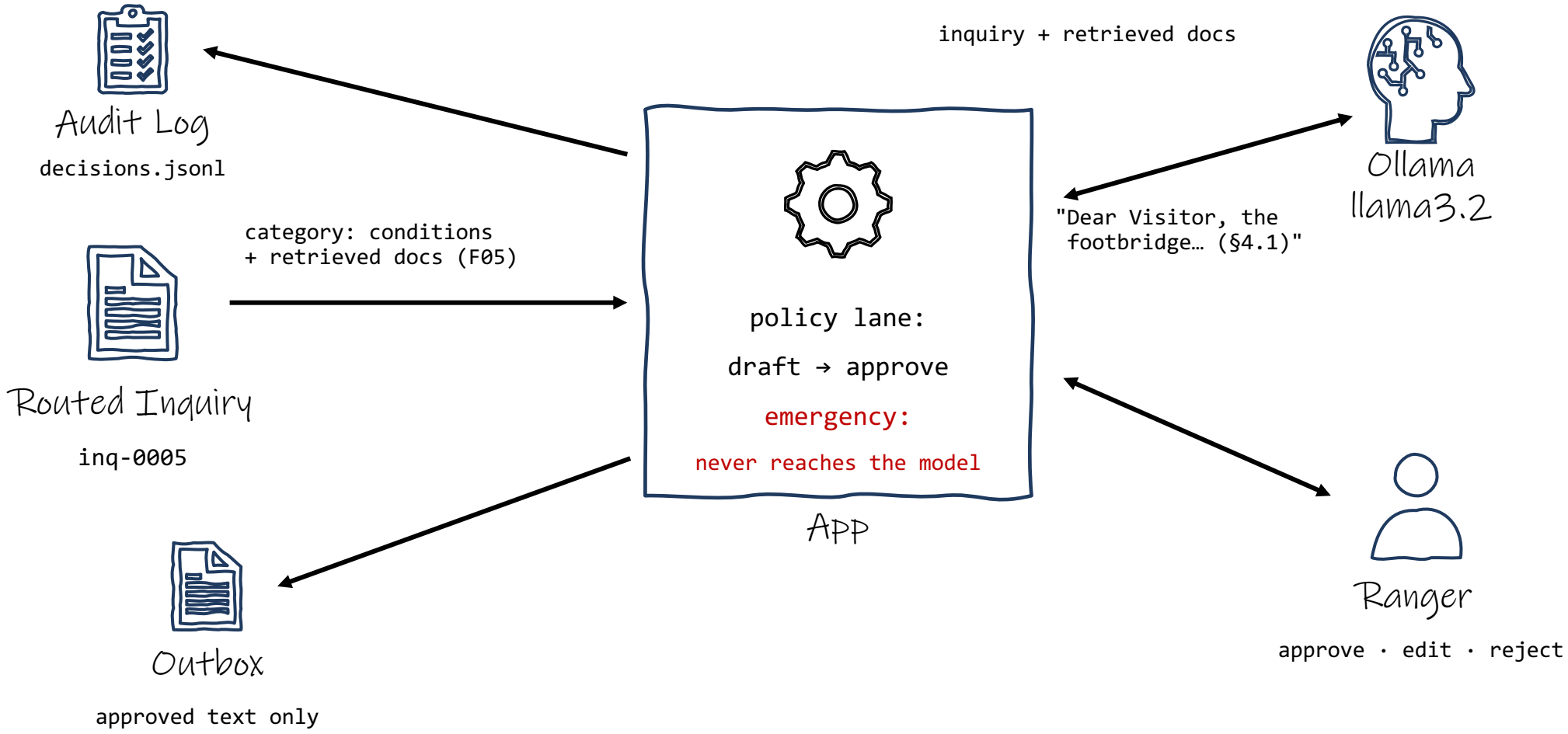
F09 · How It Works



F09 · How It Works



F09 · How It Works



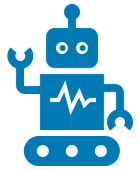


Use it when AI-generated content goes out under your name.

TRAILHEAD
TECHNOLOGY PARTNERS

F10 · Agentic Workflows

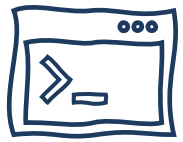
TRAILHEAD
TECHNOLOGY PARTNERS



A model that picks which tools to call,
in a loop, until the job is done.

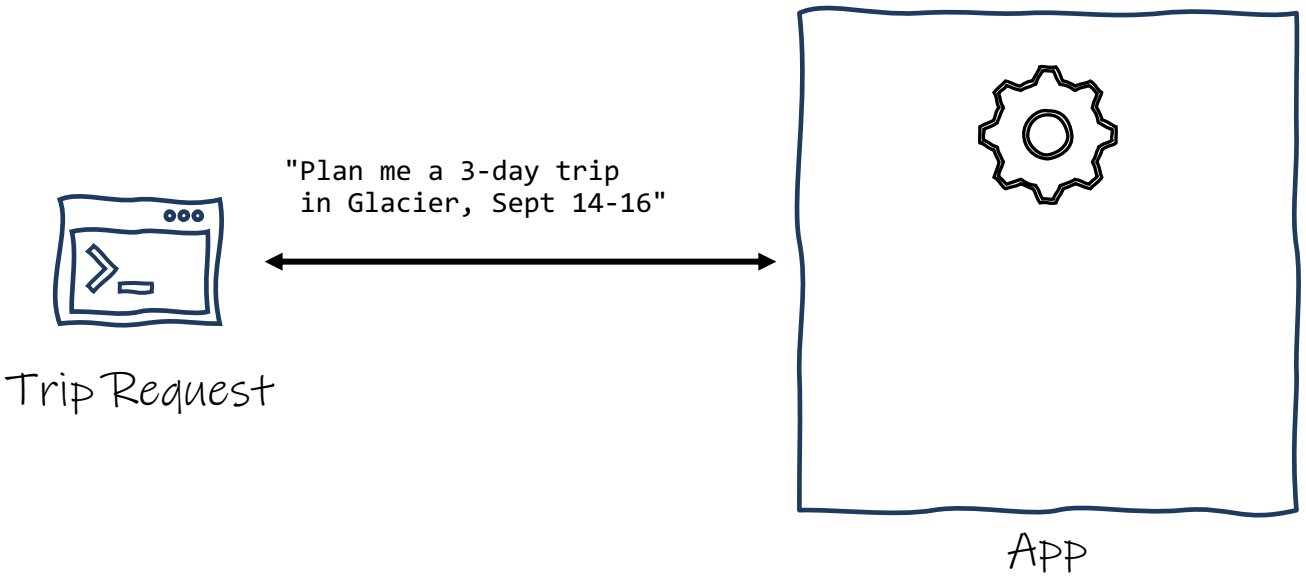
TRAILHEAD
TECHNOLOGY PARTNERS

F10 · How the Loop Works

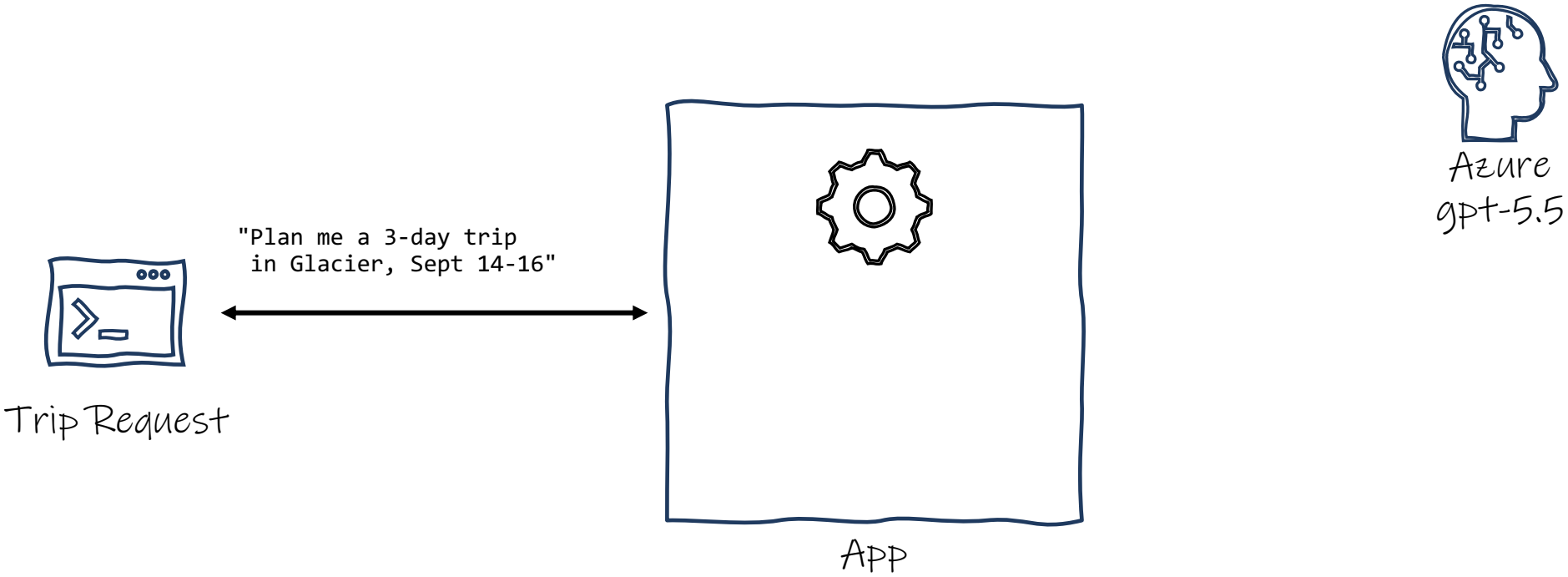


Trip Request

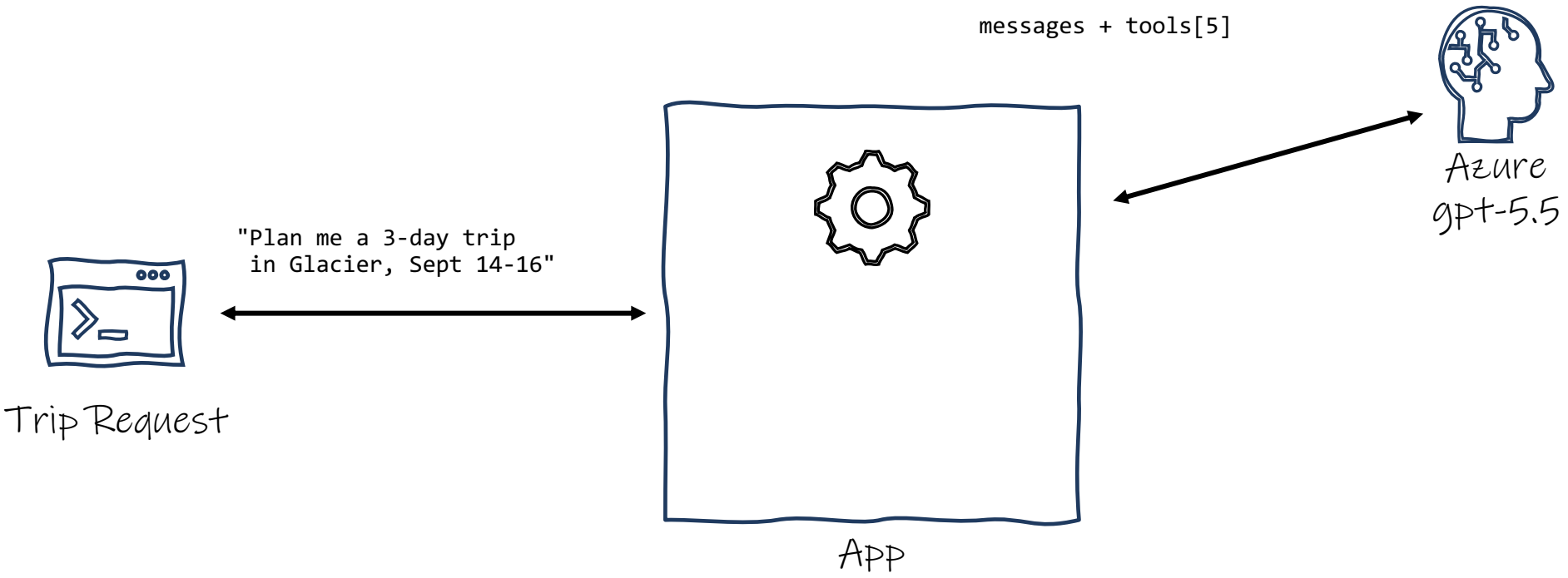
F10 · How the Loop Works



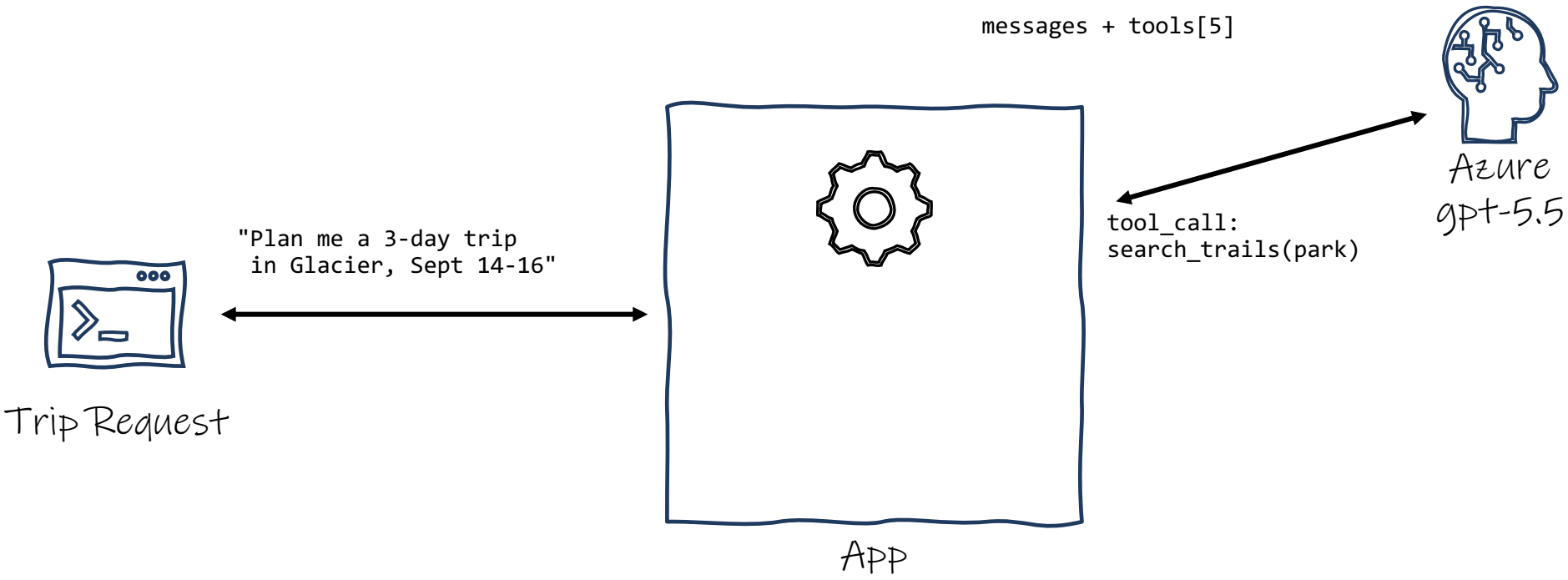
F10 · How the Loop Works



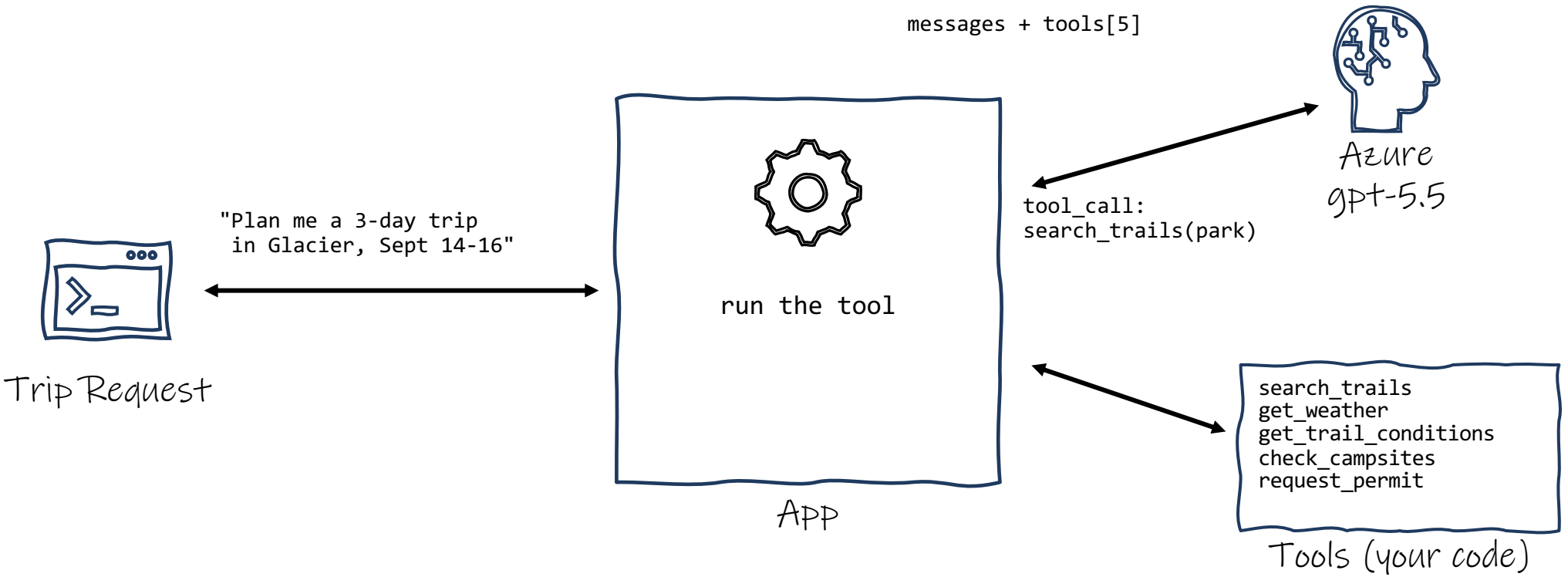
F10 · How the Loop Works



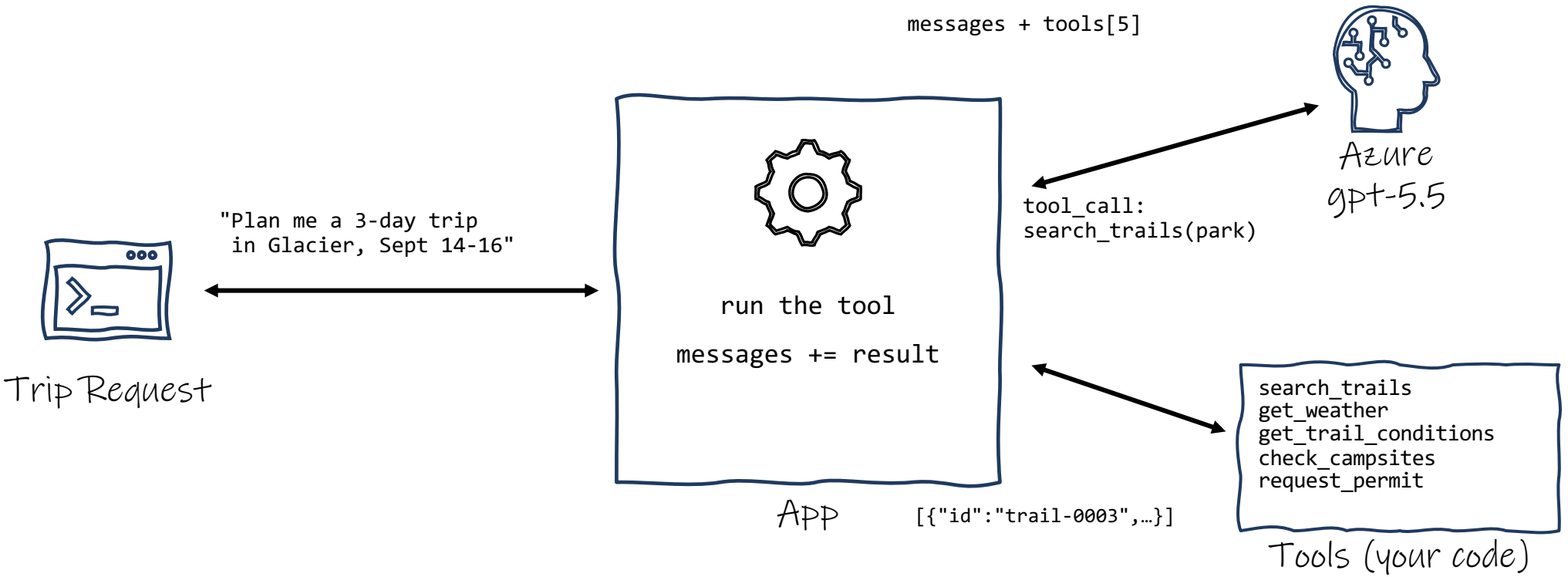
F10 · How the Loop Works



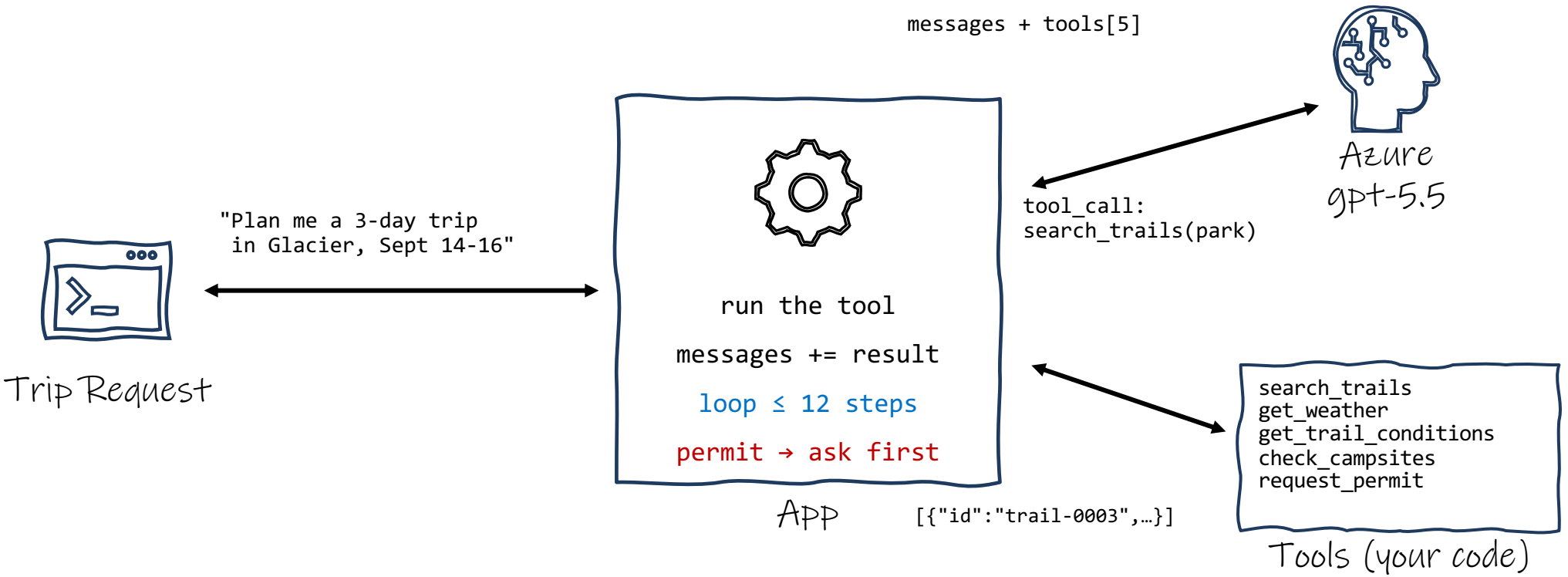
F10 · How the Loop Works



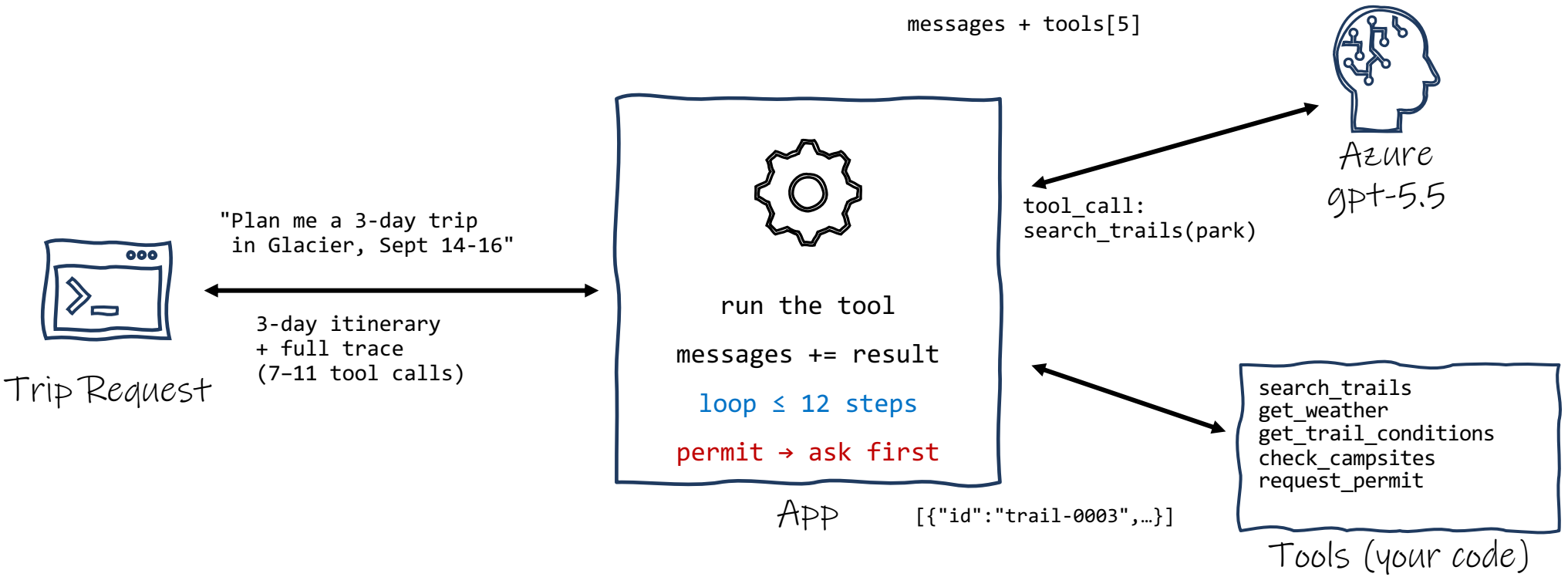
F10 · How the Loop Works



F10 · How the Loop Works



F10 · How the Loop Works



F10 · The Loop Is a Library Call

```
IChatClient client = new ChatClientBuilder(Clients.Cloud("gpt-5.5"))
    // The step budget: the only bound on the loop.
    .UseFunctionInvocation(configure: c => c.MaximumIterationsPerRequest = 12)
    .Build();
var options = new ChatOptions
{
    Tools =
    [
        AIFunctionFactory.Create(SearchTrails),
        AIFunctionFactory.Create(GetWeather),
        AIFunctionFactory.Create(GetTrailConditions),
        AIFunctionFactory.Create(CheckCampsites),
        AIFunctionFactory.Create(RequestPermit),
    ],
};
```

F10 · The Loop Is a Library Call

```
IChatClient client = new ChatClientBuilder(Clients.Cloud("gpt-5.5"))
    // The step budget: the only bound on the loop.
    .UseFunctionInvocation(configure: c => c.MaximumIterationsPerRequest = 12)
    .Build();
var options = new ChatOptions
{
    Tools =
    [
        AIFunctionFactory.Create(SearchTrails),
        AIFunctionFactory.Create(GetWeather),
        AIFunctionFactory.Create(GetTrailConditions),
        AIFunctionFactory.Create(CheckCampsites),
        AIFunctionFactory.Create(RequestPermit),
    ],
};
```

F10 · The Gate

```
[Description("Submit a backcountry permit request. This files a real " +
    "request, so use it once, at the end, after the plan is settled.")]
static string RequestPermit(
    [Description("Park name")] string park,
    [Description("Permit zone, e.g. 'Lake McDonald / Sperry'")] string zone,
    [Description("e.g. '2026-09-14 to 2026-09-16'")] string dates,
    [Description("Number of people")] int groupSize = 2)
{
    // The one irreversible action never runs on the model's say-so.
    if (Gate && !HumanApproves($"File a permit for {zone}, {dates}?"))
        return Result("""{"status": "cancelled", "message":
        "The user declined. Do not retry; say no permit was filed."}""");

    return Result(Permits["submit_response"]!.ToJsonString());
}
```

F10 · The Gate

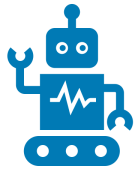
```
[Description("Submit a backcountry permit request. This files a real " +
    "request, so use it once, at the end, after the plan is settled.")]
static string RequestPermit(
    [Description("Park name")] string park,
    [Description("Permit zone, e.g. 'Lake McDonald / Sperry'")] string zone,
    [Description("e.g. '2026-09-14 to 2026-09-16'")] string dates,
    [Description("Number of people")] int groupSize = 2)
{
    // The one irreversible action never runs on the model's say-so.
    if (Gate && !HumanApproves($"File a permit for {zone}, {dates}?"))
        return Result("""{"status": "cancelled", "message":
        "The user declined. Do not retry; say no permit was filed."}""");

    return Result(Permits["submit_response"]!.ToJsonString());
}
```

DEMO · F10 Agentic Workflows



TRAILHEAD
TECHNOLOGY PARTNERS



Use it to give AI a goal and the right tools,
and let it work out the steps.

TRAILHEAD
TECHNOLOGY PARTNERS

Back to the First Slide

❌ "Where can we use AI?"

Back to the First Slide

❌ "Where can we use AI?"

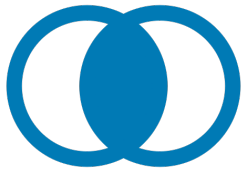
✅ "What problems can AI best solve for our users?"

Your Move

You've Seen the Patterns, Apply it To Your Systems

TRAILHEAD
TECHNOLOGY PARTNERS

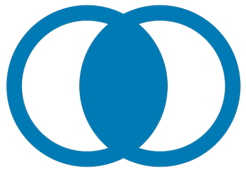
Your Move



Look for the
Overlap

TRAILHEAD
TECHNOLOGY PARTNERS

Your Move



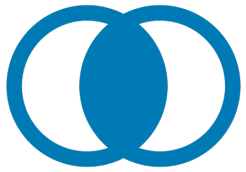
Look for the Overlap



Start Small and Easy

TRAILHEAD
TECHNOLOGY PARTNERS

Your Move



Look for the Overlap



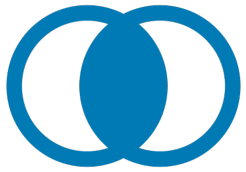
Start Small and Easy



Keep It Safe

TRAILHEAD
TECHNOLOGY PARTNERS

Your Move



Look for the Overlap



Start Small and Easy



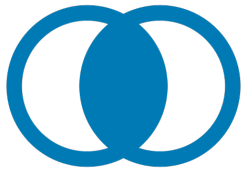
Keep It Safe



Local vs. Cloud

TRAILHEAD
TECHNOLOGY PARTNERS

Your Move



Look for the Overlap



Start Small and Easy



Keep It Safe



Local vs. Cloud



Keep Trying

TRAILHEAD
TECHNOLOGY PARTNERS

Thanks! Questions?

Jonathan "J." Tower

Founding Partner

- 13x Microsoft MVP in .NET & AI
- VP of .NET Foundation
- Blue Blazes podcast host

